The Senior Node.js Engineer's Complete Book

JavaScript, Node.js, and everything an expert backend engineer is expected to know.

About This Book

This is one volume — 15 parts, hundreds of topics — ordered from the easiest foundations of JavaScript up through distributed systems, cloud architecture, and senior-level engineering judgment.

Each part assumes the parts before it. Early parts are things every working engineer touches daily; later parts are what separates senior from mid, and staff from senior.

Two companion books start it off:

- **Book One — JavaScript In Depth** (Parts I–II): the language, front to back, from `typeof` through `Atomics` and realms.
- **Book Two — Node.js In Depth** (Parts III–IV): the runtime — libuv, streams, workers, production patterns.

Then the book widens:

- **Part V** — Asynchronous Programming (cross-cutting across language and runtime).
- **Part VI** — HTTP Frameworks: Express, NestJS, and the deep dive comparing them.
- **Part VII** — API Design: REST and GraphQL.
- **Part VIII** — Security and Authentication.
- **Part IX** — Databases and ORMs, with a dedicated PostgreSQL deep dive.
- **Part X** — System Design and Architecture, including case studies and DDD.
- **Part XI** — Performance, Scalability, and Observability.
- **Part XII** — Distributed Systems and Kafka.

- **Part XIII** — Cloud and AWS.
- **Part XIV** — Testing, DevOps, and Deployment.
- **Part XV** — Coding Challenges, Behavioral Prep, and Final Tips.

Every chapter follows the same shape:

- **What it is** — plain language.
- **Why you care** — the real-world problem it solves.
- **How it works** — a runnable example.
- **Watch out** — the traps that catch people.

Good luck. Read what surprises you slowly.

Book One — JavaScript In Depth

JavaScript In Depth

A field guide for engineers who want to *understand* the language, not just write it.

Preface

Most JavaScript guides teach you how to write code. This one teaches you how the code actually runs — what the engine does, where your mental model breaks, and why the bug you spent two hours on was inevitable given what you believed about the language.

The book is ordered deliberately: every chapter assumes the ones before it. Early chapters are things every JS developer uses daily; later chapters are things only senior engineers reach for, but depend on the earlier material. Skim the parts you know. Slow down when something surprises you — that surprise is the whole point of the book.

Each chapter follows the same shape:

- **What it is** — one or two sentences in plain language.
- **Why you care** — the real-world problem this mechanism solves.
- **How it works** — an example you can run.
- **Watch out** — the trap that catches people.

Let's begin.

Part I — Foundations

Chapter 1. Values and Types

What it is

JavaScript has exactly **eight types**. Seven are *primitives* (immutable, compared by value): `undefined`, `null`, `boolean`, `number`, `bigint`, `string`, `symbol`. The eighth is `object` (mutable, compared by reference). Functions and arrays are objects too.

Why you care

Every bug that surprises you eventually comes back to types — coercion, equality, `typeof` returning `'object'` for `null`, mutation sneaking through a reference. Knowing the type system at the bottom of your stack makes everything above it predictable.

How it works

```
typeof 42;            // 'number'
typeof 42n;           // 'bigint'
typeof 'hi';          // 'string'
typeof true;          // 'boolean'
typeof undefined;     // 'undefined'
typeof null;          // 'object'  ← historical bug, kept for compatibility
typeof Symbol();      // 'symbol'
typeof {};            // 'object'
typeof [];            // 'object'  ← arrays are objects
typeof (() => {});    // 'function' ← functions are callable objects
```

Primitives are copied when assigned. Objects are passed by *reference-value*: the variable holds a pointer.

```
let a = 10;
let b = a;
b = 99;
console.log(a); // 10 — primitives copy

let x = { n: 10 };
let y = x;
y.n = 99;
console.log(x.n); // 99 — same object
```

Watch out

- `typeof null === 'object'` is a famous footgun. Use `x === null` to check.
- `typeof undeclaredVariable` does **not** throw — it returns `'undefined'`. Every other operation on an undeclared variable is a `ReferenceError`.
- There is no "integer" type; `42` and `42.0` are the same float. Use `BigInt` when you need true integers beyond 2^{53}.

Chapter 2. Equality

What it is

JavaScript has four distinct equality algorithms. Knowing which operator uses which is the difference between code that looks right and code that *is* right.

Why you care

`Set`, `Map`, `Array.includes`, `===`, `==`, and `Object.is` do **not** agree on what "equal" means. Using the wrong one — especially when `NaN` or `-0` are in your data — leads to bugs that are impossible to spot by reading.

How it works

Algorithm	Used by	`NaN ≡ NaN`	`+0 ≡ -0`	Coerces types
Strict (`===`)	`===` , `switch`	❌	✅	❌
Abstract (`==`)	`==`	❌	✅	✅
SameValueZero	`Map` , `Set` , `Array.includes`	✅	✅	❌
SameValue	`Object.is`	✅	❌	❌

```
NaN === NaN;                        // false (Strict)
[NaN].includes(NaN);                // true  (SameValueZero)
Object.is(NaN, NaN);                // true  (SameValue)

+0 === -0;                          // true  (Strict)
Object.is(+0, -0);                  // false (SameValue)

'1' == 1;                           // true  (Abstract coerces)
'1' === 1;                          // false (Strict does not)
null == undefined;                  // true  (special case)
null === undefined;                 // false
```

Watch out

- Use `===` by default. Reach for `==` only for the `== null` idiom (matches both `null` and `undefined`).
- Can't find `NaN` in an array with `indexOf` ? Use `includes` — `indexOf` uses Strict, which thinks `NaN !== NaN` .
- `Object.is(+0, -0)` being `false` matters when bookkeeping floating-point sign bits (e.g., `1/+0 === Infinity` , `1/-0 === -Infinity`).

Chapter 3. Scope, Hoisting, and the Temporal Dead Zone

What it is

Scope is where a name is visible. JavaScript has three kinds: global, function, and block. **Hoisting** is the engine's act of reserving those names *before* any line of your code runs. The **Temporal Dead Zone (TDZ)** is the window where a `let` / `const` name exists but is not yet initialized — touching it throws.

Why you care

If you think `let` and `const` "aren't hoisted", you'll be confused when they shadow outer variables from line one. If you think `var` is block-scoped, you'll be confused when it leaks out of an `if`.

How it works

```
// var — function-scoped, hoisted, initialized to undefined
function varDemo() {
  console.log(x); // undefined (not a ReferenceError)
  var x = 1;
}

// let/const — block-scoped, hoisted but uninitialized → TDZ
function letDemo() {
  console.log(y); // ReferenceError: Cannot access 'y' before initialization
  let y = 1;
}

// Function declarations — fully hoisted (definition + name)
hello(); // works
function hello() { console.log('hi'); }

// Function expressions — only the binding is hoisted
// sayHi();              // TypeError: sayHi is not a function
var sayHi = function() {};
```

Watch out

- `var` leaks out of blocks but not functions. `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i))` logs `3, 3, 3` — a single `i` shared. `let` fixes it by creating a fresh binding per iteration.
- Class declarations are in a TDZ too — you cannot use the class before its declaration line.
- Default parameter expressions live in their own scope: `function f(a = b, b = 1) {}` throws on call because `b` is in its TDZ when `a` is evaluated.

Chapter 4. Closures

What it is

A **closure** is a function that remembers the variables from the place where it was *defined*, even when it's later called somewhere else.

Why you care

Closures are how you make private state in JS, how `setTimeout` callbacks remember their context, how React hooks work internally, how module encapsulation works without classes. They also leak memory when misused.

How it works

```
function makeCounter() {
  let count = 0;                        // captured
  return {
    inc: () => ++count,
    get: () => count,
  };
}
const c = makeCounter();
c.inc(); c.inc();
c.get(); // 2

// `count` is not accessible from outside — it's private.
```

The returned functions keep a reference to `makeCounter`'s **lexical environment** on the heap. That environment survives the function return.

Use case: memoization

```
function memoize(fn) {
  const cache = new Map();            // captured — one per memoized fn
  return (arg) => {
    if (!cache.has(arg)) cache.set(arg, fn(arg));
    return cache.get(arg);
  };
}
```

Watch out

- Closures hold entire scopes, not just the variables they reference. If an inner function captures one field of a huge object, V8 often keeps the whole scope alive. In hot paths, destructure just the fields you need into local vars.
- The classic loop bug:

  ```
  for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
  for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
  ```

Chapter 5. The `this` Binding

What it is

`this` is determined by **how a function is called**, not where it's defined — with one exception: arrow functions, which inherit `this` from their enclosing scope.

Why you care

Method references lose `this` when passed as callbacks (`elem.onclick = user.greet` — broken). Understanding the rules lets you pick the right fix (bind, arrow, or explicit wrapping) without cargo-culting.

How it works

In order of precedence:

1. `new` — `new Foo()` → `this` is the new instance.
2. **Explicit** — `fn.call(obj)`, `fn.apply(obj, args)`, `fn.bind(obj)`.
3. **Implicit** — `obj.method()` → `this` is `obj`.
4. **Default** — bare call → `undefined` in strict mode, `globalThis` in sloppy.
5. **Arrow** — lexical `this`; `call` / `apply` / `bind` cannot change it.

```
const user = {
  name: 'Alice',
  greet() { return `Hi, ${this.name}`; },
};
user.greet();                          // 'Hi, Alice'

const g = user.greet;
g();                                   // 'Hi, undefined' (sloppy) or TypeError

const bound = user.greet.bind(user);
bound();                               // 'Hi, Alice'
```

Use case: event handlers in classes

```
class Button {
  label = 'Click';
  // Arrow field → `this` is captured at instance creation
  onClick = () => console.log(this.label);
}
// new Button().onClick keeps `this` correctly even when passed elsewhere.
```

Watch out

- Arrow functions in object literals don't "inherit from the object" — there's no enclosing method. `this` in `{ fn: () => this }` is whatever surrounds the *literal*.
- `bind` returns a **new function**. Binding twice doesn't re-bind; the first wins.

Chapter 6. Coercion

What it is

Coercion is JavaScript silently converting a value to another type when an operator or API demands one. It follows a small algorithm (`ToPrimitive` → `ToNumber` / `ToString`) with surprising corners.

Why you care

Every "weird JavaScript" joke you've seen is coercion. `[] + [] === ''` , `[] + {} === '[object Object]'` , `{} + [] === 0` — all follow from one rule applied consistently. Once you know the rule, the jokes stop being confusing.

How it works

When an object needs to become a primitive, JS calls `[Symbol.toPrimitive](hint)` with `'number'` , `'string'` , or `'default'` . If that's missing, it falls back to `valueOf()` then `toString()` (or `toString()` first for hint `'string'`).

```
const money = {
  amount: 42,
  [Symbol.toPrimitive](hint) {
    if (hint === 'number') return this.amount;
    if (hint === 'string') return `$${this.amount}`;
    return `$${this.amount}`;         // default
  },
};

+money;         // 42   (hint 'number')
`${money}`;     // '$42' (hint 'string')
money + '';     // '$42' (hint 'default')
```

Use case: Date subtraction

```
const t1 = new Date();
// ...
const t2 = new Date();
const ms = t2 - t1;                    // Date's valueOf → timestamp
```

Date implements `valueOf` to return `Date.now()`, so subtraction Just Works.

Watch out

- `+` with any string coerces the other side to string: `1 + '2' === '12'`.
- `-`, `*`, `/`, `%` always coerce to numbers: `'10' - 1 === 9`.
- `==` with `{}` or `[]` triggers full coercion and often lies. Stick to `===`.

Part II — Objects

Chapter 7. Property Descriptors

What it is

Every property on an object isn't just a key/value — it has a **descriptor** controlling whether it's writable, enumerable, configurable, and whether it's a data property or an accessor (getter/setter).

Why you care

Libraries that freeze objects, browsers that lock `window` APIs, proxies that enforce invariants — all of it is descriptors. Understanding them lets you build safe APIs and explains why `Object.freeze` survives tampering.

How it works

```
const user = {};
Object.defineProperty(user, 'id', {
  value: 1,
  writable: false,                    // can't reassign
  enumerable: false,                  // hidden from for..in / Object.keys
  configurable: false,                // can't delete or reconfigure
});

user.id = 99;                         // silently ignored (sloppy) / throws (st
Object.keys(user);                    // []  — id is non-enumerable
delete user.id;                       // false; still there
```

Descriptors come in two flavors:

- **Data** — `value`, `writable`.
- **Accessor** — `get`, `set`.

```
const temp = {
  _c: 0,
  get celsius() { return this._c; },
  set celsius(v) { this._c = v; },
  get fahrenheit() { return this._c * 9/5 + 32; },
};
```

Use case: read-only API surface

```
function defineConst(obj, key, value) {
  Object.defineProperty(obj, key, { value, writable: false, configurable: false
}
```

Watch out

- Properties created by assignment (`obj.x = 1`) default to `{ writable: true, enumerable: true, configurable: true }` .
- Properties defined by class fields are `{ writable: true, enumerable: true, configurable: true }` — not hidden, not locked.
- `Object.assign` only copies **enumerable own** properties; it skips getters (it invokes them and copies the value).

Chapter 8. Getters, Setters, and Object.defineProperty

What it is

A property can be backed by functions instead of a stored value. Accessing the property runs the getter; assigning runs the setter.

Why you care

Getters give you "computed" properties that look like fields. Setters let you run validation or trigger effects on assignment — the heart of reactive frameworks like Vue 2 (before Proxy) and Svelte.

How it works

```
const person = {
  first: 'Ada',
  last: 'Lovelace',
  get full() { return `${this.first} ${this.last}`; },
  set full(v) {
    [this.first, this.last] = v.split(' ');
  },
};

person.full;                          // 'Ada Lovelace'
person.full = 'Grace Hopper';
person.first;                         // 'Grace'
```

Internally these are accessor descriptors:

```
Object.defineProperty(person, 'initials', {
  get() { return this.first[0] + this.last[0]; },
  enumerable: true,
});
```

Use case: lazy computation

```
const config = {
  get heavyThing() {
    const value = loadSlow();
    Object.defineProperty(this, 'heavyThing', { value }); // replace with data
    return value;
  },
};
```

First access computes and caches; subsequent accesses are plain property loads.

Watch out

- Getters run every time — don't do expensive work in them without memoizing.
- A setter without a getter makes reads return `undefined`, not throw. Silent bugs follow.

- Getters on prototypes see `this` as the *instance*, which is what you want — don't re-create them per instance unless they capture instance-specific closures.

Chapter 9. Prototypes and Inheritance

What it is

Every object has an internal link (`[[Prototype]]`) to another object. When you read a property the engine can't find locally, it walks the chain until it hits `null`.

Why you care

`class` is syntactic sugar over this one mechanism. Understanding the chain tells you why `instanceof` sometimes lies across iframes, why adding methods to `Array.prototype` is a terrible idea, and how `Object.create(null)` makes a safe map.

How it works

```
const animal = { eats: true };
const dog = Object.create(animal);
dog.barks = true;

dog.barks;                            // true (own)
dog.eats;                             // true (from prototype)
Object.getPrototypeOf(dog) === animal; // true

// class is the same thing with syntax
class Animal { eat() {} }
class Dog extends Animal { bark() {} }
const d = new Dog();
Object.getPrototypeOf(d) === Dog.prototype;               // true
Object.getPrototypeOf(Dog.prototype) === Animal.prototype; // true
```

Use case: safe dictionaries

`{}` inherits from `Object.prototype`, which has `hasOwnProperty`, `toString`, etc. If you store user input as keys, a key named `'toString'` collides with the inherited one.

```
const map = Object.create(null);      // no prototype — safe for arbitrary keys
map.toString = 'hello';
map.toString;                         // 'hello', not the function
```

Watch out

- Do not mutate built-in prototypes (`Array.prototype.myFn = ...`). It changes every array in the runtime and breaks libraries.
- `hasOwnProperty` on `Object.create(null)` doesn't exist. Use `Object.hasOwn(obj, key)` (ES2022) for safety.

Chapter 10. Freeze, Seal, and Preventing Extensions

What it is

Three levels of object locking: `Object.preventExtensions` (no new props), `Object.seal` (no new props + existing non-configurable), `Object.freeze` (no new, no change, no delete).

Why you care

Redux used to freeze state in dev to catch accidental mutations. Config objects you export from a library should be frozen so consumers can't mess with each other. Frozen objects also unlock minor V8 optimizations.

How it works

```
const cfg = Object.freeze({ host: 'localhost', port: 5432 });
cfg.port = 8080;                      // silently ignored (sloppy); TypeError (
cfg.user = 'admin';                   // same
delete cfg.host;                      // same
Object.isFrozen(cfg);                 // true
```

Use case: shallow immutability

```
export const LEVELS = Object.freeze(['debug', 'info', 'warn', 'error']);
```

Watch out

- `freeze` is **shallow**. Nested objects are still mutable:

  ```
  const o = Object.freeze({ inner: { n: 1 } });
  o.inner.n = 99;                      // works
  ```

  Use a recursive deep-freeze helper for true immutability.
- Freezing is one-way — there is no `unfreeze`.

Chapter 11. Symbols

What it is

A `symbol` is a primitive whose only guarantee is **uniqueness**. Two symbols with the same description are not equal.

Why you care

Symbols let libraries add metadata to user objects without name collisions. They're also how the language defines its own protocols (iteration, coercion, instance checks) — *well-known symbols*.

How it works

```
const id = Symbol('id');              // description is just a label
const id2 = Symbol('id');
id === id2;                           // false — always unique

const user = { name: 'Alice', [id]: 123 };
user[id];                             // 123
Object.keys(user);                    // ['name'] — symbols are hidden
Object.getOwnPropertySymbols(user);   // [Symbol(id)]
```

Use case: library-private state on public objects

```
const STATE = Symbol('libX.state');

export function attach(target) {
  target[STATE] = { seen: new Set() };
}
export function hasSeen(target, x) {
  return target[STATE]?.seen.has(x);
}
```

User code can't collide with `STATE` because they can't name it.

Watch out

- `Symbol()` is not a constructor — don't write `new Symbol()`.
- Symbols are hidden from `JSON.stringify`, `for..in`, and `Object.keys`. Great for privacy, confusing when debugging.
- `Symbol.for('k')` uses a **global registry** — same key returns the same symbol process-wide. Use it only when you explicitly want cross-realm shared symbols.

Chapter 12. Well-Known Symbols

What it is

Built-in symbols the language itself looks up on your objects to customize behavior. They're how you teach a class to be iterable, stringify a certain way, or be `instanceof`-compatible.

Why you care

You want your custom collection to work with `for..of` and spread? Implement `Symbol.iterator`. You want `String(myObj)` to print nicely? Use `Symbol.toStringTag`. Meta-programming starts here.

The catalogue

- `Symbol.iterator` — makes an object iterable (`for..of`, spread).
- `Symbol.asyncIterator` — `for await..of`.
- `Symbol.toPrimitive` — customize coercion (Chapter 6).
- `Symbol.toStringTag` — override the `[object X]` tag used by `Object.prototype.toString`.
- `Symbol.hasInstance` — override `instanceof`.
- `Symbol.isConcatSpreadable` — whether `Array.concat` flattens this object.
- `Symbol.species` — subclass-aware return types.
- `Symbol.matchAll`, `Symbol.replace`, etc. — hooks for string methods.

Example: custom `instanceof`

```
class Even {
  static [Symbol.hasInstance](n) {
    return typeof n === 'number' && n % 2 === 0;
  }
}
4 instanceof Even;                    // true
5 instanceof Even;                    // false
```

Example: custom `toString` tag

```
class Money {
  get [Symbol.toStringTag]() { return 'Money'; }
}
Object.prototype.toString.call(new Money()); // '[object Money]'
```

Watch out

- Overriding `Symbol.hasInstance` is a great way to confuse everyone reading your code. Use it for DSLs or matchers, not everyday classes.

Part III — Iteration and Functions

Chapter 13. Iterators and Iterables

What it is

An **iterable** is any object with a `Symbol.iterator` method that returns an **iterator**: an object with a `next()` method returning `{ value, done }`. That's the entire protocol.

Why you care

`for..of`, the spread operator (`...`), destructuring, `Array.from`, `new Map(iterable)` — all use this one protocol. Once you implement it, your type plugs into all of them for free.

How it works

```
const range = {
  from: 1,
  to: 3,
  [Symbol.iterator]() {
    let i = this.from, end = this.to;
    return {
      next: () => i <= end ? { value: i++, done: false } : { value: undefined,
    };
  },
};

for (const n of range) console.log(n); // 1 2 3
[...range];                            // [1, 2, 3]
Array.from(range, x => x * 10);        // [10, 20, 30]
```

Use case: paginate a REST API

```
async function* pages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield res.items;
    next = res.nextPage;
  }
}
for await (const batch of pages('/api/users')) process(batch);
```

Watch out

- Strings are iterable **by code point**, not code unit — so surrogate pairs (emoji) don't split: `[...'👩‍❤️‍💋‍👩']` returns the ZWJ sequence elements correctly (still more than one because of the joiner).
- Plain objects (`{}`) are **not** iterable. Use `Object.entries(obj)` or define `Symbol.iterator`.
- Iterators are usually **one-shot**. You can't `for..of` the same iterator twice — iterables are what you re-iterate.

Chapter 14. Generators

What it is

A function declared with `function*` that pauses at each `yield` and resumes when the consumer calls `next()`. The engine builds the iterator for you.

Why you care

Anything that produces a sequence lazily — infinite streams, pagination, tree traversal, state machines — reads more naturally as a generator than as a handwritten iterator.

How it works

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) { yield a; [a, b] = [b, a + b]; }
}

const g = fib();
g.next().value; // 0
g.next().value; // 1
g.next().value; // 1
g.next().value; // 2
```

Generators can also **receive** values via `next(x)`:

```
function* interview() {
  const name = yield 'What is your name?';
  const age  = yield `Hi ${name}, how old are you?`;
  return `${name} is ${age}`;
}

const it = interview();
it.next();                          // { value: 'What is your name?', done: f
it.next('Alice');                   // { value: 'Hi Alice, how old are you?',
it.next(30);                        // { value: 'Alice is 30', done: true }
```

Use case: tree traversal

```
function* walk(node) {
  yield node.value;
  for (const child of node.children) yield* walk(child);
}
```

Watch out

- `return()` and `throw()` are part of the iterator protocol — `for..of` uses them to clean up on `break` or a thrown error. If you write resources (file handles), use `try/finally` inside the generator so cleanup runs.

- Generators are not optimized as aggressively as regular functions. In hot paths, a plain iterator object is sometimes faster.

Chapter 15. Async Iteration

What it is

`Symbol.asyncIterator` returns an iterator whose `next()` returns a **Promise** for `{ value, done }`. `for await..of` consumes it.

Why you care

Streaming: reading a file chunk by chunk, paging through an API, consuming a Kafka topic. You want backpressure (only ask for the next chunk when you're ready) and clean error handling — async iteration gives you both.

How it works

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

async function* lines(path) {
  const rl = createInterface({ input: createReadStream(path) });
  for await (const line of rl) yield line;
}

for await (const line of lines('big.log')) {
  if (line.includes('ERROR')) console.log(line);
}
```

Use case: stream to DB

```
async function ingest(streamOfEvents) {
  for await (const batch of streamOfEvents) {
    await db.insertMany(batch);       // natural backpressure — next batch wait
  }
}
```

Watch out

- Use `for await..of` only inside `async` functions.
- Errors thrown inside the iterator bubble to the `for await..of` site — wrap in `try/catch`.
- Remember to `return` early if you break out mid-stream, or the source may leak a file descriptor / socket.

Part IV — Asynchronicity

Chapter 16. The Event Loop, from JavaScript's Side

What it is

JavaScript runs on **one thread** with a queue of tasks. The event loop pulls one task at a time, runs it to completion, then drains the **microtask queue**, then repeats.

Why you care

Every async bug — "why did this log before that?", "why is my UI frozen?", "why is this promise never resolving?" — comes back to the loop. Understanding the phases lets you predict order without running the code.

How it works (simplified)

```
┌──────────────┐
│  next task   │  (from the task queue: I/O callbacks, setTimeout, UI events)
└──────┬───────┘
       ↓ run to completion

┌──────────────┐
│  microtasks  │  drain ALL queued microtasks: promise reactions, queueMicrotas
└──────┬───────┘
       ↓ repeat
```

```
console.log('1');
setTimeout(() => console.log('4'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('2');

// 1, 2, 3, 4
// Script is one task. Microtasks (promise) drain before the next task (timeout
```

Use case: avoiding UI jank

Splitting a long computation into smaller chunks, yielding with `await null` or `setTimeout(fn, 0)` between them, lets the browser paint between chunks.

Watch out

- `process.nextTick` in Node is **even higher priority** than microtasks. Abusing it can starve I/O.
- A tight microtask chain (`p.then(p2).then(p3)...` recursively) can block the task queue indefinitely.

Chapter 17. Promises Internally

What it is

A Promise is a state machine with three states — **pending**, **fulfilled**, **rejected** — and a FIFO list of reactions. When it settles, every reaction is scheduled as a microtask.

Why you care

Misunderstanding Promises causes the worst async bugs: silent rejections, forgotten `await`, double `resolve` calls that do nothing, and the "promise returned from a then that returns a promise" chain that people find magical.

How it works

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve(42), 100);
});
p.then(x => x + 1)                    // returns a new promise for 43
 .then(x => Promise.resolve(x * 2))   // chain adopts — unwraps nested promise
 .then(console.log);                  // 86
```

Three rules of chaining:

1. `.then(fn)` returns a new promise for `fn`'s return value.

2. If `fn` returns a thenable, the new promise **adopts** it (unwraps one level automatically).
3. A synchronous `throw` in `fn` becomes a rejection.

Use case: parallel with `Promise.all`

```
const [user, orders, prefs] = await Promise.all([
  fetchUser(),
  fetchOrders(),
  fetchPrefs(),
]);
```

Variants: `Promise.allSettled` (no fail-fast), `Promise.race` (first to settle), `Promise.any` (first to fulfill).

Watch out

- A **forgotten** `await` returns a promise to a caller who treats it as the value. Enable lint rules (`no-floating-promises`) in TypeScript projects.
- **Double-settling** is a no-op — `resolve(1); resolve(2)` settles with 1 and ignores the second call.
- An unhandled rejection fires a `process` / `window` event; in recent Node versions it crashes the process by default.

Chapter 18. async/await

What it is

Syntactic sugar over Promises + generators. An `async` function always returns a Promise; `await` pauses the function until its Promise settles and resumes with the value (or throws the rejection).

Why you care

It makes async code read like sync code, but the mental model is different: every `await` is a microtask boundary, the stack unwinds, and errors need `try/catch` — not callbacks.

How it works

```
async function getUser(id) {
  try {
    const res = await fetch(`/u/${id}`);
    if (!res.ok) throw new Error(`${res.status}`);
    return await res.json();
  } catch (err) {
    logger.error({ err, id });
    throw err;                        // propagate
  }
}
```

Under the hood, the function is paused at each `await` — its state (local vars, pending expressions) is stored on the heap, and the runtime schedules resumption when the awaited promise settles.

Use case: sequential when needed, parallel when possible

```
// Sequential — each depends on the previous
const user = await getUser(id);
const orders = await getOrders(user.id);

// Parallel — independent
const [a, b] = await Promise.all([fa(), fb()]);
```

Watch out

- `await` inside a loop serializes — use `Promise.all(items.map(fn))` when the calls are independent.
- `return somePromise` vs `return await somePromise` differ **only** inside `try/catch`: without `await`, the rejection happens after the frame returns, so the `catch` doesn't fire.
- Top-level `await` is supported in ESM only.

Chapter 19. Microtasks vs Macrotasks

What it is

Two queues the event loop drains at different cadence. **Macrotasks** (setTimeout, I/O callbacks, UI events): one per loop turn. **Microtasks** (Promise reactions, `queueMicrotask`, `process.nextTick`): drained completely between macrotasks.

Why you care

Order-of-operations bugs. "Why did the error handler fire before the timeout I set first?" Because the handler is a microtask and the timeout is a macrotask.

How it works

```
setTimeout(() => console.log('timeout'), 0);   // macrotask
Promise.resolve().then(() => console.log('p'));// microtask
queueMicrotask(() => console.log('qm'));       // microtask
console.log('sync');

// sync, p, qm, timeout
```

Use case: defer until current work finishes without yielding to I/O

```
function scheduleFlush() {
  queueMicrotask(flush);              // runs before any timer or I/O
}
```

Watch out

- Starvation: recursive `queueMicrotask` / `process.nextTick` prevents timers and I/O from ever running.
- In browsers, the microtask queue is drained in the same loop step; in Node, `process.nextTick` runs **before** regular microtasks.

Part V — Errors

Chapter 20. Errors, Causes, and Aggregates

What it is

`Error` is a built-in class with `name`, `message`, and (non-standard but universal) `stack`. Modern specs added `cause` (wrap an original error) and `AggregateError` (multiple errors as one).

Why you care

Good error handling is half of senior engineering. Wrapping errors without losing context, distinguishing operational from programmer errors, and presenting root causes to logs are all table stakes.

How it works

```
class NotFoundError extends Error {
  constructor(msg, options) {
    super(msg, options);              // options.cause is stored automatically
    this.name = 'NotFoundError';
  }
}

try {
  fetchUser(id);
} catch (err) {
  throw new NotFoundError(`User ${id} missing`, { cause: err });
}
```

`AggregateError` is what `Promise.any` throws when *all* inputs reject:

```
try {
  await Promise.any([Promise.reject(1), Promise.reject(2)]);
} catch (e) {
  e instanceof AggregateError;        // true
  e.errors;                           // [1, 2]
}
```

Use case: typed domain errors

```
class ValidationError extends Error { constructor(issues) { super('Invalid inpu

// Middleware handler:
if (err instanceof ValidationError) return res.status(400).json({ issues: err.i
```

Watch out

- **Custom Error classes lose `instanceof` across realms** (iframes, workers). Use a sentinel property (`err.code === 'E_NOT_FOUND'`) for cross-realm checks.
- `Error.captureStackTrace(obj, constructor)` (V8 only) trims the constructor from the stack — useful in custom error hierarchies.
- Always include the original error via `cause` when wrapping — losing the root cause costs hours during incidents.

Part VI — Meta-Programming

Chapter 21. Proxy and Reflect

What it is

A `Proxy` wraps a target object and lets you intercept fundamental operations (read, write, delete, enumerate, call, construct) via **traps**. `Reflect` mirrors those operations as plain functions — use it to forward from a trap to the default behavior.

Why you care

Reactive frameworks (Vue 3), ORMs (Sequelize), mocks in tests, dynamic APIs, guard rails in development — all built on Proxy. Replacing `Object.defineProperty` approaches because Proxy is deep (handles new keys, delete, array length).

How it works

```
const target = { a: 1, b: 2 };
const logged = new Proxy(target, {
  get(t, prop, receiver) {
    console.log('get', prop);
    return Reflect.get(t, prop, receiver);
  },
  set(t, prop, value, receiver) {
    console.log('set', prop, value);
    return Reflect.set(t, prop, value, receiver);
  },
});

logged.a;                              // get a → 1
logged.a = 10;                         // set a 10
```

Traps you'll actually use: `get`, `set`, `has`, `deleteProperty`, `ownKeys`, `apply`, `construct`.

Use case: reactive state (Vue-style)

```
function reactive(obj, onChange) {
  return new Proxy(obj, {
    set(t, k, v) { t[k] = v; onChange(k, v); return true; },
  });
}

const state = reactive({ count: 0 }, (k, v) => console.log(`${k} = ${v}`));
state.count = 5;                       // logs "count = 5"
```

Watch out

- Proxies must respect **invariants**: a non-configurable, non-writable data property must return the same value from a `get` trap. Breaking an invariant throws a `TypeError`.
- Proxies are not transparent to native code: `map.set(proxy, 1)` uses the proxy as the key, not the target. Frameworks that compare identity can get confused.
- Performance: a proxy is ~10× slower than a raw access. Wrap at boundaries, not in hot loops.

Part VII — Classes

Chapter 22. Classes, Fields, and Private Members

What it is

`class` is sugar over prototypes, but modern JS adds genuinely new features on top: public fields (`x = 1`), private fields (`#x = 1`), private methods (`#fn()`), and static blocks.

Why you care

`#private` is enforced by the engine — it's actually private, unlike TypeScript's `private` which is compile-time only. Static blocks let you do complex static initialization without IIFEs or factory functions.

How it works

```
class Counter {
  #count = 0;                          // truly private
  static #instances = new Map();       // private static
  static {                             // static initialization block
    Counter.#instances.set('default', new Counter());
  }

  inc() { return ++this.#count; }
  static get(key) { return Counter.#instances.get(key); }
}

const c = new Counter();
c.inc();                              // 1
c.#count;                             // SyntaxError at parse time
```

Use case: branded checks

```
class Branded {
  #brand;
  static isBranded(x) { return #brand in x; }  // true only for real instances
}
```

Useful for distinguishing "real instance of my class" from "looks-like-a-duck".

Watch out

- Arrow methods as class fields (`onClick = () => {}`) are **per-instance** — they keep `this`, but every instance allocates a new function. Fine for event handlers on small numbers of instances; wasteful on millions.
- Class declarations are **in the TDZ** — you can't reference the class before its declaration.
- Field initializers see `this` as the instance under construction, but **not** derived fields not yet declared.

Part VIII — Modules

Chapter 23. ESM vs CommonJS

What it is

Two module systems sharing the ecosystem: the older **CommonJS** (`require` , `module.exports`) that powered Node for a decade, and the standard **ECMAScript Modules** (`import` , `export`).

Why you care

Most pain in Node tooling traces back to these two coexisting. Package authors, bundler config, `.cjs` / `.mjs` extensions, `"type": "module"` in `package.json` , default export shape — all downstream of this split.

Key differences

	CommonJS	ESM
Loader	Synchronous, on demand	Asynchronous, parsed before execution
Bindings	**Copy** of exports at `require` time	**Live** bindings (reads see current value)
`this` at top level	`module.exports`	`undefined`
Circular deps	Returns the partial `exports` at the cycle point	Works via live bindings; TDZ errors if used too early
Top-level `await`	Not supported	Supported
File extensions	Optional	Required in specifiers

How it works

```
// ESM
export const x = 1;
export default function fn() {}

import fn, { x } from './mod.js';
import * as mod from './mod.js';
```

```
// CommonJS
module.exports = { x: 1, fn: () => {} };
const { x, fn } = require('./mod');
```

Use case: the "default export shape" trap

```
// CJS module
module.exports = { a: 1 };

// ESM consumer
import mod from './x.cjs';   // mod is { a: 1 }

// ESM module
export default { a: 1 };

// CJS consumer
const mod = require('./x.mjs');  // mod is { default: { a: 1 } }  ← wrapper!
```

Watch out

- `require()` of an ESM module was a hard error until recent Node. Even today, prefer `await import()`.
- Some packages ship dual (`main` + `module` + `exports`) — misconfigure and consumers pick the wrong file. Validate with tools like `@arethetypeswrong/cli`.

Part IX — Primitives and Data

Chapter 24. Numbers and IEEE 754

What it is

All `number` values in JS are 64-bit IEEE 754 floats. There is no separate integer type. This explains `0.1 + 0.2 !== 0.3` and every other "float weirdness" question.

Why you care

If you handle money, timestamps, IDs, or scientific data, you *must* know where precision ends and how it fails.

How it works

```
Number.MAX_SAFE_INTEGER;              // 9007199254740991 (2^53 - 1)
9007199254740993 === 9007199254740992; // true — same representable float
0.1 + 0.2;                            // 0.30000000000000004
0.1 + 0.2 === 0.3;                    // false
Math.abs((0.1 + 0.2) - 0.3) < 1e-10;  // true — compare with a tolerance
```

Use case: money in cents

```
// Store amounts as integer minor units
const cents = 1099;                   // $10.99
const total = cents * quantity;       // exact
function format(c) { return `$${(c / 100).toFixed(2)}`; }
```

Never do float arithmetic on currency.

Watch out

- `JSON.parse('{"id":9999999999999999}')` produces `10000000000000000` — the parser snaps to the nearest float. Use strings for IDs > 2^{53}.
- `parseInt('08')` used to return `0` in ancient browsers because of octal interpretation. Always pass a radix: `parseInt(s, 10)`.

Chapter 25. BigInt

What it is

An integer primitive with arbitrary precision. Created with the `n` suffix (`10n`) or `BigInt(x)`.

Why you care

IDs > 2^{53}, cryptographic arithmetic, exact math — anywhere a 64-bit float lies, BigInt tells the truth.

How it works

```
const big = 9007199254740993n;
big + 1n;                              // 9007199254740994n — exact
big > 2n ** 60n;                       // false

// Mixing with Number throws
1n + 1;                                // TypeError
Number(big);                           // back to lossy float
```

Use case: Snowflake / Twitter IDs

Twitter IDs are 64-bit. Parsing as `Number` loses the bottom bits.

```
const id = BigInt(json.id_str);       // always safe
```

Watch out

- JSON doesn't serialize BigInt. Use `id.toString()` or a custom replacer/reviver.
- `Math.*` doesn't accept BigInt — use `bigint`-specific libraries for sqrt/abs etc.
- Performance: slower than Number. Don't use for counters unless you need the range.

Chapter 26. Strings and Unicode

What it is

JS strings are sequences of **UTF-16 code units**. Most characters are one unit; some (emoji, rare CJK) are two ("surrogate pairs"). Unicode code points are a separate, logical layer.

Why you care

Slicing by index, counting `.length`, iterating with a `for` loop — all operate on **code units**, and can split a character in half. Users see garbage; you don't see the bug until an emoji shows up.

How it works

```
const s = '👍';
s.length;                              // 2 — two code units (surrogate pair)
s[0];                                  // '\uD83D' — half a character
[...s].length;                         // 1 — spread uses the iterator (code poi
Array.from(s);                         // ['👍']
```

Use case: safe truncation

```
function truncate(str, maxPoints) {
  const pts = [...str];                // by code point
  return pts.length > maxPoints ? pts.slice(0, maxPoints).join('') + '…' : str;
}
```

Watch out

- Regex `.` without `/u` flag matches one code unit — also splits emoji. Always use `/u` for user-visible text.
- "Grapheme clusters" (e.g., 👨‍👩‍👦 = man + ZWJ + woman + ZWJ + boy) are not code points either. Use `Intl.Segmenter` for true user-perceived characters.

Chapter 27. Regular Expressions

What it is

A mini-language for pattern matching over strings, exposed as `RegExp` literals (`/pattern/flags`) or `new RegExp(str, flags)` . Flags change behavior: `g` (global), `i` (case-insensitive), `m` (multiline), `u` (unicode), `s` (dotall), `y` (sticky), `d` (indices).

Why you care

The wrong regex has hung more production servers than most bugs. Understanding flags and lookbehind/named groups saves you from writing 300-line parsers by hand.

How it works

```
// Named capture groups
'2026-04-21'.match(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/).groups;
// { y: '2026', m: '04', d: '21' }

// Lookbehind
'price: $42'.match(/(?<=\$)\d+/);     // ['42']

// matchAll — iterate all matches with groups
for (const m of 'a1 b2'.matchAll(/(?<letter>\w)(?<num>\d)/g)) {
  console.log(m.groups);              // { letter: 'a', num: '1' }, then { lett
}

// Sticky (y) — anchor at lastIndex, never scan forward
const re = /\d+/y;
re.lastIndex = 2;
re.exec('ab123');                     // null — no match AT 2
```

Use case: tokenizer

```
function* tokens(input) {
  const patterns = [
    ['num', /\d+/y],
    ['ws',  /\s+/y],
    ['op',  /[+\-*/]/y],
  ];
  let i = 0;
  while (i < input.length) {
    for (const [name, re] of patterns) {
      re.lastIndex = i;
      const m = re.exec(input);
      if (m) { yield [name, m[0]]; i += m[0].length; break; }
    }
  }
}
```

Watch out

- **Catastrophic backtracking**: `/(a+)+b/` on `'aaaaaa…x'` takes exponential time. On untrusted input, use safe regex engines (RE2 via `re2`) or a real parser.
- A `/g` regex used with `.test()` or `.exec()` mutates `lastIndex`. Reusing the same regex across calls without resetting leads to every-other-call misses. Prefer `matchAll` or a fresh instance.

Chapter 28. JSON

What it is

`JSON` is a subset of JS — a notation for objects, arrays, strings, numbers, `true`, `false`, `null`. `JSON.stringify` serializes; `JSON.parse` deserializes.

Why you care

It's the wire format of the web. Edge cases (circular refs, `undefined`, functions, BigInt, Dates) bite in production and aren't obvious from `{a:1}` examples.

How it works

```
JSON.stringify({ a: 1, b: undefined, c: () => {}, d: Symbol() });
// '{"a":1}'  — undefined/function/symbol are DROPPED from objects
// In arrays they become null: JSON.stringify([undefined]) === '[null]'

JSON.stringify(new Date());           // '"2026-04-21T…Z"' — Date has a toJSON
JSON.stringify({ big: 1n });          // TypeError — BigInt doesn't serialize

// Circular references throw
const o = {}; o.self = o;
JSON.stringify(o);                    // TypeError
```

`toJSON` hook: any object that implements `toJSON()` has its return value used instead.

Use case: safe logging

```
function safeStringify(v) {
  const seen = new WeakSet();
  return JSON.stringify(v, (_, val) => {
    if (typeof val === 'bigint') return val.toString();
    if (typeof val === 'object' && val !== null) {
      if (seen.has(val)) return '[Circular]';
      seen.add(val);
    }
    return val;
  });
}
```

Watch out

- Numbers round to floats — big IDs lose precision. Transmit as strings.
- `JSON.parse` reviver runs bottom-up; you can reshape during parse.
- `Date` round-trips as a string, not a Date — `JSON.parse` will not revive it without a custom reviver.

Part X — Memory and Execution

Chapter 29. Memory Model and Garbage Collection

What it is

JS has automatic memory management. Objects live on the **heap**; primitives live on the stack or are inlined in objects. The GC finds objects no longer reachable from a root (globals, live stack frames, active closures) and reclaims them.

Why you care

Leaks don't mean your code uses `new`; they mean your code keeps references it no longer needs. Caches, event listeners, and long-lived closures are the usual culprits.

How it works

V8 uses a **generational, incremental, mostly concurrent** GC:

- **Young generation** (new space) — small, collected often via a copying scavenger. Most objects die young.
- **Old generation** — survivors are promoted here. Collected via mark-sweep, occasionally compacted.
- **Incremental marking** — big marks split into tiny slices between tasks, so pause times stay sub-millisecond.

Use case: finding a leak

```
// Node: run with --inspect, connect DevTools, take heap snapshots
// before and after N iterations. Diff to find retained objects.

// Or track with:
process.memoryUsage();                // { rss, heapTotal, heapUsed, external }
```

Common leak patterns

- **Unbounded caches.** Use LRU: `import LRU from 'lru-cache'`.
- **Listeners not removed.** `emitter.on('x', h)` without a matching `off`.
- **Closures capturing big objects.** `req` captured by a handler you scheduled for later, where only `req.id` was needed.
- **Module-level state.** Adding to a top-level `Map` per request without eviction.

Watch out

- Setting a variable to `null` doesn't help if another reference holds it.
- `--max-old-space-size` raises Node's heap limit; **always investigate before raising**. Usually the limit is a symptom, not the problem.

Chapter 30. Weak References

What it is

Data structures and references that **don't count** for GC: `WeakMap`, `WeakSet`, `WeakRef`, `FinalizationRegistry`.

Why you care

Caches keyed by objects shouldn't keep their keys alive. Tagging user objects with library metadata shouldn't leak. Weak refs let libraries associate data with objects without extending their lifetime.

How it works

```
const cache = new WeakMap();
function compute(obj) {
  if (!cache.has(obj)) cache.set(obj, expensive(obj));
  return cache.get(obj);
}
// When `obj` is no longer referenced anywhere else, its cache entry disappears
```

`WeakRef` : "hold this *if* it's still alive."

```
const ref = new WeakRef(largeObj);
// ...later...
const maybe = ref.deref();             // the object or undefined
```

`FinalizationRegistry` : run a callback after an object is collected.

```
const reg = new FinalizationRegistry((token) => closeFd(token));
reg.register(bufferObj, bufferObj.fd);
```

Watch out

- WeakMap is **not iterable** — you can't list keys or size. That's by design (it would expose GC timing).
- Finalization timing is **not guaranteed**. Don't rely on it for correctness, only as a safety net.
- Keys must be objects (primitives would defeat weakness) — though recent specs allow symbols.

Chapter 31. The V8 Pipeline

What it is

V8 compiles your JS through multiple tiers, starting fast and optimizing hot code.

Why you care

Knowing when V8 optimizes and when it deoptimizes explains benchmarks, guides performance work, and demystifies "why did my function suddenly get 10× slower?"

The tiers

1. **Parser** → AST.
2. **Ignition** — bytecode interpreter. Starts executing immediately.
3. **Sparkplug** — cheap baseline JIT.

4. **Maglev** — mid-tier optimizing JIT (newer).
5. **TurboFan** — top-tier optimizing JIT, aggressive inlining & speculation.

If an optimization's speculation is violated (types change, prototypes mutate), V8 **deoptimizes** — throws out the optimized code and falls back to Ignition.

Example

```
function add(a, b) { return a + b; }

// Called 10_000 times with numbers → TurboFan compiles a fast int-add
for (let i = 0; i < 10_000; i++) add(1, 2);

// Now call it with strings → speculation invalid → deopt
add('x', 'y');
```

Watch out

- Monomorphic call sites are fast. Polymorphic (2–4 shapes) are OK. **Megamorphic** (5+) are slow.
- Avoid changing object shapes after creation — shape == hidden class (next chapter).

Chapter 32. Hidden Classes and Inline Caches

What it is

Every object in V8 has an invisible "hidden class" (internally a *Map*) describing its property layout. Access by property name is compiled as a fixed-offset memory load — as long as the shape matches.

Why you care

Two objects that *look* the same but were built differently can have different hidden classes, making loops 10× slower without any visible difference in code.

How it works

```
// Same hidden class — properties added in the same order
function A() { this.x = 1; this.y = 2; }
function B() { this.x = 1; this.y = 2; }
// a and b share a shape, access to .x is a fixed offset.

// Different hidden class — properties added in different orders
function C() { this.y = 2; this.x = 1; }
// c does NOT share a shape with a — call sites that see both go polymorphic.
```

Inline caches (ICs) on each access/call site remember the shape they saw. One shape = monomorphic (fast). 2–4 = polymorphic. ≥5 = megamorphic (dictionary lookup).

Practical rules

- Initialize all properties in a constructor, in the same order.
- Don't `delete` properties on hot objects (forks the shape).
- Don't add properties to existing objects in unrelated code paths.
- Classes help: the engine sees a single constructor and the field initializer order.

Watch out

- Adding a single property conditionally (`if (admin) this.role = ...`) creates two shapes. For hot objects, initialize to `undefined` and assign later.
- Monkey-patching built-ins after warm-up throws away compiled code across the whole process.

Part XI — Binary and Shared Memory

Chapter 33. ArrayBuffer and Typed Arrays

What it is

`ArrayBuffer` is a fixed-size region of raw bytes. **Typed arrays** (`Uint8Array`, `Int32Array`, `Float64Array`, ...) are typed **views** over a buffer.

Why you care

Binary protocols, WebGL, crypto, streaming, file I/O, wasm interop — whenever you handle bytes, you reach for this.

How it works

```
const buf = new ArrayBuffer(16);      // 16 bytes of zeros
const u32 = new Uint32Array(buf);     // 4 × uint32 views
const u8  = new Uint8Array(buf);      // 16 × uint8 views over the same bytes

u32[0] = 0xDEADBEEF;
u8[0];                                // 0xEF on little-endian (your laptop)
```

`DataView` gives you explicit endianness:

```
const dv = new DataView(buf);
dv.setUint32(0, 0xDEADBEEF, /*littleEndian*/ false);
dv.getUint32(0, false);               // 0xDEADBEEF — big-endian, network order
```

Use case: parsing a binary header

```
function parseHeader(bytes) {
  const dv = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
  return {
    magic: dv.getUint32(0, false),
    version: dv.getUint16(4, false),
    length: dv.getUint32(6, false),
  };
}
```

Watch out

- `buffer.slice(s, e)` copies. `new Uint8Array(buf, offset, length)` is zero-copy (a view).
- Endianness matters for protocols — always use `DataView` when reading from the wire.

Chapter 34. SharedArrayBuffer and Atomics

What it is

A buffer that can be **shared** between threads (Web Workers, Node Worker Threads) — real shared memory, not message-passed copies. `Atomics` provides the operations you need for safe concurrent access.

Why you care

Lock-free queues between workers, shared counters, high-throughput producer/consumer pipelines. The only way to do cheap thread coordination in JS.

How it works

```
// Main thread
import { Worker } from 'node:worker_threads';
const shared = new SharedArrayBuffer(4);
const counter = new Int32Array(shared);
const w = new Worker('./w.js', { workerData: shared });

// w.js
import { workerData } from 'node:worker_threads';
const counter = new Int32Array(workerData);
Atomics.add(counter, 0, 1);           // atomic increment
Atomics.notify(counter, 0, 1);        // wake one waiter
```

Coordination primitives: `Atomics.wait(arr, index, expected)` blocks until the cell changes; `Atomics.notify` wakes waiters.

Watch out

- In browsers, SharedArrayBuffer requires **cross-origin isolation** (COOP/COEP headers) due to Spectre.
- Without `Atomics`, concurrent reads/writes are undefined behavior — you will see half-updated values.

Chapter 35. Structured Clone and Transferables

What it is

`structuredClone(value)` creates a deep copy that handles Maps, Sets, Dates, RegExps, TypedArrays, cycles — things `JSON.parse(JSON.stringify(x))` silently drops or breaks.

Why you care

`postMessage` between workers, `IndexedDB` storage, reliable deep copy — all use the structured clone algorithm. Knowing what's clonable saves you from surprises.

How it works

```
const a = { d: new Date(), m: new Map([['k', 1]]), set: new Set([1]) };
const b = structuredClone(a);
b.d instanceof Date;                  // true
b.m.get('k');                         // 1

const o = {}; o.self = o;
structuredClone(o).self === structuredClone(o); // cycles handled
```

Transferables are even better: instead of copying a big buffer, you transfer ownership — source side becomes unusable, destination gets the bytes at no copy cost.

```
const big = new Uint8Array(50_000_000).buffer;
worker.postMessage(big, [big]);       // transfer the buffer
big.byteLength;                       // 0 — detached
```

Watch out

- Functions, DOM nodes, and Error objects aren't structured-clonable in all environments.
- Transfer is **destructive**. Don't touch the source after posting.

Part XII — Advanced Edges

Chapter 36. Realms and `globalThis`

What it is

A **realm** is a complete JS environment — its own globals, built-ins (Array, Object, Promise, etc.), and `globalThis`. Each iframe, each Worker, each Node `vm.createContext` is a separate realm.

Why you care

`instanceof Array` **returns false** for an array from another realm (each realm has its own `Array.prototype`). Cross-realm checks trip up libraries.

How it works

```
// Browser, in the main page
const iframe = document.createElement('iframe');
document.body.appendChild(iframe);
const Arr2 = iframe.contentWindow.Array;
const a = new Arr2();
a instanceof Array;                  // false!
Array.isArray(a);                    // true — realm-aware check
```

Safe cross-realm checks:

- `Array.isArray(x)`
- `Object.prototype.toString.call(x) === '[object Date]'`
- Sentinel property on your own classes: `err.code === 'E_FOO'`

Watch out

- `globalThis` refers to the current realm's global. Don't cache "the" globalThis in a library — it breaks when the library is loaded from different realms.

Chapter 37. `eval` — Direct vs Indirect

What it is

`eval(str)` runs the string as JS. It's deeply discouraged for user input (injection, performance), but even when safe, there are two modes with different scope rules.

Why you care

Interview bait. Also: tooling (bundlers, REPLs, dev tools) uses it, and knowing the rules makes some bugs less mysterious.

How it works

```
// Direct eval: uses caller's scope
function f() {
  const local = 1;
  return eval('local');               // 1
}

// Indirect eval: runs at global scope
function g() {
  const local = 1;
  return (0, eval)('local');          // ReferenceError: local is not defined
}
```

The aliasing trick `(0, eval)(...)` forces indirect eval — the one you actually want if you must eval at all.

Watch out

- Strict mode disables many of eval's tricks (can't create new vars in the enclosing scope).
- Modern code uses `new Function(...)` (always indirect, isolated scope) or `vm.runInNewContext` (Node) instead.

Chapter 38. Intl — Internationalization

What it is

The `Intl` namespace provides locale-aware formatting: numbers, currencies, dates, collation, pluralization, list formatting, relative times, and grapheme segmentation.

Why you care

Correct formatting for users worldwide, without shipping locale data or maintaining regex hacks. The browser/Node already has ICU data; use it.

Examples

```
new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' }).format(1
// '1.234,56 €'

new Intl.DateTimeFormat('en-US', { dateStyle: 'full', timeStyle: 'short' }).for
// 'Tuesday, April 21, 2026 at 10:15 AM'

new Intl.RelativeTimeFormat('en').format(-3, 'day');  // '3 days ago'

// Grapheme segmentation — true user-perceived characters
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
[...seg.segment('👩‍❤️‍👨a')].length;   // 2 — the family emoji counts as one graphem
```

Watch out

- Constructing an `Intl.*` instance is expensive — cache them if used in a loop.
- Locale data comes from ICU; small Node/browser builds (e.g., `node-lite`) may have reduced locale coverage.

Chapter 39. Decorators (Stage 3/4)

What it is

Functions that modify classes, methods, fields, or accessors at definition time. Decorators landed as stage 3 and are supported by TypeScript (new-style in 5.x) and modern bundlers.

Why you care

Replaces a lot of boilerplate in frameworks (NestJS, MobX, TypeORM). Less pattern-matching, more declarative code.

How it works

```
function log(_target: any, context: ClassMethodDecoratorContext) {
  const name = String(context.name);
  return function (this: any, ...args: any[]) {
    console.log(`→ ${name}`, args);
    // @ts-ignore
    const r = _target.apply(this, args);
    console.log(`← ${name}`, r);
    return r;
  };
}

class Math2 {
  @log
  double(n: number) { return n * 2; }
}
```

Watch out

- The "legacy" decorators (TypeScript's old `experimentalDecorators`) and the stage-3 spec have different APIs. Pick one per project.
- Decorator evaluation order is top-to-bottom for declaration but bottom-to-top for application — they wrap like onion layers.

Appendix A — Performance Cheat Sheet

1. **Monomorphism beats cleverness.** Keep hot call sites seeing one shape.
2. **Initialize properties in a fixed order.** Don't `delete`.
3. **Avoid `arguments` in hot code.** Use rest params.
4. **Cache Intl formatters.** Construction is expensive.
5. **Use Maps for dynamic keys**, objects for fixed structure.
6. **Avoid `try/catch` in hot inner loops** (pre-TurboFan versions couldn't optimize through them; TurboFan can, but it still pessimizes in practice — measure).
7. **`for` with `let i` is still faster than `.forEach`** in most cases by a small margin. Use whichever you read faster; measure when it matters.
8. **Buffers and typed arrays** beat string concatenation for binary work.
9. **Profile with `--prof` (Node) / the DevTools performance tab.** Measure, don't guess.

Appendix B — Gotchas Quick Reference

Symptom	Likely cause
`this` is undefined in a callback	Lost method binding; use arrow or `bind`
`0.1 + 0.2 !== 0.3`	IEEE 754 rounding — compare with tolerance
`NaN !== NaN`	Strict equality spec; use `Number.isNaN` or `includes`
Array of `undefined` after `new Array(n).map(...)`	`new Array(n)` creates *sparse* — use `Array.from({length:n}, fn)`
`for..in` iterates over inherited string keys	Use `for..of`, `Object.entries`, or `Object.keys`
`JSON.stringify` dropped a field	It was `undefined`, a function, or a symbol
A big number changes after JSON round-trip	Lost precision beyond 2^{53} — use strings or BigInt
Regex matches skip every other call	`/g` + `.exec` / `.test` leaves `lastIndex` set
`instanceof` returns false for an object that should match	Cross-realm; use `Array.isArray` / sentinel property
Memory grows, never shrinks	Unbounded cache, listener leak, or closure holding too much

---

Afterword

JavaScript punishes guesses and rewards patience. When something surprises you, resist the urge to add a workaround until you can explain what happened. The explanation will almost always point to a chapter in this book — usually an earlier one than you expected.

Good luck. Read the spec when you're stuck. Remember that the language is smaller than it looks: a handful of mechanisms, composed relentlessly.

Book Two — Node.js In Depth

Node.js In Depth

What the runtime actually does when you run `node server.js`.

Preface

The JavaScript book taught you the language. This one teaches you the runtime that runs the language on servers: what the process does, how I/O becomes events, why streams exist, when to reach for Worker Threads instead of Cluster, and what the phrase "non-blocking I/O" really means when you're staring at a slow endpoint at 2 a.m.

Same structure as before: **What it is**, **Why you care**, **How it works**, **Watch out**.

Part I — The Runtime

Chapter 1. What Node.js Really Is

What it is

Node.js is a program that embeds **V8** (Google's JavaScript engine) and wires it up to **libuv** (a C library for async I/O and a thread pool) plus a set of C++ bindings exposing OS primitives (files, sockets, timers, crypto, DNS, etc.). Your JS is the script language; V8 runs it; libuv does the actual work that JS can't do itself.

Why you care

"Single-threaded, non-blocking, event-driven" is three phrases that mean the same thing: *your JavaScript runs on one thread, and everything slow (disk, network, DNS, crypto) is delegated elsewhere*. Once you see the split, Node stops being magic.

The components

```
┌──────────────────────────────────────────────────┐
│  Your JS code                                    │
├──────────────────────────────────────────────────┤
│  Node.js core modules (fs, http, net, crypto…)   │
├───────────────┬──────────────────────────────────┤
│               │   C++ bindings                   │
│     V8        ├──────────────────────────────────┤
│  (JS engine)  │   libuv (event loop, thread pool,  │
│               │   async I/O, timers)             │
├───────────────┴──────────────────────────────────┤
│  Operating system (epoll / kqueue / IOCP)        │
└──────────────────────────────────────────────────┘
```

- **V8** compiles and runs JS.
- **libuv** implements the event loop, a thread pool (default 4 threads), and cross-platform async I/O.

- **C++ bindings** glue them: `fs.readFile` is a JS function that calls a C++ function that asks libuv to read the file.

Watch out

- "Non-blocking" applies to I/O, not CPU. A tight `while` loop in JS blocks the event loop. Crypto, image processing, JSON.parse on huge strings — all can block.
- Node is the *runtime*; npm is the *package manager*. They're separate projects that ship together.

Chapter 2. Node vs the Browser

What it is

Both run JS on V8, but they expose different globals and APIs, and they run in different trust and security models.

The table

	Browser	Node
Global object	`window` (also `globalThis`)	`global` (also `globalThis`)
Module system	ESM by default	CJS by default; ESM opt-in
DOM / `fetch`	Yes / Yes	No DOM; `fetch` since Node 18
File system	Via `File` / `FileReader`	`fs`, `fs/promises`
Timers	`setTimeout` returns a number	Returns a `Timeout` object
Sandboxed	Yes — origin-isolated	No — full OS access
Entry point	`<script>`	`node file.js`
Event loop	Integrated with rendering	libuv, no rendering

Watch out

- `fetch`, `URL`, `FormData`, `Blob`, `crypto.subtle` are all available in modern Node — but some edge behaviors differ.
- `Buffer` is Node-specific. In browsers use `Uint8Array` or `Blob`.
- In Node 18+ you can mostly write universal code, but don't assume — test both sides when it matters.

Chapter 3. The Event Loop (libuv)

What it is

A phase machine that runs one iteration ("tick") at a time. Each phase has its own queue; callbacks added during a phase run in that or the next tick depending on the phase.

The six phases

```
   ┌───────────────────────────┐
┌─>│           timers          │  setTimeout, setInterval callbacks whose time
│  ├───────────────────────────┤
│  │     pending callbacks     │  deferred I/O (rare: e.g., TCP errors)
│  ├───────────────────────────┤
│  │       idle, prepare       │  internal
│  ├───────────────────────────┤
│  │                           │  ← wait here for I/O (epoll/kqueue)
│  │           poll            │  run I/O callbacks
│  │                           │
│  ├───────────────────────────┤
│  │           check           │  setImmediate callbacks
│  ├───────────────────────────┤
└──│      close callbacks      │  'close' events (sockets, handles)
   └───────────────────────────┘
```

Between every phase, Node drains:

1. `process.nextTick` queue (highest priority).
2. Microtask queue (Promise reactions, `queueMicrotask`).

Then the loop moves on.

Example: the classic `setTimeout` vs `setImmediate` race

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
// Order is NOT guaranteed outside an I/O callback.
// Inside an I/O callback, setImmediate always wins:
fs.readFile('x', () => {
  setTimeout(() => console.log('t'), 0);
  setImmediate(() => console.log('i'));
  // 'i' first — poll phase enters check before timers on next tick
});
```

Use case: knowing what blocks

```
const t0 = Date.now();
fs.readFile('huge.log', () => {
  console.log('I/O done at', Date.now() - t0);
});
// Tight loop blocks the event loop — I/O callback waits
while (Date.now() - t0 < 2000) { /* blocks */ }
```

Watch out

- `process.nextTick` runs **before** Promise microtasks. Recursive `nextTick` can starve I/O.
- The thread pool (default 4) handles fs, crypto, DNS, zlib. CPU-bound crypto can saturate it — bump `UV_THREADPOOL_SIZE` or move work to Worker Threads.

Chapter 4. Timers in Detail

What it is

`setTimeout`, `setInterval`, `setImmediate`, `process.nextTick`, `queueMicrotask` — five ways to defer work, with different phases and priorities.

Priority order

```
currently running stack
  ├─ process.nextTick queue     ← drained first, fully
  ├─ microtasks (Promise)       ← drained next, fully
  ├─ timers (setTimeout)        ← phase: timers
  ├─ I/O callbacks              ← phase: poll
  ├─ setImmediate               ← phase: check
  └─ close events               ← phase: close
```

Example

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
console.log('sync');

// sync
// nextTick
// promise
// timeout   (or immediate first; order non-deterministic here)
// immediate
```

Use case: deferring a heavy computation so a response can flush

```
function defer(fn) { setImmediate(fn); }

app.post('/process', (req, res) => {
  res.json({ queued: true });
  defer(() => heavyJob(req.body));          // returns to client immediately
});
```

Watch out

- `setTimeout(fn, 0)` is clamped to ~1ms minimum. Use `setImmediate` or `queueMicrotask` for "next tick".
- `setInterval` drifts — use `setTimeout` that reschedules if precision matters.

Part II — Modules and Process

Chapter 5. CommonJS and ESM in Node

What it is

Node speaks both module systems. Which one your file uses is decided by extension and `package.json` settings.

The rules

File	Module system
`.mjs`	ESM
`.cjs`	CJS
`.js` with `"type": "module"` in nearest `package.json`	ESM
`.js` otherwise	CJS

Examples

```
// CJS
const fs = require('node:fs');           // sync
module.exports = { add: (a, b) => a + b };
```

```
// ESM
import fs from 'node:fs';                // async under the hood
export function add(a, b) { return a + b; }
export default add;
```

Interop

- ESM can `import` CJS: you get `module.exports` as the default, and named exports when statically analyzable.
- CJS loading ESM was impossible for years; modern Node (22+) supports synchronous ESM via `require()` with flags; prefer dynamic `import()` for compatibility.

Use case: `import.meta.url` for the current file path

```
import { fileURLToPath } from 'node:url';
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

(`__filename` / `__dirname` exist automatically in CJS; ESM needs the above.)

Watch out

- `require.cache` lets you inspect/evict module caches (handy in tests). ESM has no equivalent — modules load once per process.
- Top-level `await` works in ESM only; the whole module becomes async.
- Named imports from CJS sometimes fail if the library does `module.exports = something` (not `module.exports.x = ...`). Use `import pkg from 'x'; const { a } = pkg;` .

Chapter 6. The `process` Object

What it is

A global object representing the current Node process. Environment, CLI args, stdio, exit, signals, resource usage — all live here.

Why you care

Every production concern — reading config, handling SIGTERM, exiting cleanly, measuring memory — goes through `process` .

The essentials

```
process.argv;                          // ['node', '/path/script.js', ...args]
process.env.NODE_ENV;                  // string or undefined
process.cwd();                         // current working directory
process.pid;                           // process id
process.platform;                      // 'darwin' | 'linux' | 'win32'
process.version;                       // 'v22.2.0'
process.memoryUsage();                 // { rss, heapTotal, heapUsed, external
process.uptime();                      // seconds since start

process.stdout.write('no newline');
process.stderr.write('error');
process.stdin.on('data', (chunk) => { /* ... */ });
```

Signals and exit

```
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);         // Ctrl-C
process.on('uncaughtException', (err) => { log(err); process.exit(1); });
process.on('unhandledRejection', (reason) => { log(reason); process.exit(1); })

async function shutdown(sig) {
  log(`got ${sig}`);
  await server.close();
  await db.destroy();
  process.exit(0);
}
```

Watch out

- `process.exit()` terminates **synchronously** — pending I/O is dropped. Prefer closing resources and letting the loop drain naturally.
- Environment variables are strings. Always parse: `process.env.PORT ? Number(process.env.PORT) : 3000`.

Chapter 7. EventEmitter

What it is

A base class for objects that emit named events. Most Node core objects (streams, HTTP servers, processes) inherit from it.

How it works

```
import { EventEmitter } from 'node:events';

class Counter extends EventEmitter {
  inc() { this.n = (this.n || 0) + 1; this.emit('tick', this.n); }
}

const c = new Counter();
c.on('tick', (n) => console.log('n =', n));
c.inc(); c.inc();
// n = 1
// n = 2
```

The methods you'll use

- `.on(event, fn)` / `.off(event, fn)` — listen / remove.
- `.once(event, fn)` — auto-remove after first fire.
- `.emit(event, ...args)` — synchronous call to all listeners.
- `.removeAllListeners(event?)`.
- `.setMaxListeners(n)` — default warn at 10 (indicates a possible leak).

Use case: domain events in a service

```
class OrderService extends EventEmitter {
  async create(data) {
    const order = await db.orders.insert(data);
    this.emit('order.created', order);
    return order;
  }
}
```

Decouples side effects: analytics, webhooks, email, etc., subscribe independently.

Watch out

- **Emit is synchronous.** Throwing inside a listener can crash the emitter. Wrap listener bodies in try/catch or use `once('error', ...)`.
- Listener leaks: always remove listeners when the subject outlives them, or the MaxListenersExceeded warning will appear.
- For async event handlers, consider `events.on(emitter, name)` — an async iterator of events.

Part III — I/O and Streams

Chapter 8. Buffer

What it is

A fixed-size chunk of raw bytes, allocated **outside** V8's heap. The pre-`Uint8Array` way Node handled binary data — kept for compatibility and zero-copy interop with libuv.

Why you care

Everything that hits the wire or the disk starts as a Buffer. Knowing how to slice, encode, and concatenate them without unnecessary copies matters for throughput.

How it works

```
const b = Buffer.from('hello', 'utf8');  // 5 bytes
b.length;                                // 5
b.toString();                            // 'hello'
b.toString('hex');                       // '68656c6c6f'
b.toString('base64');                    // 'aGVsbG8='

const out = Buffer.alloc(10);            // zero-filled
out.write('hi', 0, 'utf8');
```

Buffer extends `Uint8Array`, so you can pass one where a typed array is expected.

Use case: building a binary response

```
const header = Buffer.alloc(4);
header.writeUInt32BE(body.length, 0);
const packet = Buffer.concat([header, body]);
socket.write(packet);
```

Watch out

- `Buffer.allocUnsafe(n)` skips zero-filling — **fast but can leak old memory content** if you don't overwrite it. Use only when you're about to fill the whole buffer.
- `Buffer.from('...')` without an encoding defaults to UTF-8, which *usually* is right, but not for ASCII-only hot paths.
- Buffer memory counts as `external` in `process.memoryUsage`, not `heapUsed` — a memory leak in buffers hides from heap snapshots unless you look at the external field.

Chapter 9. Streams

What it is

Streams are Node's abstraction for sequences of data handled **chunk by chunk** instead of all at once. Four types: **Readable** (source), **Writable** (sink), **Duplex** (both), **Transform** (duplex where output is derived from input).

Why you care

Loading a 2 GB file with `fs.readFile` blows up your heap. `createReadStream` processes it in 64 KB pieces at constant memory. HTTP, TCP, compression, crypto, file I/O — all streams underneath.

The modes

- **Flowing** (`readable.on('data', ...)` or `.pipe(...)`) — data is pushed as it arrives.
- **Paused** (`readable.read()`) — you pull when ready.
- **Object mode** — chunks are JS objects instead of bytes.

How it works

```
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
import { createGzip } from 'node:zlib';

await pipeline(
  createReadStream('input.log'),
  createGzip(),
  createWriteStream('input.log.gz'),
);
```

`pipeline` propagates errors, cleans up resources, and returns a Promise — always prefer it over `.pipe()` chains.

Backpressure — the point of streams

When the destination is slow, `write()` returns `false`. You **stop writing**, wait for `'drain'`, then continue. `pipeline` handles this for you; with manual code, you must.

```
function writeAll(stream, items) {
  for (const item of items) {
    if (!stream.write(item)) {
      return new Promise(resolve => stream.once('drain', resolve))
        .then(() => writeAll(stream, items.slice(items.indexOf(item) + 1)));
    }
  }
  stream.end();
}
```

Transform streams — your own pipeline stage

```
import { Transform } from 'node:stream';

const upper = new Transform({
  transform(chunk, _enc, cb) { cb(null, chunk.toString().toUpperCase()); }
});

await pipeline(process.stdin, upper, process.stdout);
```

Watch out

- Mixing `.pipe()` and `'data'` listeners gives you duplicated reads.
- Not handling errors on a stream crashes the process on `unpipe` / `close`.
- Object-mode streams ignore the 64 KB highWaterMark — the unit is 1 object, so a high-volume pipeline needs custom throttling.

Chapter 10. File System

What it is

Three flavors: callback (`fs`), Promise (`fs/promises`), and synchronous (`fs.*Sync`). Underneath, libuv does the work on the thread pool.

Examples

```
import { readFile, writeFile, readdir, stat } from 'node:fs/promises';

const data = await readFile('config.json', 'utf8');
const parsed = JSON.parse(data);

await writeFile('out.txt', 'hello', 'utf8');
const names = await readdir('.');
const s = await stat('file.log');
console.log(s.size, s.mtime);
```

Use case: streaming a huge file

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

const rl = createInterface({ input: createReadStream('big.log') });
for await (const line of rl) {
  if (line.includes('ERROR')) console.log(line);
}
```

Watch out

- Never use `fs.*Sync` in a server — it blocks the event loop for the duration of the disk I/O. Fine for boot/config; never for per-request work.
- File system is on libuv's thread pool. CPU-bound crypto contending for the same pool can stall `fs` calls; bump `UV_THREADPOOL_SIZE` if needed.
- Paths are OS-specific. Always use `path.join` / `path.resolve` — never concatenate strings with `/`.

Chapter 11. HTTP and HTTPS

What it is

Node's built-in HTTP server and client, built on raw TCP. Express, Fastify, and every framework wrap this — understanding it pays off when things get weird.

A minimal server

```
import { createServer } from 'node:http';

const server = createServer(async (req, res) => {
  if (req.method === 'POST' && req.url === '/echo') {
    const chunks = [];
    for await (const chunk of req) chunks.push(chunk);
    const body = Buffer.concat(chunks).toString('utf8');
    res.writeHead(200, { 'content-type': 'application/json' });
    res.end(JSON.stringify({ body }));
    return;
  }
  res.writeHead(404); res.end('not found');
});

server.listen(3000);
```

A minimal client (modern Node)

```
const r = await fetch('https://api.example.com/users', {
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({ name: 'Alice' }),
});
const data = await r.json();
```

Keep-alive

```
import { Agent } from 'node:https';
const agent = new Agent({ keepAlive: true, maxSockets: 100 });
// pass as { agent } to fetch/https.request
```

Reusing sockets saves TCP handshakes and TLS renegotiation — essential for high-throughput clients.

Watch out

- `req.on('data')` chunks can split JSON mid-byte. Buffer fully before parsing or use a framework.
- `res.end()` must always be called — or the connection hangs until the client times out.
- Default `maxSockets` for the global agent is `Infinity` — you can exhaust file descriptors under load. Configure agents.

Chapter 12. Net — Raw TCP and Unix Sockets

What it is

`net.createServer` gives you a raw TCP server; `net.connect` gives you a client. HTTP is built on top of this.

Use case: a simple protocol

```
import { createServer } from 'node:net';

createServer((sock) => {
  sock.on('data', (buf) => sock.write(buf));   // echo
  sock.on('end', () => console.log('client gone'));
}).listen(9000);
```

Watch out

- TCP is a byte stream, not messages. You need your own framing (length prefix, delimiter, or a protocol like HTTP/MQTT/Protobuf-over-TCP).
- Unix domain sockets (`net.createServer().listen('/tmp/sock')`) are faster than localhost TCP — use them between co-located processes (e.g., Nginx ↔ Node, sidecars).

Part IV — Concurrency

Chapter 13. Child Processes

What it is

Four ways to run another program:

- `spawn(cmd, args)` — long-running, streams stdout/stderr, low overhead.
- `exec(cmd)` — buffers output, convenient for quick shell commands, **shell injection risk if you interpolate untrusted input**.
- `execFile(file, args)` — like exec but no shell.
- `fork(modulePath)` — spawns another Node process and sets up an IPC channel.

Examples

```
import { spawn, fork } from 'node:child_process';

const ls = spawn('ls', ['-la', '/tmp']);
for await (const chunk of ls.stdout) process.stdout.write(chunk);
const code = await new Promise(r => ls.on('close', r));
```

```
// parent.js
const child = fork('./worker.js');
child.send({ cmd: 'compute', input: 42 });
child.on('message', (m) => console.log('result', m));
```

Use case: running an untrusted tool (image resize, ffmpeg)

```
const p = spawn('ffmpeg', ['-i', input, '-vf', 'scale=640:-1', output]);
```

Watch out

- `exec` with user input is a textbook shell injection. Use `execFile` or `spawn` with argument arrays.
- Children don't auto-die when the parent crashes. Use `detached: false` and kill them in shutdown handlers.
- Large stdout can deadlock if you never read it — the pipe fills and the child blocks.

Chapter 14. Cluster

What it is

A module that lets a primary process fork multiple worker processes, each a copy of your Node app, all sharing the same listening socket.

Why it exists

Node is single-threaded per process. To use multiple CPU cores on one machine, you run multiple processes. Cluster makes that easy — the OS handles load balancing across workers.

How it works

```
import cluster from 'node:cluster';
import os from 'node:os';

if (cluster.isPrimary) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork();
  cluster.on('exit', (w) => { console.log('worker died, restart'); cluster.fork
} else {
  // same code path as a single-process server
  import('./server.js');
}
```

Use case vs PM2 vs Kubernetes

- **Cluster** — single-machine, quick, no extra ops.

- **PM2** — cluster + restart + log management + zero-downtime reload, one CLI.
- **Kubernetes / ECS** — treat each pod as a single Node process; let the orchestrator do the scaling. Modern preferred path.

Watch out

- Workers share nothing but a listening port. Per-worker in-memory caches are per-worker — use Redis for shared state.
- Sticky sessions (WebSocket) need session affinity; cluster round-robins by default.

Chapter 15. Worker Threads

What it is

Real OS threads, each with its own V8 instance, event loop, and memory. They talk via `postMessage` (structured clone) or `SharedArrayBuffer` + `Atomics`.

Why you care

Worker Threads are how you run CPU-bound work (image processing, data crunching, regex) without blocking the main event loop. **Cluster spawns processes; Worker Threads are threads.**

Example

```
// main.js
import { Worker } from 'node:worker_threads';

const w = new Worker('./heavy.js', { workerData: { n: 40 } });
w.on('message', (result) => console.log('got', result));
w.on('error', console.error);
w.on('exit', (code) => console.log('exit', code));
```

```
// heavy.js
import { parentPort, workerData } from 'node:worker_threads';
function fib(n) { return n < 2 ? n : fib(n-1) + fib(n-2); }
parentPort.postMessage(fib(workerData.n));
```

Use case: a worker pool

Workers are expensive to start (~50–100 ms). For short jobs, keep a pool:

```
import { Worker } from 'node:worker_threads';
class Pool {
  constructor(path, size) {
    this.idle = [];
    this.queue = [];
    for (let i = 0; i < size; i++) this.idle.push(new Worker(path));
  }
  run(task) {
    return new Promise((resolve, reject) => {
      const go = (w) => {
        w.once('message', resolve);
        w.once('error', reject);
        w.postMessage(task);
      };
      const w = this.idle.shift();
      if (w) go(w);
      else this.queue.push(go);
    });
  }
}
```

(Production pools add error recovery, timeouts, worker recycling. See `piscina`.)

Watch out

- No shared memory except `SharedArrayBuffer`. All other values are structured-cloned on `postMessage` (copies — not free).
- Don't use threads for I/O — libuv already handles that. Threads are for **CPU work**.

Chapter 16. AsyncLocalStorage

What it is

A way to attach per-request context to *every* async callback spawned during that request, without passing it as a parameter. Implemented on top of async_hooks.

Why you care

Request IDs, user identity, tenant IDs in logs — you need them everywhere. Passing them through every function signature is noisy; `als.run()` scopes them automatically to all async descendants.

Example

```
import { AsyncLocalStorage } from 'node:async_hooks';

const als = new AsyncLocalStorage();

function log(msg) {
  const store = als.getStore();
  console.log(`[${store?.requestId ?? '-'}] ${msg}`);
}

app.use((req, res, next) => {
  als.run({ requestId: req.headers['x-request-id'] || crypto.randomUUID() }, ne
});

app.get('/x', async (req, res) => {
  log('handling');                      // logs with the right requestId
  await doWork();                       // doWork can also call log() and see th
  res.end();
});
```

Watch out

- `als.run()` creates a new scope — everything async inside it shares the store. Outside, the store is gone.

- Older Node had measurable overhead for async_hooks; recent versions have it down to near-zero.
- Some third-party thread-bound code (e.g., native addons with their own event loops) may break propagation. Test before relying on it in hot paths.

Part V — Production Concerns

Chapter 17. Error Handling Patterns

What it is

Two classes of errors with very different handling:

- **Operational errors** — expected, recoverable: bad input, 404, timeout, disk full. Handle per-request, return an error response, keep running.
- **Programmer errors** — bugs, invariant violations: null dereference, wrong types, broken state machine. **Crash and restart.**

Why the distinction matters

Trying to "handle" a programmer error leaves the process in an unknown state. Subsequent requests see weird, hard-to-debug failures. Crashing is the safe choice; a supervisor restarts.

Patterns

```
// 1. Custom error classes with codes
class NotFoundError extends Error {
  constructor(msg) { super(msg); this.name = 'NotFoundError'; this.code = 'E_NO
}

// 2. Central async handler (Express pattern)
app.use(async (err, req, res, next) => {
  if (res.headersSent) return next(err);
  const status = err.status || (err.code === 'E_NOT_FOUND' ? 404 : 500);
  req.log.error({ err });
  res.status(status).json({ error: err.message });
});

// 3. Process-wide safety nets
process.on('uncaughtException', (err) => { logger.fatal({ err }); process.exit(
process.on('unhandledRejection', (err) => { logger.fatal({ err }); process.exit
```

Watch out

- `throw` inside an EventEmitter listener is **not** caught — it propagates to `uncaughtException`.
- Never `process.exit(1)` inside a library — let callers decide.
- Rejected promises in a for-loop are not awaited if you forget `await` — silent failures.

Chapter 18. Graceful Shutdown

What it is

When you receive `SIGTERM` (Kubernetes sends this before `SIGKILL`), stop accepting new work, finish what's in flight, close DBs and caches, and exit with 0.

Why you care

Ungraceful shutdowns drop requests, leave transactions half-committed, and corrupt logs. Every serious service needs a 30-second drain window.

Pattern

```
let shuttingDown = false;

const server = app.listen(port);

async function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;
  logger.info({ signal }, 'shutting down');

  // 1. Stop accepting new HTTP connections
  server.close(() => logger.info('http closed'));

  // 2. Drain in-flight (wait up to N seconds)
  await drainInFlight(30_000);

  // 3. Close external resources
  await db.destroy();
  await redis.quit();

  process.exit(0);
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT',  () => shutdown('SIGINT'));
```

Watch out

- `server.close()` waits for existing keep-alive connections; destroy idle sockets yourself or they'll keep the process alive:

  ```
  server.closeIdleConnections();     // Node 18.2+
  ```

- Load balancers need a moment to notice you're gone. Many teams add `/healthz` that returns 503 during shutdown, then actually close a few seconds later.

Chapter 19. Logging in Production

What it is

Structured logs (JSON lines, key/value) — never free-form text in production. One line per event, with request id, level, timestamp, and message.

Why you care

Splunk, Datadog, CloudWatch, Loki all love JSON. Free-form logs are grep-only, require regex, and break under pressure.

Example with `pino`

```
import pino from 'pino';
const log = pino({ level: process.env.LOG_LEVEL || 'info' });

log.info({ userId, route: req.path }, 'request');
log.error({ err }, 'failed');
```

Output:

```
{"level":30,"time":1713619200000,"msg":"request","userId":42,"route":"/x"}
```

Use case: request-scoped logging with AsyncLocalStorage

Combine Chapter 16's ALS with pino's child loggers:

```
const log = pino();
const als = new AsyncLocalStorage();

app.use((req, res, next) => {
  const reqLog = log.child({ requestId: req.headers['x-request-id'] });
  als.run({ log: reqLog }, next);
});

// anywhere downstream:
als.getStore().log.info('processing');
```

Watch out

- Log volume costs money. Keep `debug` / `trace` off in prod; log expensive payloads behind a level check (`log.isLevelEnabled('debug') && log.debug(...)`).
- Never log secrets, tokens, or full request bodies by default. Use redaction config.

Chapter 20. Configuration

What it is

Environment variables — not hardcoded values, not files committed to git. Validated at boot, typed, fail-fast if missing.

Pattern

```
import { z } from 'zod';

const Env = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
});

export const env = Env.parse(process.env);
// Typed, validated, and available everywhere via `import { env } from './env'`
```

Watch out

- Don't read `process.env.X` scattered across the codebase — boot-time validation is the whole point.
- Secrets belong in a secret manager (AWS Secrets Manager, Vault), not `.env` files committed by accident. Add `.env` to `.gitignore` forever.

Part VI — Advanced Runtime

Chapter 21. Async Hooks (briefly)

What it is

A low-level API that lets you observe every async resource lifecycle: created, before-callback, after-callback, destroyed. `AsyncLocalStorage` is built on top of it.

Why you care (a little)

You almost never use async_hooks directly. It's the foundation for tracing (OpenTelemetry), profilers, and `AsyncLocalStorage`. Know it exists; reach for `AsyncLocalStorage` 99% of the time.

Example

```
import async_hooks from 'node:async_hooks';
const hook = async_hooks.createHook({
  init(asyncId, type, triggerAsyncId) { /* ... */ },
  destroy(asyncId) { /* ... */ },
});
hook.enable();
```

Watch out

- Enabling async_hooks has overhead — historically ~10–20%, now much lower but still non-zero. Don't enable if you don't need it.

Chapter 22. Crypto

What it is

Built-in bindings to OpenSSL: hashing, HMAC, symmetric/asymmetric ciphers, key derivation, random bytes, TLS. Plus a web-compatible `crypto.webcrypto` interface.

Why you care

Every auth system, session signer, webhook verifier, encryption-at-rest implementation goes through this. Rolling your own is a security hole.

Common tasks

```
import { randomBytes, createHash, createHmac, timingSafeEqual, scrypt } from 'n

// Random token
const tok = randomBytes(32).toString('hex');

// Hash (non-password!): SHA-256
createHash('sha256').update('hello').digest('hex');

// HMAC for signatures
const sig = createHmac('sha256', secret).update(payload).digest('hex');

// Constant-time compare (don't use === for signatures)
const ok = timingSafeEqual(Buffer.from(a, 'hex'), Buffer.from(b, 'hex'));

// Password hashing — scrypt (preferred) or argon2 via library
const key = await new Promise((resolve, reject) =>
  scrypt('password', 'salt', 64, (err, k) => err ? reject(err) : resolve(k))
);
```

Watch out

- **Never hash passwords with SHA-256.** Use scrypt, argon2, or bcrypt with a salt per user.
- Use `timingSafeEqual` for comparing secrets, HMACs, tokens — `a === b` leaks timing.
- Crypto operations run on libuv's thread pool. Bulk crypto can starve `fs`; size your pool (`UV_THREADPOOL_SIZE`) accordingly.

Chapter 23. Debugging and Profiling

What it is

Node has a built-in V8 inspector. Run with `--inspect` and connect Chrome DevTools, VS Code, or the Node CLI debugger.

CLI flags worth knowing

```
--inspect                attach DevTools (non-blocking start)
--inspect-brk            break on first line (handy for entry-point bugs)
--prof                   sample CPU → isolate-*.log → process with --prof-proce
--cpu-prof               write .cpuprofile (open in DevTools)
--heap-prof              sample heap allocations
--trace-warnings         stack traces on warnings
--trace-deprecation      stack traces on deprecations
--unhandled-rejections=strict    crash on unhandled rejections (default in new
```

Clinic.js — higher-level tooling

```
npx clinic doctor  -- node server.js   # detect event loop lag / I/O issues
npx clinic flame   -- node server.js   # flame graph of on-CPU time
npx clinic heap    -- node server.js   # heap profiling
```

Event loop lag monitoring

```
import { monitorEventLoopDelay } from 'node:perf_hooks';
const h = monitorEventLoopDelay({ resolution: 20 });
h.enable();
setInterval(() => {
  const p99ms = h.percentile(99) / 1e6;
  if (p99ms > 100) logger.warn({ p99ms }, 'event loop lagging');
  h.reset();
}, 5000);
```

Watch out

- Profiling adds overhead. Don't leave `--cpu-prof` on in production unless you're collecting with sampling and rate-limiting.
- A single hot sync function (JSON.parse of a huge body, a regex backtrack) can cause 100 ms+ loop lag — always measure with monitorEventLoopDelay in prod.

Chapter 24. Native Addons and N-API

What it is

C/C++ code compiled into a shared library and loaded into Node via `require`. Used to wrap OS APIs, expose native libraries (like image codecs, sqlite, v8 internals), or escape V8 for performance.

Why you care (a little)

You'll *use* native addons (bcrypt, sharp, better-sqlite3) often. You'll *write* one rarely. If you do, you'll use **N-API** (the stable ABI) via `node-addon-api` (C++) or `napi-rs` (Rust).

Tradeoffs

- Pro: access to native code, faster than JS for tight numeric loops.
- Con: platform builds (prebuilt binaries or node-gyp), harder to debug, can crash the process, slower to start (dynamic linking).

Watch out

- A native crash kills Node with no JS stack trace. Use sanitizers while developing.
- Prebuilt binaries via `prebuild` / `node-pre-gyp` are the difference between "npm install works" and "users yell at you on Windows".

Appendix — Node.js Production Checklist

- ☐ Env vars validated at boot (zod/envalid); no `process.env.X` scattered around.
- ☐ Structured JSON logs with a requestId per request (ALS).
- ☐ `SIGTERM` / `SIGINT` handler with drain + resource close + `process.exit(0)`.
- ☐ `uncaughtException` / `unhandledRejection` handlers that log fatally and exit.
- ☐ `server.keepAliveTimeout` and `headersTimeout` set above your LB's idle timeout.
- ☐ `UV_THREADPOOL_SIZE` tuned if you do heavy crypto/fs/DNS.
- ☐ Event loop delay monitored and alerted on (> 100 ms p99).
- ☐ Graceful startup: readiness probe returns 200 only after DB/cache connected.
- ☐ Graceful shutdown: liveness returns 503 during drain.
- ☐ Security headers (helmet), CORS pinned, CSRF on cookie auth, rate limit.
- ☐ Secrets in a manager, not `.env` in git.
- ☐ Dependabot / `npm audit` in CI.
- ☐ Docker image: non-root user, minimal base (distroless/alpine), init process (`--init` or `tini`).
- ☐ Tracing and metrics (OpenTelemetry → Datadog/Grafana).

Afterword

Node.js rewards engineers who stop treating it as "JavaScript on the server" and start treating it as "V8 + libuv + a carefully chosen API surface". Everything unusual about Node — event loop phases, backpressure, `uncaughtException`, cluster vs workers — maps cleanly to that architecture once you can see it.

Read the libuv book. Read the Node changelog. Turn on `monitorEventLoopDelay`. Your future self will thank you.

Part V — Asynchronous Programming

These questions draw from day-to-day work — the common async patterns and pitfalls you'll see in code review.

3. Asynchronous Programming

Q29. Callbacks vs Promises vs async/await

- **Callbacks** — original pattern; leads to "callback hell" and error-handling nightmares.
- **Promises** — chainable; solve composition and errors (`.catch`).
- **async/await** — syntactic sugar over Promises; looks synchronous, uses `try/catch` .

Q30. Promise states and guarantees

A promise is in one of three states: **pending → fulfilled | rejected**. Once settled, it's immutable. `.then` callbacks always run asynchronously (as microtasks), even on an already-resolved promise.

Q31. `Promise.all` vs `allSettled` vs `race` vs `any`

```
await Promise.all([a, b, c]);       // reject if ANY rejects; fail-fast
await Promise.allSettled([a, b, c]);// waits for all; never rejects, returns st
await Promise.race([a, b]);         // first to settle (resolve OR reject)
await Promise.any([a, b]);          // first to FULFILL; rejects only if all re
```

Interview trap: if one of the promises in `Promise.all` rejects, **the others keep running** — you've just lost control of them. If they have side effects (DB writes, API calls), you may want `allSettled` instead.

Q32. How would you implement `Promise.all` yourself?

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = new Array(promises.length);
    let remaining = promises.length;
    if (remaining === 0) return resolve(results);
    promises.forEach((p, i) => {
      Promise.resolve(p).then(
        (val) => {
          results[i] = val;
          if (--remaining === 0) resolve(results);
        },
        reject // fail fast
      );
    });
  });
}
```

Q33. Running async tasks with concurrency limit

```
async function pLimit(tasks, limit) {
  const results = [];
  const executing = new Set();
  for (const task of tasks) {
    const p = Promise.resolve().then(task);
    results.push(p);
    executing.add(p);
    p.finally(() => executing.delete(p));
    if (executing.size >= limit) await Promise.race(executing);
  }
  return Promise.all(results);
}
```

Or use the `p-limit` package.

Q34. Microtasks vs Macrotasks

- **Microtasks**: `Promise.then`, `queueMicrotask`, `process.nextTick` (nextTick has even higher priority in Node).
- **Macrotasks**: `setTimeout`, `setImmediate`, I/O callbacks.

After each macrotask, the microtask queue is **fully drained** before the next macrotask runs. That's why a misbehaving promise chain can starve I/O.

Q35. Async error handling pitfalls

```
// ❌ Uncaught — .then handler errors without .catch
fetchUser().then(u => JSON.parse(u.data));

// ❌ Swallowed in forEach
users.forEach(async (u) => { await save(u); }); // no one awaits these

// ✅ Proper
for (const u of users) { await save(u); }                    // sequential
await Promise.all(users.map(u => save(u)));                   // parallel
```

Always listen for `unhandledRejection` at the process level in production.

Q36. Why does `async` always return a Promise?

Because ECMAScript defines `async` functions to wrap their return value in `Promise.resolve()`. Even `async function f() { return 1 }` gives you a Promise.

Part VI — HTTP Frameworks

Chapter — Express.js

4. Express.js

Q37. What is middleware?

A function with signature `(req, res, next)` that sits in the request pipeline. It can:

- Modify `req` / `res`
- End the request (`res.send`)
- Pass control via `next()`
- Pass error via `next(err)`

```
app.use((req, res, next) => {
  req.startTime = Date.now();
  res.on('finish', () => console.log(Date.now() - req.startTime, 'ms'));
  next();
});
```

Q38. Error-handling middleware

An error handler has **4 arguments** — Express recognizes it by arity.

```
app.use((err, req, res, next) => {
  req.log.error(err);
  res.status(err.status ?? 500).json({ error: err.message });
});
```

Errors from sync code are caught automatically. For **async routes**, throw via `next(err)` or use a wrapper:

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get('/user', asyncHandler(async (req, res) => {
  const u = await db.findUser(req.query.id);
  res.json(u);
}));
```

Express 5 handles this automatically.

Q39. Middleware execution order

Middleware runs **in the order registered**. Mounting order matters: put body parsers before routes, put error handlers **last**.

```
app.use(helmet());            // security headers
app.use(cors());
app.use(express.json());      // body parser
app.use('/api', apiRouter);
app.use(notFound);
app.use(errorHandler);        // last
```

Q40. How do you structure a production Express app?

- Layered: **routes** → **controllers** → **services** → **repositories**.
- Validation (zod/joi) at the controller boundary.
- Centralized error handler returning RFC 7807 (`application/problem+json`).
- DI container (awilix, tsyringe) for testability.
- Request-scoped logging (pino + correlation ID).
- Graceful shutdown.

Q41. `req.params` vs `req.query` vs `req.body`

- `req.params` — URL path variables (`/users/:id` → `req.params.id`).
- `req.query` — URL query string (`?limit=10`).
- `req.body` — request body (requires body-parser middleware).

Q42. Common security middleware

- `helmet` — sets secure HTTP headers.
- `cors` — configures Cross-Origin Resource Sharing.
- `express-rate-limit` — throttle per-IP.
- `express-validator` / `zod` — input validation.
- `hpp` — HTTP parameter pollution protection.

Chapter — NestJS

5. NestJS

Q43. What is NestJS and why use it?

A progressive Node framework written in TypeScript, inspired by Angular. It provides:

- **Dependency Injection** out of the box
- **Modular architecture** (everything is a module)
- **Decorators** for routes, validation, docs
- Built-in support for **GraphQL, WebSockets, microservices, CQRS**
- Testable by design

Q44. Core building blocks

- **Module** — group of related code (`@Module`).
- **Controller** — HTTP route handler (`@Controller`).
- **Provider / Service** — business logic, injectable (`@Injectable`).
- **Pipe** — transform/validate input (`@UsePipes`).
- **Guard** — authorization (`@UseGuards`).
- **Interceptor** — wrap execution (logging, caching, transform response).
- **Filter** — exception handling (`@UseFilters`).

Q45. Request lifecycle in NestJS

```
Middleware → Guards → Interceptors (before) → Pipes → Controller → Service
                                  ↓
                        Interceptors (after) → Filters (if error) → Response
```

Q46. How does DI work in NestJS?

Based on **constructor injection**. Under the hood it reads TypeScript's emitted metadata (`emitDecoratorMetadata`) to know which tokens to inject.

```
@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User) private repo: Repository<User>,
    private logger: Logger,
  ) {}
}
```

Scopes: `DEFAULT` (singleton), `REQUEST`, `TRANSIENT`.

Q47. Custom decorator example

```
export const CurrentUser = createParamDecorator(
  (data: unknown, ctx: ExecutionContext) =>
    ctx.switchToHttp().getRequest().user,
);

@Get('me')
profile(@CurrentUser() user: User) { return user; }
```

Q48. Guards vs Middleware vs Interceptors

Concern	Use
Generic request processing (logging, parsing)	Middleware
Authorization decisions ("can this run?")	Guards
Cross-cutting (timing, caching, transform)	Interceptors
Transforming/validating input	Pipes
Handling thrown exceptions	Filters

Q49. How would you test a NestJS service?

```
const module = await Test.createTestingModule({
  providers: [
    UsersService,
    { provide: getRepositoryToken(User), useValue: mockRepo },
  ],
}).compile();

const service = module.get(UsersService);
```

Chapter — Express Deep Dive (vs NestJS)

25. Deep Dive — Express.js (vs NestJS)

Express is a minimal, unopinionated HTTP framework. Nest is an opinionated framework that uses Express (or Fastify) as the HTTP layer underneath. Many "Express questions" really test how comfortable you are with the raw building blocks Nest hides.

Each Q below answers the Express question first, then includes a **Δ vs Nest** note where the answer differs.

Q236. How does Express actually route a request?

Express maintains an internal **router stack** — an ordered array of layers (each layer has a path matcher and a handler). When a request comes in, Express walks the stack in declaration order; for each layer that matches, it invokes the handler. If the handler calls `next()`, the walk continues; if it calls `next(err)`, Express jumps to the next **error-handling middleware** (one with arity 4: `(err, req, res, next)`).

```
app.use(logger);                      // matches every path
app.get('/users/:id', getUser);      // matches GET /users/:id
app.use('/admin', adminRouter);      // mounts a sub-router
app.use(errorHandler);                // 4-arg, runs on next(err)
```

Routes are just specialized middleware with a method filter. `app.get('/x', h)` is sugar over a layer that matches `GET /x`.

Δ vs Nest: Nest builds the same router (Express or Fastify) at startup from `@Controller` / `@Get` / `@Post` decorators. You don't see the stack, but it's there. Nest's middleware/guards/interceptors/pipes/filters are layered abstractions on top of that single underlying call chain.

Q237. What's the difference between `app.use`, `app.METHOD`, and `Router`?

- `app.use([path], fn)` — applies `fn` to every method on `path` (or all paths if omitted). Used for cross-cutting middleware.
- `app.get/post/put/...` — method-specific terminal handlers.
- `express.Router()` — a mountable, isolated mini-app. Lets you split routes across files and mount them: `app.use('/api/v1', apiRouter)`.

Routers are the closest Express has to Nest's `@Module` — a way to compose route groups. They don't provide DI or lifecycle, just routing isolation.

Δ vs Nest: Nest replaces both with `@Module` + `@Controller`. A controller belongs to a module; the module wires its providers. There is no manual `app.use(router)` — Nest discovers controllers via decorators at startup.

Q238. Express middleware execution order — the gotchas

Order is **declaration order** within an app/router. A few traps:

1. **Body parsers must come before routes that read `req.body`.**

2. **Error middleware** must be declared **last**, after all routes.
3. **Async errors don't bubble automatically in Express 4.** A rejected promise in `async (req,res,next)` is silently swallowed unless you `.catch(next)` or use `express-async-errors`. Express 5 fixes this — async errors propagate to the error handler natively.
4. `app.use('/api', m)` runs `m` for `/api`, `/api/users`, etc. — prefix match, not exact.
5. Mounted routers see paths **relative** to the mount point (`req.path` = `/users`, not `/api/users`).
6. `req.url` mutates during routing (Express strips the matched mount prefix). Use `req.originalUrl` for logging.

Δ vs Nest: Nest enforces a strict pipeline order — **Middleware → Guards → Interceptors (before) → Pipes → Handler → Interceptors (after) → Exception Filters**. You don't choose the order; you choose which type to write. Async errors are always caught — every async handler is awaited and exceptions go through the filter chain.

Q239. Writing async-safe Express code (the `next(err)` problem)

Native Express 4 doesn't await your handler:

```
// BUG: rejection is unhandled, request hangs until the client times out
app.get('/x', async (req, res) => {
  const data = await db.find();
  res.json(data);
});
```

Three fixes:

```
// 1. Wrap manually
app.get('/x', (req, res, next) => handler(req, res).catch(next));

// 2. Wrapper helper
const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).ca
app.get('/x', asyncH(async (req, res) => { ... }));

// 3. Patch Express globally
import 'express-async-errors';
```

In Express **5** (production-ready) this is built-in: returning a rejected promise from a handler/middleware automatically calls `next(err)`.

Δ vs Nest: Nest awaits every handler internally. `throw` from a controller method goes straight to the exception filter — no wrappers needed.

Q240. Production-grade error handling

```
// 4-arg signature — Express recognizes this as an error handler
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);          // delegate to default handle
  const status = err.status || err.statusCode || 500;
  req.log?.error({ err, requestId: req.id }, 'request failed');
  res.status(status).json({
    error: status >= 500 ? 'Internal Server Error' : err.message,
    requestId: req.id,
  });
});

process.on('unhandledRejection', (err) => { req.log.fatal({ err }); process.exi
process.on('uncaughtException',  (err) => { req.log.fatal({ err }); process.exi
```

Rules:

- Distinguish **operational errors** (4xx/5xx you can recover from — keep running) from **programmer errors** (bugs, invariants — crash and let your supervisor restart).
- Never leak internal messages on 5xx.
- Always include a correlation ID.
- Crash on `unhandledRejection` / `uncaughtException` — recovering from these usually leaves the process in a corrupted state.

Δ vs Nest: Nest provides built-in `HttpException` hierarchy and a default exception filter that formats errors as JSON. You can write a custom `@Catch()` filter to override globally or per-controller. The "operational vs programmer" distinction is the same — Nest still lets bugs crash, you still need process-level handlers.

Q241. Production-ready Express folder structure

```
src/
  config/        # env loading + validation (zod / envalid)
  loaders/       # express.ts, db.ts, logger.ts — start order
  routes/        # one router per resource
  controllers/   # request/response shape (HTTP concerns only)
  services/      # business logic (no req/res)
  repositories/  # DB access
  middlewares/   # auth, validate, rateLimit, requestId
  schemas/       # zod / joi validation schemas
  errors/        # AppError, NotFoundError, ...
  utils/
  app.ts         # builds the app, no .listen()
  server.ts      # starts http server, handles signals
```

Two principles:

- **`app.ts` doesn't call `.listen()`.** Tests can `import('./app')` and supertest it without binding a port.
- **Controllers don't talk to the DB.** They call services. Services don't know about HTTP. This is what lets you change Express → Fastify → Lambda without rewriting business logic.

Δ vs Nest: Nest gives you this structure by force — `@Module`, `@Controller`, `@Injectable()` services, repositories. The trade-off: less freedom, less boilerplate, harder to deviate when you actually need to.

Q242. Validation — the Express way

Express has none built-in. Pick one:

- **zod** (current default) — TS-first, infer types from schemas.
- **joi** — popular, mature, no TS inference.
- **express-validator** — chained validators, friendly for simple cases.
- **ajv** — JSON Schema, fastest at runtime.

```
import { z } from 'zod';

const CreateUserSchema = z.object({
  email: z.string().email(),
  age: z.number().int().min(13),
});

const validate = (schema) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) return res.status(400).json({ errors: r.error.flatten() });
  req.body = r.data;
  next();
};

app.post('/users', validate(CreateUserSchema), createUser);
```

Validate at the boundary; trust your types inside.

Δ vs Nest: Nest has built-in `ValidationPipe` (powered by `class-validator` + `class-transformer`). You decorate DTO classes (`@IsEmail()`, `@Min(13)`) and Nest validates the incoming body against the controller's typed parameter automatically. zod-based pipes (`nestjs-zod`) are also common.

Q243. Auth in Express — the building blocks

Two common stacks:

Session + cookie (server-rendered apps):

```
import session from 'express-session';
import RedisStore from 'connect-redis';
app.use(session({
  store: new RedisStore({ client: redis }),
  secret: process.env.SESSION_SECRET,
  cookie: { httpOnly: true, secure: true, sameSite: 'lax', maxAge: 86400_000 },
}));
```

JWT (SPAs, mobile, microservices):

```
import jwt from 'jsonwebtoken';
const auth = (req, res, next) => {
  const token = req.headers.authorization?.replace('Bearer ', '');
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.sendStatus(401); }
};
app.get('/me', auth, (req, res) => res.json(req.user));
```

For OAuth/OIDC, **passport** has 500+ strategies but is older and verbose. Modern apps usually use **openid-client** directly or delegate to a managed identity provider (Auth0, Cognito, Clerk).

Δ vs Nest: Nest wraps the same logic in **Guards** (`@UseGuards(JwtAuthGuard)`) backed by `@nestjs/passport`. The strategy classes are the same Passport strategies, just wired through DI. Cleaner per-route declaration; same underlying mechanics.

Q244. Security middleware checklist

```
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';
import compression from 'compression';
import hpp from 'hpp';

app.disable('x-powered-by');                    // hide Express
app.set('trust proxy', 1);                      // when behind ALB/CloudFront
app.use(helmet());                              // CSP, HSTS, X-Frame-Options,
app.use(cors({ origin: ALLOWED_ORIGINS, credentials: true }));
app.use(express.json({ limit: '100kb' }));     // bound body size
app.use(hpp());                                 // strip duplicated query param
app.use(rateLimit({ windowMs: 60_000, max: 100, standardHeaders: true }));
app.use(compression());
```

Beyond middleware: parameterized queries (no string concat), bcrypt/argon2 for passwords, `secure` + `httpOnly` cookies, secrets from env (never committed), `npm audit` + Dependabot in CI.

Δ vs Nest: identical — Nest wraps the Express server, so `app.use(helmet())` works on the underlying `app.getHttpAdapter().getInstance()`. There's also `@nestjs/throttler` for rate limiting natively.

Q245. Streaming responses & file uploads

Streaming a large response:

```
import { pipeline } from 'node:stream/promises';
app.get('/export', async (req, res) => {
  res.setHeader('Content-Type', 'text/csv');
  await pipeline(db.queryStream('SELECT ...'), csvFormatter(), res);
});
```

`pipeline` propagates errors and cleans up sockets correctly — never use `.pipe()` chains in production.

Uploads: `multer` is the de-facto multipart parser:

```
import multer from 'multer';
const upload = multer({ dest: '/tmp', limits: { fileSize: 10 * 1024 * 1024 } })
app.post('/upload', upload.single('file'), (req, res) => res.json({ path: req.f
```

For large files, do **direct-to-S3** with presigned URLs instead of proxying bytes through your server.

Δ vs Nest: Nest exposes streaming via `StreamableFile` (`return new StreamableFile(stream)`) and uses `@nestjs/platform-express` which wraps multer (`@UseInterceptors(FileInterceptor('file'))`). Same primitives, decorator-based ergonomics.

Q246. WebSockets in Express

Express itself doesn't speak WebSocket — you upgrade the underlying `http.Server`:

```
import { createServer } from 'node:http';
import { WebSocketServer } from 'ws';
const server = createServer(app);
const wss = new WebSocketServer({ server });
wss.on('connection', (ws, req) => { ws.on('message', (m) => ws.send(`echo: ${m}
server.listen(3000);
```

Or `socket.io` for room/broadcast/reconnect features. Behind an ALB, set the listener to allow WebSocket upgrades and the target group's health checks accordingly.

Δ vs Nest: Nest has first-class WebSocket gateways (`@WebSocketGateway` , `@SubscribeMessage('event')`) with adapters for `ws` , `socket.io` , or custom transports. DI works inside gateways; same WS semantics underneath.

Q247. Testing Express APIs

```
import request from 'supertest';
import app from '../src/app';

describe('GET /users/:id', () => {
  it('200 on existing user', async () => {
    const res = await request(app).get('/users/1');
    expect(res.status).toBe(200);
    expect(res.body).toMatchObject({ id: 1 });
  });
  it('404 on missing', async () => {
    await request(app).get('/users/9999').expect(404);
  });
});
```

`supertest` boots the app on an ephemeral port (or accepts a request handler directly), runs HTTP requests, and tears down. Combine with a real DB + transactional rollback per test for integration confidence; reserve mocks for external HTTP / SDKs.

Δ vs Nest: identical pattern with `Test.createTestingModule({ imports: [AppModule] }).compile()` then `app.getHttpServer()` into supertest. Nest also lets you swap providers (`.overrideProvider(MyService).useValue(mock)`) in the testing module without touching production code.

Q248. When to choose Express vs NestJS vs Fastify

Need	Pick
Tiny API, full control, minimal deps	**Express**
Large team, opinionated structure, DI, decorators, microservices boilerplate	**NestJS**
Maximum throughput per core, modern plugin system, schema-first validation	**Fastify**
Lambda-first / serverless	**Fastify or Express** (smaller cold start than Nest)
Existing Express app needs structure but not a rewrite	Stay on Express, adopt **InversifyJS** or **tsyringe** for DI

Performance: Fastify > Express > Nest-on-Express. Nest can run on Fastify (`@nestjs/platform-fastify`) and recovers most of the gap, at the cost of a few incompatible plugins.

Hiring/familiarity is often the bigger factor than benchmarks. A team that knows Nest will ship faster on Nest than on a "faster" Express they have to reinvent every cross-cutting concern in.

Part VII — API Design

Chapter — REST

6. REST API Design

Q50. Design principles of REST

- **Stateless** — server holds no client state; each request is self-contained.
- **Resource-oriented URLs** — nouns, not verbs: `/users/42/orders`, not `/getUserOrders`.
- **HTTP methods have meaning** — GET, POST, PUT, PATCH, DELETE.
- **Use proper status codes**.
- **HATEOAS** — responses link to related actions (in strict REST; rarely done).
- **Content negotiation** via `Accept`.
- **Versioning** via URL, header, or media type.

Q51. HTTP methods — semantics and idempotency

Method	Safe	Idempotent	Typical use
GET	Yes	Yes	Read
HEAD	Yes	Yes	Metadata only
OPTIONS	Yes	Yes	CORS preflight
POST	No	No	Create, RPC-ish
PUT	No	**Yes**	Replace
PATCH	No	No (usually)	Partial update
DELETE	No	Yes	Remove

Idempotent = calling it N times has the same effect as calling it once. **Safe** = no server state change.

Q52. Status codes that matter

- `200` OK — generic success
- `201` Created — new resource; include `Location` header
- `202` Accepted — async, processing later
- `204` No Content — success, empty body (DELETEs)
- `301/302/307/308` — redirects (308 keeps method and body)
- `400` Bad Request — malformed input
- `401` Unauthorized — not authenticated
- `403` Forbidden — authenticated but not allowed
- `404` Not Found
- `409` Conflict — state collision (e.g., duplicate email)
- `410` Gone
- `422` Unprocessable Entity — semantic validation failure
- `429` Too Many Requests — rate-limited
- `500` Internal Server Error
- `502/503/504` — upstream/availability problems

Q53. API versioning strategies

1. **URI path**: `/v1/users` — most visible, easiest.
2. **Header**: `Accept: application/vnd.app.v2+json`.
3. **Query param**: `?version=2` — discouraged.

Q54. Pagination strategies

- **Offset-based** — `?limit=20&offset=100`. Simple but breaks with inserts and scales poorly.
- **Cursor-based (keyset)** — `?limit=20&after=<encoded_cursor>`. Stable, fast for big datasets. **Prefer this at scale.**

```
GET /events?limit=20
{ "data": [...], "next": "eyJpZCI6IjU2In0=" }
```

Q55. Idempotency for POST

POST isn't idempotent, but clients may retry on network failure. Support an `Idempotency-Key` header: on the server, store the key→response mapping for 24h so retries return the original result instead of creating duplicates. (Stripe's standard.)

Q56. How would you design `POST /transfers`?

- Validate `amount > 0`, distinct accounts, currencies match.
- Require `Idempotency-Key`.
- Do it in a DB transaction: lock both accounts (ordered by ID to avoid deadlocks), check balance, update, insert ledger entries.
- Return `201` with transfer ID; enqueue post-processing (notifications) to a queue, never inline.

Chapter — GraphQL

10. GraphQL

Q86. What is GraphQL and how is it different from REST?

A query language and runtime where clients specify **exactly** which fields they need. One endpoint, strongly-typed schema.

Aspect	REST	GraphQL
Endpoints	Many	One
Over/underfetching	Common	Minimal
Versioning	Explicit (`/v1/`)	Evolve schema (deprecate fields)
Caching	HTTP caching out-of-box	Harder; needs extra infra
Error codes	HTTP statuses	Always 200; errors in body
File uploads	Trivial	Requires extension (`graphql-upload`)

Q87. Schema, queries, mutations, subscriptions

```
type User  { id: ID!, email: String!, posts: [Post!]! }
type Post  { id: ID!, title: String!, author: User! }

type Query { user(id: ID!): User }
type Mutation { createPost(input: CreatePostInput!): Post! }
type Subscription { postAdded: Post! }
```

Q88. Resolvers and the N+1 problem

Each field can have a resolver. If a query returns 100 posts, the naive `post.author` resolver runs 100 SQL queries — the GraphQL N+1 problem.

Fix: DataLoader. It batches and caches (per-request).

```
const userLoader = new DataLoader(async (ids) => {
  const users = await db.user.findMany({ where: { id: { in: ids as string[] } }
  const map = new Map(users.map(u => [u.id, u]));
  return ids.map(id => map.get(id as string));
});

// resolver
author: (post, _, ctx) => ctx.loaders.user.load(post.authorId),
```

Q89. Apollo Server setup (Node)

```
const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, {
  context: async ({ req }) => ({
    user: await getUserFromToken(req.headers.authorization),
    loaders: makeLoaders(),
  }),
});
```

Q90. Authorization in GraphQL

- **Field-level** — check in resolvers (or directives like `@auth(role: ADMIN)`).

- **Schema-level** — gate entire types.
- Keep **business rules** in services, not resolvers.
- Avoid exposing internal IDs/raw DB shapes — use DTOs.

Q91. Schema stitching vs Federation

- **Stitching** — merge remote schemas into a gateway. Older, simpler, less scalable.
- **Federation (Apollo)** — each subgraph owns its types; gateway composes them. Better for microservices.

Q92. GraphQL downsides / gotchas

- Query complexity → potential DoS. Use depth limiting, cost analysis, persisted queries.
- HTTP caching is hard (queries are POSTs with varied bodies).
- Client must know GraphQL; learning curve.
- Introspection can leak schema — disable in prod if unused.

Part VIII — Authentication, Authorization, and Security

7. Authentication, Authorization & Security

Q57. JWT — structure and how it works

A JWT has three base64url-encoded parts separated by dots: `header.payload.signature`.

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkFsaWNlIn0.d8Zj...
```

- **Header** — `{ "alg": "HS256", "typ": "JWT" }`
- **Payload** — claims: `sub`, `iat`, `exp`, `iss`, `aud`, plus custom
- **Signature** — HMAC or RSA over `base64(header).base64(payload)`

The server **verifies** the signature with a secret (HS256) or public key (RS256). If valid and not expired, trust the payload.

Q58. JWT pros, cons, and pitfalls

Pros: stateless, easy to use across services, self-contained. **Cons:**

- Cannot be revoked before expiry without extra infra (blocklist).
- If the payload is large, it bloats every request.
- Secret leak = total compromise.
- Algorithm confusion attack (`alg: none`, swapping RS256→HS256 with public key as secret) — always pin the algorithm on verify.

Q59. Access tokens vs refresh tokens

- **Access token** — short-lived (5–15 min), sent on every request.
- **Refresh token** — long-lived (days/weeks), stored securely, used to mint a new access token.

Typical flow:

1. Login → server returns `{ accessToken, refreshToken }`.
2. On 401, client uses refresh token at `/auth/refresh`.
3. Rotate refresh token on each use; store rotations to detect theft.

Where to store tokens in a browser?

- Access token → memory (never localStorage; XSS steals it).
- Refresh token → **httpOnly, Secure, SameSite=Strict cookie**.

Q60. Session-based auth vs JWT — when to use which?

Factor	Sessions	JWT
Storage	Server (DB/Redis)	Client
Revocation	Trivial (delete session)	Hard without blocklist
Scalability	Needs sticky session or shared store	Stateless
Use case	Web app with a single backend	Microservices, mobile, 3rd-party APIs

For a classic web app, cookies + server session is usually the simpler, safer choice.

Q61. OAuth 2.0 — core flows

- **Authorization Code + PKCE** — for web/SPAs/mobile. The gold standard.
- **Client Credentials** — machine-to-machine.
- **Device Code** — smart TVs, CLI tools.
- **Refresh Token** — getting new access tokens.
- **Implicit** — deprecated; don't use.
- **Password** — deprecated; don't use.

Q62. Password hashing

- **Never** store plaintext. **Never** use MD5 or SHA-1 or SHA-256 directly.
- Use **bcrypt** (good default), **scrypt**, or **argon2** (best).

```
import bcrypt from 'bcrypt';
const hash = await bcrypt.hash(password, 12);  // 12 rounds = ~250ms
const ok = await bcrypt.compare(password, hash);
```

Bcrypt generates the salt; the hash encodes algorithm, rounds, salt, and digest in one string.

Q63. OWASP Top 10 — what does a senior need to know?

1. **Broken Access Control** — always check authz **per resource**, not just "is logged in".
2. **Cryptographic Failures** — HTTPS everywhere, don't roll your own crypto.
3. **Injection** — parameterized queries, never string-concat SQL.
4. **Insecure Design**.
5. **Security Misconfiguration** — disable stack traces in prod, lock down defaults.
6. **Vulnerable/Outdated Components** — `npm audit`, Dependabot.
7. **Identification and Authentication Failures**.
8. **Software and Data Integrity Failures** — npm supply chain, SRI.
9. **Security Logging and Monitoring Failures**.
10. **Server-Side Request Forgery (SSRF)** — validate outgoing URLs, block internal IPs.

Q64. SQL injection prevention

Always use parameterized queries.

```
// ❌ Vulnerable
db.query(`SELECT * FROM users WHERE email = '${email}'`);

// ✅ Parameterized
db.query('SELECT * FROM users WHERE email = $1', [email]);
```

ORMs do this automatically, but the moment someone uses a raw query, you're back at risk.

Q65. XSS and CSRF — what are they?

XSS (Cross-Site Scripting) — attacker injects JS into a page another user views. Defense: escape output, use Content-Security-Policy, `httpOnly` cookies.

CSRF (Cross-Site Request Forgery) — attacker tricks an authenticated user's browser into submitting a request. Defense: `SameSite` cookies, CSRF tokens, check `Origin` / `Referer`.

Q66. CORS explained

CORS is a browser mechanism to allow a site on origin A to call an API on origin B. For "simple" requests, the browser just checks the `Access-Control-Allow-Origin` response header. For "preflighted" requests (custom headers, non-GET/POST, etc.), the browser first sends an `OPTIONS` request.

```
app.use(cors({
  origin: 'https://app.example.com',
  credentials: true,              // allow cookies
  methods: ['GET','POST','PUT','DELETE'],
}));
```

Common mistake: `Access-Control-Allow-Origin: *` **with** `credentials: true` — browsers reject this; you must echo the specific origin.

Q67. Rate limiting strategies

- **Fixed window** — N requests per minute. Bursts at boundaries.
- **Sliding window log** — accurate, memory-hungry.
- **Sliding window counter** — balance of both.
- **Token bucket** — allows bursts up to bucket size.
- **Leaky bucket** — enforces constant rate.

In Node: `express-rate-limit` + Redis store for distributed deployments.

Q68. Common Node-specific vulnerabilities

- **Prototype pollution** — merging untrusted objects into `Object.prototype`.
- **ReDoS** — catastrophic backtracking in regex against attacker input.
- **Command injection** via `child_process.exec` with interpolated user input — use `execFile` + args array.
- **Path traversal** — accepting `../../etc/passwd` into `fs.readFile` — always `path.resolve` and verify prefix.
- **Open redirect** — `res.redirect(req.query.next)` → validate allowlist.

Part IX — Databases and ORMs

Chapter — Databases (SQL & NoSQL)

8. Databases — SQL & NoSQL

Q69. SQL vs NoSQL — when to choose what?

SQL (PostgreSQL, MySQL)

- Structured data, well-defined relations
- Strong consistency, ACID
- Complex queries, joins, aggregations
- Financial data, transactional systems

NoSQL (MongoDB, DynamoDB, Cassandra)

- Flexible/evolving schema
- Horizontal scale, massive datasets
- Document model matches app (nested, denormalized)
- Analytics, caching, real-time feeds, event logs

Reality: most backends end up using **both** — Postgres as source of truth, Redis for caching, Mongo/Dynamo for specific workloads.

Q70. ACID properties

- **Atomicity** — all or nothing.
- **Consistency** — DB moves from one valid state to another (constraints hold).
- **Isolation** — concurrent transactions don't see each other's intermediate state.
- **Durability** — once committed, it's persisted (survives crashes).

Q71. CAP theorem

Under a network partition (P), you must choose between **Consistency** and **Availability**:

- **CP** systems (MongoDB default, HBase) — refuse requests rather than return stale data.
- **AP** systems (Cassandra, DynamoDB) — return possibly-stale data; reconcile later.

In practice you also think about **PACELC** — in the absence of partition, tradeoff is Latency vs Consistency.

Q72. Transaction isolation levels

From weakest to strongest:

- **Read Uncommitted** — dirty reads allowed (rare).
- **Read Committed** — Postgres default. No dirty reads; non-repeatable reads possible.
- **Repeatable Read** — MySQL default. Snapshot per transaction.
- **Serializable** — as if transactions ran one after another.

Phenomena:

- Dirty read: read uncommitted data.
- Non-repeatable read: same row has different values across reads.
- Phantom read: same range query returns new rows.

Q73. Indexes — how do they work?

An index is a secondary data structure (usually a **B-tree**) that lets the DB find rows by key without a full scan. Tradeoffs: faster reads, slower writes, more disk space.

Types:

- **B-tree** — default, good for equality and range.
- **Hash** — equality only, fast.
- **GIN** (Postgres) — full-text, JSONB, arrays.
- **BRIN** — for huge, naturally-ordered tables (append-only logs).
- **Partial** — index only rows matching a predicate.
- **Composite** — multi-column; order matters. `(a, b)` helps `WHERE a = ?` and `WHERE a = ? AND b = ?`, **not** `WHERE b = ?`.

Q74. What is the N+1 query problem?

Fetching a list, then firing one query per item to get relations:

```
// N+1
const posts = await db.query('SELECT * FROM posts');
for (const p of posts) {
  p.author = await db.query('SELECT * FROM users WHERE id = $1', [p.author_id])
}
```

Fix: **JOIN**, **IN**-batching, or DataLoader.

```
const posts   = await db.query('SELECT * FROM posts');
const ids     = [...new Set(posts.map(p => p.author_id))];
const authors = await db.query('SELECT * FROM users WHERE id = ANY($1)', [ids])
const map     = new Map(authors.map(a => [a.id, a]));
posts.forEach(p => p.author = map.get(p.author_id));
```

Q75. Database normalization vs denormalization

Normalization (1NF–3NF+) — eliminate redundancy; update in one place. Good for OLTP.
Denormalization — duplicate data for read performance. Good for OLAP, dashboards, NoSQL.

Pragmatic answer: normalize first, denormalize when metrics demand it.

Q76. MongoDB — when to embed vs reference?

- **Embed** when the child: is only accessed via the parent, doesn't grow unbounded, is small.
- **Reference** when the child: is independently queryable, shared, large, or grows unbounded.

The 16 MB document limit and "prefer working set in RAM" rule shape most design choices.

Q77. MongoDB aggregation pipeline

```
db.orders.aggregate([
  { $match: { status: 'paid' } },
  { $group: { _id: '$customerId', total: { $sum: '$amount' }, count: { $sum: 1
  { $sort: { total: -1 } },
  { $limit: 10 },
  { $lookup: { from: 'customers', localField: '_id', foreignField: '_id', as: '
]);
```

Stages you should know: `$match`, `$project`, `$group`, `$sort`, `$limit`, `$unwind`, `$lookup`, `$facet`, `$addFields`.

Q78. PostgreSQL features worth knowing

- **JSONB** — indexed JSON with full query support.
- **Array types** — first-class.
- **CTEs** and recursive CTEs (`WITH RECURSIVE`).
- **Window functions** (`ROW_NUMBER`, `RANK`, `LAG`, `LEAD`).
- **Materialized views**.
- **Row-level security**.
- **Triggers, rules**.
- **Partitioning**.
- `EXPLAIN ANALYZE` — the most important command for performance work.

Q79. Replication and sharding

- **Replication** — copies of the same data on multiple nodes. Gives HA and read scaling. Types: async (risk of data loss on failover), sync (slower).
- **Sharding** — splits data by key across nodes. Gives write scaling but complicates joins and transactions.

Q80. Connection pooling — why does it matter in Node?

Every DB connection is expensive. Node's single process would exhaust connections quickly under load. Pools (built into `pg`, `mysql2`, Mongo drivers) reuse a fixed number of connections — typically 10–20 per process. When running N replicas, multiply: `N × pool_size ≤ db_max_connections`.

For serverless (Lambda), use a **proxy** like RDS Proxy or PgBouncer to reuse pools across invocations.

Chapter — ORMs (Prisma, Sequelize, TypeORM)

9. ORMs — Prisma, Sequelize, TypeORM

Q81. Compare the three

Feature	Prisma	Sequelize	TypeORM
Type safety	Excellent (generated)	Weak	Good (decorators)
Schema source	`schema.prisma` DSL	JS models	Decorated classes
Migrations	First-class	CLI	CLI
Active Record / DM	Data Mapper	Active Record	Both
Raw SQL escape	`$queryRaw`	`sequelize.query`	`query`
Community	Fastest growing	Oldest, huge	Stable, NestJS default
Downsides	Extra generate step, opinionated	Type-unsafe, clunky	Maintenance concerns

Q82. Prisma example

```
model User {
  id        String   @id @default(cuid())
  email     String   @unique
  posts     Post[]
  createdAt DateTime @default(now())
}
```

```
const user = await prisma.user.create({
  data: { email: 'a@b.com', posts: { create: [{ title: 'Hi' }] } },
  include: { posts: true },
});
```

Q83. Migrations — best practices

- **Never** edit a migration that's been applied in production. Create a new one.
- Forward-compatible deploys: **expand** → **migrate** → **contract**.
  1. Add new column nullable.
  2. Dual-write.
  3. Backfill.
  4. Switch reads.
  5. Drop old column in a later deploy.
- Keep migrations small and fast — avoid long table locks.

Q84. Transactions with an ORM

Prisma:

```
await prisma.$transaction(async (tx) => {
  await tx.account.update({ where: { id: from }, data: { balance: { decrement:
  await tx.account.update({ where: { id: to   }, data: { balance: { increment:
  if ((await tx.account.findUnique({ where: { id: from } }))!.balance < 0) thro
});
```

TypeORM has `queryRunner.startTransaction()` / `commitTransaction()` and `dataSource.transaction(cb)`.

Q85. When would you drop the ORM and write raw SQL?

- Reports with complex joins, window functions, CTEs.
- Bulk operations (`INSERT ... ON CONFLICT` , `COPY`).
- Queries where the ORM generates inefficient SQL (check `EXPLAIN`).
- DB-specific features (PostgreSQL ranges, JSONB paths, `LATERAL` joins).

Chapter — PostgreSQL Deep Dive

19. Deep Dive — PostgreSQL

Q159. MVCC — how does it actually work?

Postgres uses **Multi-Version Concurrency Control**: every row has hidden columns `xmin` (the transaction that inserted it) and `xmax` (the one that deleted/updated it). UPDATE doesn't overwrite — it writes a **new tuple version** and sets `xmax` on the old one.

A transaction sees only tuples whose `xmin` committed before it started and whose `xmax` either didn't commit or is in the future, per its **snapshot**.

Consequences:

- **Readers never block writers, writers never block readers.**
- Old versions accumulate (**bloat**) until VACUUM removes them.
- Long-running transactions hold back the **xmin horizon**, preventing VACUUM from cleaning anything newer than them. This is the #1 cause of unexpected bloat.

Q160. VACUUM, autovacuum, and bloat

VACUUM marks dead tuples as reusable space (does **not** return space to the OS). **VACUUM FULL** rewrites the whole table — locks it exclusively. Avoid in production; use `pg_repack` instead.

Autovacuum runs in the background, triggered when the dead-tuple count exceeds `autovacuum_vacuum_scale_factor * rows + threshold`. Defaults are too lazy for high-write tables — tune per-table:

```
ALTER TABLE orders SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.01
);
```

HOT updates (Heap-Only Tuple) — if no indexed column changes and there's room in the same page, the new version stays in the same block and indexes don't have to be touched. Maximize HOT by leaving fillfactor headroom: `WITH (fillfactor = 80)`.

Monitor bloat via `pg_stat_user_tables.n_dead_tup` and the `pgstattuple` extension.

Q161. WAL, checkpoints, and replication

WAL (Write-Ahead Log) — every change is appended to WAL before being applied to data pages. Crash recovery replays WAL from the last checkpoint.

Checkpoint flushes dirty pages to disk and trims WAL. Tuned via `checkpoint_timeout` (default 5min) and `max_wal_size`. Aggressive checkpoints = small WAL but heavy I/O spikes; relaxed checkpoints = recovery is slower.

Replication ships WAL to replicas:

- **Streaming replication (physical)** — byte-for-byte copy of WAL. Replicas are read-only standbys, identical layout.
- **Logical replication** — decodes WAL into row-level changes (INSERT/UPDATE/DELETE) per publication. Replicas can have a different schema, version, or only a subset of tables. Used for zero-downtime upgrades and CDC pipelines.
- **Replication slots** — pin WAL on the primary until consumers ack. **Always monitor** `pg_replication_slots.confirmed_flush_lsn` — an abandoned slot will fill the disk.

Q162. Index types — when to use which

Type	Use case
B-tree (default)	Equality, ranges, ORDER BY, LIKE 'prefix%'
Hash	Equality only. Rarely better than B-tree; smaller for huge equality-only loads.
GIN	"Many values per row" — JSONB, arrays, full-text (`tsvector`), trigrams (`pg_trgm`)
GiST	Geometric, range, trigram, full-text. Lossy but flexible.
SP-GiST	Non-balanced trees (quad-trees, radix). Niche.
BRIN	Huge, naturally-ordered tables (time-series). Block-range index — tiny but coarse.
Bloom (extension)	Multi-column equality where any combination of columns may be queried

Partial indexes index only a subset (`WHERE deleted_at IS NULL`) — smaller, faster. **Expression indexes** index a function value (`CREATE INDEX ON users (lower(email))`).

Covering indexes include extra columns (`INCLUDE (name, email)`) for index-only scans.

Q163. Reading EXPLAIN ANALYZE

```
Nested Loop  (cost=0.43..56.75 rows=1 width=44) (actual time=0.025..0.832 rows=
  ->  Index Scan using orders_user_id_idx on orders  (...) (actual rows=12 ...)
        Index Cond: (user_id = 42)
  ->  Index Scan using users_pkey on users  (...) (actual rows=1 loops=12)
        Index Cond: (id = orders.user_id)
Planning Time: 0.140 ms
Execution Time: 0.901 ms
```

What to look for:

- **`actual rows` vs `estimated rows`** — if off by 100×, statistics are stale (`ANALYZE` the table) or correlated columns need extended stats (`CREATE STATISTICS`).
- **`Seq Scan` on a large table with a filter** — missing or unused index.
- **`loops=N`** in nested loops — the inner side runs N times. If N is huge, a `Hash Join` or `Merge Join` would beat it.
- **`Buffers: shared hit=… read=…`** (with `BUFFERS` flag) — read = disk fetch. High `read` = cold cache or table too big for shared_buffers.
- **`Sort Method: external merge Disk`** — `work_mem` is too small for the sort.

Use `EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT TEXT)` and paste into explain.dalibo.com.

Q164. Partitioning

Native **declarative partitioning** since PG 10:

```
CREATE TABLE events (id bigserial, occurred_at timestamptz NOT NULL, ...)
  PARTITION BY RANGE (occurred_at);

CREATE TABLE events_2026_04 PARTITION OF events
  FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');
```

Strategies:

- **RANGE** — time-series (most common).

- **LIST** — by tenant, region.
- **HASH** — even spread when no natural key.

Wins: drop old data with `DROP TABLE events_2025_01` (instant vs. DELETE), partition pruning makes scans target one partition, autovacuum scales per partition.

Watch out: every query must include the partition key to benefit from pruning. Unique constraints must include the partition key. Use `pg_partman` to automate creating/retiring partitions.

Q165. Row-level locks and the SKIP LOCKED queue pattern

Postgres can be a job queue without Redis/Kafka:

```
BEGIN;
SELECT id, payload FROM jobs
  WHERE status = 'pending'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 10;
-- process the rows
UPDATE jobs SET status = 'done' WHERE id = ANY($1);
COMMIT;
```

`FOR UPDATE` takes a row lock; `SKIP LOCKED` makes other workers grab the next free row instead of blocking. Throughput scales linearly until you hit lock contention or vacuum overhead. Used in production by GitLab, Sidekiq Pro, graphile-worker.

Q166. Advisory locks

App-level locks not tied to any row:

```
SELECT pg_try_advisory_lock(123);  -- non-blocking
SELECT pg_advisory_xact_lock(123); -- released at COMMIT/ROLLBACK
```

Use cases: cluster-wide singletons (only one cron worker runs at a time), preventing concurrent migrations, leader election in small clusters.

Q167. JSONB — indexing and pitfalls

`JSONB` stores parsed JSON in a binary format. Indexable with GIN:

```
CREATE INDEX ON docs USING GIN (data);                    -- supports @> ? ?| ?
CREATE INDEX ON docs USING GIN (data jsonb_path_ops);     -- smaller, supports
CREATE INDEX ON docs ((data->>'status'));                 -- B-tree on one fiel
```

When to use JSONB: truly schemaless data, sparse attributes, denormalized read models. **When NOT to:** anything you'll filter, sort, or aggregate by frequently — promote those to columns. JSONB columns can't have foreign keys, statistics on inner fields are weak, and updates rewrite the whole document (no partial-field bloat avoidance).

Q168. LISTEN / NOTIFY

Lightweight pub/sub without an external broker:

```
NOTIFY orders, '{"id":42}';
LISTEN orders;
```

Limits: payload < 8 KB, messages dropped if no listener is connected, no persistence, no replay. Good for **cache invalidation** signals, low-volume real-time updates. For anything durable, use a queue table + LISTEN as a wake-up signal.

Q169. PgBouncer — pooling modes that matter

Mode	Pool reuse boundary	What breaks
Session	Connection close	Nothing (transparent) — minimal benefit
Transaction	Each COMMIT	Session features: `SET`, prepared statements (pre-PG14), `LISTEN`, advisory locks across statements
Statement	Each statement	Multi-statement transactions. Don't use.

Transaction mode is the standard. Common gotchas:

- Don't `SET search_path` on the session — set it per query or via `pgbouncer.ini`.
- `LISTEN` doesn't work; use a dedicated direct connection.

- ORMs that rely on server-side prepared statements need the protocol-level prepared statement support added in PgBouncer 1.21+.

Sizing: app pool = `(transaction_pool_size / app_replicas)`, and Postgres `max_connections` ≥ `pool_size * pgbouncer_instances + reserved`.

Q170. Common Postgres pitfalls at scale

- **`SELECT count(*) FROM big_table`** is slow (must scan to count visible tuples). Use approximations: `pg_class.reltuples` or maintain a counter table.
- **`OFFSET 100000 LIMIT 20`** — keyset pagination wins (`WHERE created_at < $cursor ORDER BY created_at DESC`).
- **`ORDER BY random() LIMIT 1`** — full scan + sort. Use `TABLESAMPLE` or precomputed random IDs.
- **Long transactions** — block VACUUM, accumulate locks, kill replication. Set `idle_in_transaction_session_timeout` and `statement_timeout`.
- **Index on every column** — write amplification. Audit with `pg_stat_user_indexes` (`idx_scan = 0` = unused).
- **DDL during traffic** — most ALTERs take an `ACCESS EXCLUSIVE` lock. Use `CREATE INDEX CONCURRENTLY`, `ALTER TABLE … ADD COLUMN … (no default in PG ≥11)`, and `lock_timeout`.
- **`NOT IN (subquery)`** — NULLs make it return wrong results. Use `NOT EXISTS`.
- **`enum` types** — adding a value is fine, removing requires recreating the type. Prefer a lookup table for anything that may evolve.
- **TOAST surprises** — large field values get stored out-of-line and (de)compressed; a slow query may be paying for TOAST decompression, not the join.

Part X — System Design and Architecture

Chapter — System Design Fundamentals

12. System Design & Architecture

Q100. Monolith vs Microservices

Monolith pros: simpler ops, simpler transactions, lower latency between modules, easier local dev. **Microservices pros:** independent deploys, scale critical services separately, team autonomy, fault isolation. **Cost:** network complexity, data consistency, observability, deployment infrastructure.

Rule: start with a **modular monolith**. Split services when team/scale friction demands it.

Q101. Synchronous vs asynchronous communication

- **Sync (HTTP, gRPC)** — simple, but couples availability.
- **Async (queue, event bus)** — decoupled, retriable, but eventual consistency.

Rule: **orchestration for synchronous workflows, choreography + events for decoupled systems.**

Q102. Message queues — why use them?

- Decouple producers and consumers.
- Handle traffic spikes (buffering).
- Retries with backoff and DLQ (dead-letter queue).
- Enable async workflows (email, thumbnails, reports).

```
Web App → SQS → Worker → DB + S3
```

Q103. At-least-once vs exactly-once delivery

Most queues give **at-least-once**. Design your consumers to be **idempotent** — safe to process the same message twice. Use a unique message ID + dedupe store (Redis set, DB constraint).

Q104. Caching strategies

- **Cache-aside (lazy loading)** — app checks cache, then DB on miss, then writes back.
- **Read-through** — cache sits in front of DB transparently.
- **Write-through** — write to cache and DB simultaneously.
- **Write-behind** — write to cache, flush to DB async.

Pitfalls: stale data, cache stampede, thundering herd. Mitigations: short TTLs, jittered TTLs, request coalescing (single-flight).

Q105. Redis — use cases

- Session store
- Rate limiter (atomic `INCR` + `EXPIRE`)
- Distributed lock (`SET NX PX`; for correctness, use Redlock)
- Pub/sub
- Queues (`LPUSH` / `BRPOP`, or Streams)
- Leaderboards (sorted sets)
- Geospatial queries
- Caching

Q106. Horizontal vs vertical scaling

- **Vertical** — bigger box. Cheap until it isn't; hard limit.
- **Horizontal** — more boxes. Requires stateless design, load balancing, shared data layer.

Q107. Load balancer layers

- **L4 (TCP)** — fast, port-based (AWS NLB).
- **L7 (HTTP)** — can route by path/header/cookie, TLS termination, WAF (AWS ALB, NGINX).

Session stickiness only when needed (WebSockets); otherwise prefer stateless + external session store.

Q108. Database scaling

- **Vertical** — bigger instance.
- **Read replicas** — route reads to replicas, writes to primary.
- **Partitioning** — split tables by range/hash (same DB).
- **Sharding** — split across DBs.
- **CQRS** — separate read model (denormalized, indexed for queries) from write model.

Q109. Eventual consistency — how do you deal with it?

- Show optimistic UI updates and reconcile.
- Use **event sourcing** to rebuild views.
- Expose **idempotent operations** and retries.
- Use **sagas** or the **outbox pattern** for distributed transactions.

Q110. Outbox pattern

When you need to both write to the DB and publish an event without losing either:

1. In the same DB transaction, insert the event into an `outbox` table.
2. A separate poller reads `outbox`, publishes to the broker, marks as sent.

This avoids the dual-write problem (DB commits but broker publish fails, or vice versa).

Q111. Circuit breaker pattern

Wrap remote calls with a breaker that trips (fails fast) after repeated failures. States: **closed** → **open** → **half-open**. Prevents cascading failures and gives the downstream service time to recover. Libraries: `opossum`.

Q112. Design Twitter (high level)

- **Write path**: tweet → append to user's timeline → fan-out on write to a limited follower set. For celebrities (> N followers) use fan-out on read.
- **Timeline**: precomputed in Redis; merged with recent live tweets.
- **Storage**: Cassandra/Dynamo for tweets, sharded by user ID.
- **Search**: Elasticsearch.
- **Media**: S3 + CDN.
- **Rate limits**: Redis-backed token bucket per user.

Q113. Design a URL shortener

- Generate short keys with base62 over an auto-increment ID, or use a hash (collision handling needed).
- Store in key-value DB (Redis/Dynamo) for O(1) lookup.
- Edge-cache popular URLs in CDN.
- Analytics via async queue → data warehouse.
- Expirations via TTL.

Chapter — System Design Case Studies

21. Deep Dive — System Design Case Studies

For every case study: **clarify** (functional + non-functional reqs, scale numbers) → **back-of-the-envelope** (QPS, storage) → **API + data model** → **architecture** → **bottlenecks + scale-out** → **failure modes**. Talk through tradeoffs, don't just present a final picture.

Q186. Design a distributed rate limiter

Reqs: 100 req/s per API key, multi-region, sub-millisecond decision.

Algorithm choices:

- **Fixed window** — `INCR key:userId:minute, EXPIRE 60`. Easy, but bursts at boundaries (200 req in 1 sec across the boundary).
- **Sliding window log** — store every request timestamp. Accurate, expensive memory.
- **Sliding window counter** — weighted average of current + previous window. Good balance.
- **Token bucket** — `tokens = min(capacity, tokens + (now - last_refill) * rate); if tokens >= cost, deduct`. Handles bursts naturally.

Architecture:

```
Client → API Gateway / Envoy → Redis cluster (token bucket per key)
                             → Local in-process LRU cache (allow N before con
```

Use Redis with a **Lua script** so check + decrement is atomic. For multi-region, run a Redis cluster per region and accept that limits are per-region (or use CRDT counters / Cloudflare-style approximate global counters).

Failure mode: Redis down → fail open (allow) for availability, fail closed for security-critical limits. Always have a circuit breaker.

Q187. Design a real-time chat system (WhatsApp-scale)

Reqs: 1B users, 100B msgs/day, deliver in <1s, offline delivery, group chats up to 1000 members.

Connection layer:

- Persistent **WebSocket** connections terminated at edge gateways. ~1M concurrent per box with C/Go (Node ~200k).
- Sticky sessions or a **presence registry** (Redis) mapping `userId → gatewayId`.

Data model:

- Messages partitioned by `chatId` in Cassandra/ScyllaDB (timeseries-friendly, append-heavy).
- Per-user **inbox** (a queue of message refs) for offline delivery.
- Read receipts as separate writes — don't update the message row.

Send flow:

1. Sender posts to the gateway → fan-out service.
2. Fan-out service writes the message (single source of truth), then for each recipient: look up gateway → push if online; else queue in their inbox.
3. Recipient ack updates the read receipts table.

Group chats — fan-out-on-write is fine up to ~1000; for larger groups (channels), switch to **fan-out-on-read** (recipient pulls).

End-to-end encryption (WhatsApp's Signal protocol) — server only sees ciphertext + routing info; key exchange is per-pair. Forward secrecy via ratcheting.

Q188. Design a payment system / ledger

Hard reqs: never lose a payment, never double-charge, full auditability.

Data model — double-entry ledger:

```
entries(id, account_id, amount, currency, transaction_id, posted_at)
-- every transaction inserts two+ entries that sum to zero
balances = SUM(entries.amount) per account_id  -- materialized for speed
```

Never UPDATE entries — they're immutable. A "refund" is a new transaction that reverses the original.

Transactions: wrap entry inserts in a single DB transaction with **SERIALIZABLE** isolation, or use balance check + `UPDATE … WHERE balance >= amount` (optimistic).

Idempotency: every API call carries `Idempotency-Key`; the (key, transaction_id) is unique. Replays return the original transaction.

External charges (Stripe etc.): outbox pattern — write a `pending_charge` row and an outbox event in one tx; a worker calls Stripe and updates status. Stripe webhook reconciles.

Reconciliation: nightly job sums entries → must equal sum of balances. Any drift = page oncall.

Why not eventual consistency for balances? Overdrafts are unrecoverable; show "pending" to the user but never let two debits both pass the balance check.

Q189. Design a notification service

Channels: push (APNs/FCM), email (SES/SendGrid), SMS (Twilio), in-app.

Architecture:

```
Producer services ─► Kafka topic: notifications
                                 ↓
                        Notification Router
                    ┌──────────┼──────────┐
                    ▼          ▼          ▼
               Push worker  Email worker  SMS worker
                    ↓          ↓          ▼
                  APNs        SES       Twilio
```

Components:

- **Preferences store** — per-user channel opt-in/out, quiet hours, locale.
- **Templating** — store templates by `(template_id, locale, channel)`; render at send time.

- **Throttling** — per-user (don't spam) and per-tenant (fairness).
- **Deduplication** — `(userId, eventId)` key with TTL prevents duplicate pushes if upstream retries.
- **Delivery tracking** — store `sent`, `delivered`, `clicked`, `bounced` from provider webhooks.

Failure handling: providers fail. Retry with backoff per channel; after N retries → DLQ → manual triage. Bounce/complaint webhooks must auto-disable a destination to keep sender reputation healthy.

Q190. Design a search system (e.g. product catalog)

Stack: OpenSearch / Elasticsearch + a sync pipeline from your source-of-truth DB.

Sync:

- **CDC** (Debezium → Kafka → indexer) — near real-time, handles deletes correctly.
- Or batch reindex nightly + delta updates.

Indexing:

- Separate **read index** and **build index**; flip an alias atomically when reindexing.
- Multi-field mappings: `name.raw` (keyword) for sorting, `name.text` (analyzed) for search, `name.ngram` for typo tolerance.
- Use **synonyms** and **stemmers** per locale.

Query:

- `multi_match` across name/description with field boosts.
- **Function score** for popularity boost (`log1p(sales) + recency_decay`).
- Filters (category, price) as `filter` clauses (cached, no scoring).

Ranking iteration: log queries + clicks; train a learning-to-rank model offline; serve via OpenSearch LTR plugin.

Pitfalls: mapping changes require reindex; field explosion (one field per dynamic attribute) blows memory; deep pagination is slow — use `search_after`.

Q191. Design a feed / timeline

Two strategies:

- **Pull (fan-out-on-read)** — at read time, query top-N posts from each followed account, merge in memory. Good for users with 10k followees, bad for celebrities (you do the work at every read).
- **Push (fan-out-on-write)** — when user posts, write a row to each follower's timeline. Reads are O(1). Bad for celebrities (write 100M rows per post).
- **Hybrid (Twitter):** push for normal users, pull for celebrities, merge at read. Caches the merged timeline.

Storage: timeline = Redis sorted set per user, capped at last ~1000 entries. Cold pages → DB.

Ranking: chronological is easy. ML ranking adds a feature store, model serving, and an A/B framework.

Edge cases: unfollow must purge entries; deleted post must invalidate caches; new follow must backfill.

Q192. Design a URL shortener at scale

API: `POST /shorten {url}` → `{short: "abc12"}`; `GET /abc12` → 301 redirect.

ID generation: counter → base62 encode. Options:

- Single Postgres sequence (simple, central).
- Snowflake-style 64-bit ID (timestamp + machine + counter).
- Hash(url) — collisions need probing.

Storage: KV store (DynamoDB / Redis) keyed by short code → URL + metadata. Reads dominate writes 100:1.

Cache: CloudFront / CDN edge caches the redirect (`Cache-Control: public, max-age=86400`). Origin only sees cache misses + writes.

Analytics: redirect handler emits an event (Kafka/Kinesis); aggregated downstream. Don't synchronously write to a clicks table — it's the slowest part of the request.

Custom URLs / spam: uniqueness check (`INSERT ... ON CONFLICT`), bloom filter for negative cache, abuse pipeline scanning incoming URLs against threat feeds.

Q193. Design a file upload service (S3 multipart-style)

Direct-to-S3 (preferred):

1. Client requests `POST /uploads` → server returns a **presigned multipart upload URL** + upload ID.
2. Client splits file into chunks (≥5MB), uploads each part directly to S3 with the presigned URL. Resumable: re-upload only failed parts.
3. Client calls `POST /uploads/:id/complete` → server calls S3 `CompleteMultipartUpload`.

Server never proxies bytes — saves bandwidth and CPU.

Server-mediated (legacy / non-S3): write chunks to disk; reassemble on completion; checksum (`Content-MD5` per part); virus scan via Lambda/ClamAV.

Considerations: lifecycle rule to abort incomplete multipart uploads after N days (otherwise you pay for orphaned parts), per-user quota, MIME sniffing on the server (not the `Content-Type` header).

Q194. Design a job scheduler (cron at scale)

Naive: `cron` on one box. Single point of failure.

Distributed:

- **Schedule store** (Postgres): `(job_id, schedule_cron, next_run_at, owner)`.
- A worker pool polls `WHERE next_run_at <= now() FOR UPDATE SKIP LOCKED LIMIT N`, executes, updates `next_run_at`.
- Use advisory lock per job to prevent concurrent runs.
- For very high job counts, partition by `hash(job_id)` and assign partitions via consistent hashing.

Higher-level options:

- **Temporal / Cadence** — durable workflows with retries, signals, sagas. Default for new systems with non-trivial scheduling.
- **AWS EventBridge Scheduler** — managed, scales to millions of schedules.
- **Quartz (JVM)**, **graphile-worker** (Node + Postgres), **Sidekiq** (Ruby + Redis).

Hard parts: missed runs after downtime (catch up vs skip?), timezone & DST, jobs that overlap their own runtime, observability (a job failing silently is worse than a normal request failure).

Q195. Design an audit log

Reqs: tamper-evident, queryable, retained 7 years, rare reads.

Write path:

- Every mutating action emits an event: `(actor, action, resource, before, after, ts, request_id)`.
- Events go to an **append-only** store. Immutable schema; never UPDATE/DELETE.
- For tamper-evidence, hash-chain entries: `hash_n = sha256(hash_{n-1} || event_n)`. Periodically anchor the chain hash externally (S3 Object Lock, blockchain, signed by HSM).

Storage: S3 in Parquet (cheap, cold, scannable via Athena). Hot index in OpenSearch for recent ~30 days.

Reads: rare — admin and compliance queries. Optimize for write throughput, not read latency.

Retention: S3 lifecycle policies + Object Lock (compliance mode) for legal hold.

Don't: log secrets / PII without redaction. Don't make the audit write block the user request — async via Kafka with an SLA on lag (alert if > 1 min).

Chapter — DDD and Clean Architecture

22. Deep Dive — DDD & Clean Architecture

Q196. Why DDD? Strategic vs tactical

Strategic DDD — splits a complex problem domain into **bounded contexts**, each with its own model and ubiquitous language. The point: prevent one giant model that means different things to different teams (`Customer` in Sales ≠ `Customer` in Billing).

Tactical DDD — within one bounded context, the building blocks: **entities, value objects, aggregates, domain events, repositories, services**. The point: a domain model that's expressive, behavior-rich, and decoupled from infrastructure.

You don't need both. A CRUD admin tool needs neither. A complex business domain (insurance, logistics, fintech) usually needs strategic DDD even if tactical patterns are kept

light.

Q197. Bounded contexts and the context map

A **bounded context** is the explicit boundary inside which a model is consistent and unambiguous. Often (not always) maps to a microservice.

A **context map** documents the relationships between contexts:

- **Partnership** — two teams succeed or fail together; coordinate releases.
- **Customer/Supplier** — downstream depends on upstream; upstream takes downstream's needs into account.
- **Conformist** — downstream uses upstream's model as-is (no leverage).
- **Anti-corruption Layer (ACL)** — downstream wraps upstream behind an adapter so the upstream's model doesn't pollute the local one.
- **Shared Kernel** — two contexts share a small piece of model. Risky — needs strict change discipline.
- **Open Host Service** — upstream exposes a published, versioned API for many downstreams.
- **Published Language** — a shared, formal contract (events, OpenAPI) between contexts.

Context boundaries usually align with **subdomains**: **core** (your competitive advantage — invest), **supporting** (necessary but generic — build but light), **generic** (commodities — buy/use SaaS).

Q198. Entities vs value objects

- **Entity** — has identity that persists across changes. `Order(id=42)` is the same order whether the status is `pending` or `shipped`. Compared by ID.
- **Value object** — defined entirely by its attributes. `Money(100, "USD")` is the same as any other `Money(100, "USD")`. Compared by value, immutable, no identity.

```
class Money {
  constructor(readonly amount: number, readonly currency: string) {
    if (amount < 0) throw new InvalidMoney('negative');
    Object.freeze(this);
  }
  add(other: Money): Money {
    if (other.currency !== this.currency) throw new CurrencyMismatch();
    return new Money(this.amount + other.amount, this.currency);
  }
}
```

Push behavior into value objects (`money.add()`, `email.normalize()`) — they're cheap, safe, testable.

Q199. Aggregates and aggregate roots — the rules

An **aggregate** is a cluster of entities and value objects treated as a single transactional unit. It has one **root entity**; outside callers can only reference the root, never inner objects.

The four invariant rules:

1. **One aggregate per transaction.** If you must change two aggregates, it implies eventual consistency (domain event → handler updates the second).
2. **Reference other aggregates by ID, not by object reference.** `Order` holds `customerId`, not a `Customer` instance.
3. **The root enforces invariants.** All mutations go through root methods; inner state is encapsulated.
4. **Keep aggregates small.** Big aggregates lock-contend and load slowly. If invariants don't span two entities, they're separate aggregates.

```
class Order {  // aggregate root
  private items: OrderItem[] = [];
  private constructor(readonly id: OrderId, readonly customerId: CustomerId) {}

  addItem(productId: ProductId, qty: number, price: Money) {
    if (this.items.length >= 100) throw new TooManyItems();
    this.items.push(new OrderItem(productId, qty, price));
    this.addEvent(new ItemAddedToOrder(this.id, productId, qty));
  }
}
```

Q200. Domain events

Aggregates record events when something domain-relevant happens. Events:

- Are **named in past tense** (`OrderPlaced` , `PaymentCaptured`).
- Carry the data subscribers need (immutable, serializable).
- Are dispatched **after the transaction commits** (otherwise a rollback leaves a phantom event).

Pattern: aggregate.addEvent() → repository collects events on save → after COMMIT, publish to in-process bus + outbox table for cross-service.

This is where "single aggregate per transaction" stops feeling like a constraint — events make multi-aggregate workflows natural.

Q201. Repositories — persistence ignorance

A repository looks like an in-memory collection of aggregates: `orderRepo.findById(id)` , `orderRepo.save(order)` . It hides whether the data lives in Postgres, DynamoDB, or memory.

```
interface OrderRepository {
  findById(id: OrderId): Promise<Order | null>;
  save(order: Order): Promise<void>;
}
```

Implementation lives in the **infrastructure** layer (e.g., `PostgresOrderRepository`). Domain code depends only on the interface. This makes it possible to test domain logic without a database and swap persistence later.

Don't expose query builders or `findBy<RandomCombination>` — those are read-side concerns. Use a separate **query service** / read model for those (CQRS).

Q202. Application services vs domain services

- **Application service** (use case) — orchestrates a single transaction: load aggregates, call domain methods, save, dispatch events. Thin, no business rules.
- **Domain service** — business logic that doesn't naturally belong to an entity (`PriceCalculator`, `TransferFundsService` operating on two `Account` aggregates). Stateless, in the domain layer.

```
class PlaceOrderUseCase {
  constructor(private orders: OrderRepository, private events: EventBus) {}
  async execute(cmd: PlaceOrderCommand) {
    const order = Order.place(cmd.customerId, cmd.items);  // domain logic in e
    await this.orders.save(order);
    await this.events.publishAll(order.pullEvents());
  }
}
```

Q203. Clean Architecture — the dependency rule

Layers, with dependencies pointing **inward only**:

```
┌──────────────────────────────────────────────────────────┐
│  Frameworks & Drivers (web, db, devices, UI)             │
│  ┌──────────────────────────────────────────────────┐    │
│  │  Interface Adapters (controllers, presenters,    │    │
│  │   gateways, ORM mappings)                        │    │
│  │  ┌──────────────────────────────────────┐        │    │
│  │  │  Application (use cases)             │        │    │
│  │  │  ┌──────────────────────────────┐     │        │    │
│  │  │  │  Domain (entities, VOs,      │     │        │    │
│  │  │  │   aggregates, domain events) │     │        │    │
│  │  │  └──────────────────────────────┘     │        │    │
│  │  └──────────────────────────────────────┘        │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

Domain has zero imports from outer layers. Use cases depend on domain. Adapters implement interfaces declared in inner layers (dependency inversion). This is what makes the inside testable in isolation and replaceable from the outside.

Q204. Hexagonal (ports & adapters)

Same idea, different vocabulary. The core is the **application + domain**. Around it:

- **Driving (primary) ports** — interfaces the application exposes (use case interfaces). **Driving adapters**: HTTP controllers, CLI, message consumers — they invoke the port.
- **Driven (secondary) ports** — interfaces the application needs (`OrderRepository`, `PaymentGateway`). **Driven adapters**: Postgres repo, Stripe client, in-memory fake.

The application sits between, agnostic of either side. You can run it against an HTTP adapter in prod and a test runner adapter in CI, against Postgres in prod and an in-memory adapter in tests — same core.

Q205. NestJS folder layout for DDD

```
src/
  modules/
    orders/
      domain/                  # pure TS, no Nest decorators
        order.aggregate.ts
        order-item.vo.ts
        money.vo.ts
        events/
          order-placed.event.ts
        order.repository.ts    # interface
      application/
        commands/
          place-order/
            place-order.command.ts
            place-order.handler.ts
        queries/
          get-order/
            get-order.query.ts
            get-order.handler.ts
        ports/
          payment-gateway.port.ts
      infrastructure/
        persistence/
          order.entity.ts      # ORM/typeorm/prisma model
          order.repository.impl.ts
          order.mapper.ts      # ORM model ↔ aggregate
        gateways/
          stripe-payment.gateway.ts
      presentation/
        http/
          orders.controller.ts
          dto/
        events/
          order-placed.consumer.ts
      orders.module.ts
  shared/
    kernel/                    # cross-cutting domain primitives (Money, Email,
```

`domain/` has zero `@Injectable()` — it's pure. Wiring lives in `orders.module.ts` (`provide: ORDER_REPOSITORY, useClass: PostgresOrderRepository`). Use cases get the interface, not the implementation.

Q206. CQRS — when and how

Command-Query Responsibility Segregation — separate write model (rich domain, transactional) from read model (denormalized, optimized for queries).

```
Write side:  Command → Use case → Aggregate → Repository → DB
                                      │
                                      └→ Domain event → Projector
                                                          ↓
Read side:   Query ──────────────────────────────────→ Read model (cache, ES, de
```

When CQRS pays off:

- Read patterns are very different from the write model (e.g., dashboards joining 8 tables).
- Read load >> write load.
- You're already eventing for other reasons.

When it's overkill: simple CRUD with the same model on both sides.

NestJS has `@nestjs/cqrs` if you want bus + handler scaffolding, but you can roll it yourself in 30 lines.

Q207. Event sourcing — when to reach for it

Instead of storing the current state, store the sequence of events. Current state = fold(events).

```
Event stream for Order(42):
  OrderPlaced { items: [...] }
  ItemAdded { product: X }
  PaymentCaptured { amount: ... }
  OrderShipped { tracking: ... }
```

Pros: perfect audit log, time travel, easy to add new projections retroactively, natural fit with CQRS and DDD events.

Cons: schema evolution is harder (you can never delete old event shapes — only "upcast" them), querying current state requires projections, eventual consistency between write store and read models, snapshots needed for long streams.

Use it for domains where the **history is the truth**: ledgers, audit-heavy systems, regulated domains. Avoid it for "we wanted CRUD but with extra steps."

Q208. Anti-corruption layer

When integrating with a legacy or external system whose model conflicts with yours, build an ACL — adapters + mapping that translate the external model into your bounded context's model on the way in and back on the way out.

```
External "Customer" model                Your "Buyer" aggregate
  { firstName, lastName,                   Buyer {
    addr1, addr2, addr3,         →           name: PersonName,
    flagsBitmask, ... }                      address: Address,
                                             tier: BuyerTier }
```

The ACL lives in the **infrastructure** layer. Without it, the external model leaks into your domain and corrupts it (hence the name).

Q209. Anemic vs rich domain model

Anemic — entities are property bags; logic lives in service classes (`OrderService.addItem(order, item)`). Looks like Java circa 2005. Easy to start with, terrible to maintain — invariants get violated because there's no single guardian.

Rich — behavior lives on the entity (`order.addItem(item)`). The entity enforces its invariants. This is what DDD asks for.

Sign of an anemic model: lots of `getOrder…(): X; setOrder…(x): void` and a sibling `Service` doing all the logic. Sign of a rich model: aggregates expose verbs, not setters.

Q210. Transaction boundaries in DDD

The application service starts and commits the transaction. **One aggregate per transaction**. Multi-aggregate workflows go through domain events + a separate handler that opens its own transaction:

```
PlaceOrderUseCase tx:
  - load Order
  - order.place()
  - save Order
  - publish events  (after commit, via outbox)

InventoryReservationOnOrderPlaced handler tx:
  - load InventoryItem
  - inventory.reserve()
  - save InventoryItem
```

If the second tx fails, you compensate (cancel the order, refund payment) — not roll back the first. This is the saga pattern in DDD clothing.

Part XI — Performance, Scalability, and Observability

Chapter — Performance Fundamentals

13. Performance & Scalability

Q114. How do you find a memory leak in production Node?

1. Reproduce in staging or enable heap snapshots in prod.
2. Take **two heap snapshots** a few minutes apart under load.
3. Compare retained sizes; look at constructors that grew.
4. Inspect retainer paths to find the root cause.

Tools: `node --inspect`, Chrome DevTools, `clinic.js`, `heapdump`, `0x`.

Q115. How do you diagnose high CPU?

- Attach a profiler: `node --prof` then `node --prof-process`; or `clinic flame`; or `0x`.
- Look for hot functions, regex backtracking, sync crypto, JSON.parse on huge payloads, tight loops.
- Offload CPU work to worker threads or an external service.

Q116. Event loop lag — what is it and how do you measure it?

If a callback blocks the event loop, timers drift and I/O stalls. Measure via `perf_hooks.monitorEventLoopDelay()`.

```
const { monitorEventLoopDelay } = require('perf_hooks');
const h = monitorEventLoopDelay({ resolution: 20 });
h.enable();
setInterval(() => console.log('p99 lag:', h.percentile(99) / 1e6, 'ms'), 1000);
```

Q117. Common Node.js perf wins

- `JSON.stringify/parse` is expensive — avoid reparsing.
- Use streaming for large payloads.
- Batch DB queries (DataLoader).
- Cache hot data in Redis or local LRU.
- Compile regexes once, not per call.
- Avoid `try/catch` in hot paths (pre-V8 10 it disabled optimizations; still a slight cost).
- Use `undici` instead of `node-fetch` for high-throughput HTTP clients.
- Keep objects at fixed shapes — V8 can inline-cache.
- Use `Pino` over `Winston` for logging (much faster).

Q118. Scaling a Node.js service

1. **Cluster** across CPU cores.
2. **Horizontal** scale behind a load balancer.
3. **Cache** at every layer (CDN, Redis, in-memory).
4. **Queue** slow work.
5. **Read replicas** for the DB.
6. **Drop sync I/O** (fs.readFileSync at boot only).

Chapter — Performance & Observability Deep Dive

23. Deep Dive — Performance & Observability

Q211. The pillars of observability — and the fourth one

The classic three: **logs, metrics, traces**. The fourth often missed: **profiles** (continuous CPU/heap/lock profiling).

Pillar	Cardinality	Cost	Best for
Metrics	Low (label combos)	Cheapest	Aggregates: rate, error rate, latency percentiles
Logs	Unbounded	Expensive at volume	Forensics on a single request
Traces	Per-request, sampled	Medium	"Where in the call chain did time go?"
Profiles	Continuous, low overhead	Cheap	"Which functions burn CPU/allocate?"

You need all four to debug production issues confidently. Logs tell you *what happened*, metrics *how often*, traces *where*, profiles *why* (in CPU terms).

Q212. RED vs USE — pick the right framework

- **RED** (services / requests) — **R**ate, **E**rrors, **D**uration. For every endpoint, dashboard those three. Originated by Tom Wilkie at Weaveworks.
- **USE** (resources) — **U**tilization, **S**aturation, **E**rrors. For every resource (CPU, memory, disk, network), dashboard those three. Brendan Gregg's framework.

Use both. RED tells you the user-visible symptom. USE tells you which resource is the cause. A service with high duration (RED) often correlates with high saturation (USE) on a downstream DB.

Q213. SLI, SLO, error budgets

- **SLI** — what you measure (e.g., "% of requests served < 200ms").
- **SLO** — your target (e.g., "99.9% of requests < 200ms over 30 days").
- **SLA** — the contractual version (with consequences). Often looser than the SLO.
- **Error budget** — `1 - SLO`. With a 99.9% SLO, you have 43 minutes of "burn" per month.

Operational policy: when you've burned > 50% of the budget early in the window, freeze risky deploys; when you've stayed under, you're allowed to ship faster. Makes "stability vs velocity" a measurable trade-off, not an argument.

Multi-window burn-rate alerts beat threshold alerts: alert if you'd burn the monthly budget in a few hours at the current rate.

Q214. Instrumenting Node with OpenTelemetry

OTel is the vendor-neutral standard. Auto-instrumentation hooks common libs (HTTP, Express, Nest, Postgres, Redis, Kafka, Mongo).

```
// tracing.js — required BEFORE any other import
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentatio
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({ url: process.env.OTEL_ENDPOINT }),
  instrumentations: [getNodeAutoInstrumentations()],
});
sdk.start();
```

Run with `node --import ./tracing.js app.js`. Add custom spans for business logic:

```
import { trace } from '@opentelemetry/api';
const tracer = trace.getTracer('orders');
await tracer.startActiveSpan('orders.place', async (span) => {
  span.setAttribute('orders.itemCount', items.length);
  try { /* work */ } finally { span.end(); }
});
```

Send to Jaeger / Tempo / Honeycomb / Datadog via OTLP — the exporter is the only thing that changes per backend.

Q215. Structured logging done right

Rules:

- **JSON, one line per event.**
- Always include `timestamp`, `level`, `service`, `traceId`, `spanId`, `requestId`, `userId` (when present).
- Don't log secrets, tokens, full request bodies. Have a denylist.
- Levels: `error` → page, `warn` → review, `info` → audit trail, `debug` → off in prod.
- Use a fast logger: **pino** (~5× faster than Winston, ~50μs per log).

```
import pino from 'pino';
const log = pino({ redact: ['req.headers.authorization', 'password'] });
log.info({ userId, orderId }, 'order placed');
```

Get `traceId` / `spanId` from OTel context via AsyncLocalStorage so every log line is correlatable to a trace.

Q216. Sampling traces

100% sampling is expensive at scale. Strategies:

- **Head-based** — decide at the root span. Cheap, fast, but you lose interesting traces (errors) at random.
- **Tail-based** — collect everything, decide at the collector after the trace completes. Lets you keep all errors and slow traces. Needs a buffer (Tempo, OTel collector with `tail_sampling` processor).

A sensible default: 100% of errors, 100% of slow (`> p95`), 1–5% of everything else.

Q217. Continuous profiling

Profilers like **Pyroscope** / **Parca** / **Datadog Profiler** sample CPU, heap, and goroutines/asyncs continuously in production at < 1% overhead. You get "what was the system doing at this moment?" for any historical timestamp.

Killer feature: **diff** the profile from before vs after a deploy to see exactly which functions started taking more CPU. Saves hours of guessing.

In Node: `--cpu-prof` and `--heap-prof` flags emit `.cpuprofile` / `.heapprofile` files. Open in Chrome DevTools or feed to a continuous profiler.

Q218. Benchmarking Node APIs

- **autocannon** — Node-native, low overhead. Good for HTTP.

  ```
  autocannon -c 100 -d 30 http://localhost:3000/api
  ```

- **k6** — JS-scripted, scenario-based load tests. Good for sustained tests with ramps.
- **wrk** — C, lowest overhead. Use when the load generator itself is the bottleneck.

Best practices:

- Run on a separate box from the SUT. Localhost benchmarks lie.
- **Warm up** for 30–60s before measuring.
- Report **p50/p95/p99 + max**, not just averages.
- Hold one variable; change one. Test against a stable baseline.
- Use the **same payload distribution** as production.

Q219. Reading flame graphs

X-axis = time-on-CPU (sample count), **not** wall-clock time. Y-axis = stack depth (caller below callee). Width = how much CPU. Look for **plateaus** (wide flat areas) — those are your hot functions.

```
+---------------------+
|       handler       |   <- if this is wide, it's the bottleneck
+---+-----+-----+-----+
|JSON|qury| ... |     |
+---+-----+-----+-----+
```

In Node, `0x` and clinic `flame` produce these. Look at **self time** (top of stack) vs **total time** (everything called below). Differential flamegraphs (red = added CPU, blue = removed) are gold for regression hunting.

Q220. GC tuning in Node

Default Node ships with reasonable V8 GC defaults. Tune when you have evidence (`--trace-gc` , `perf_hooks` GC events).

Common knobs:

- `--max-old-space-size=4096` — raise old-space cap when you OOM with `JavaScript heap out of memory`.
- `--max-semi-space-size=64` — bigger young space = fewer scavenges, but each is bigger.
- `--gc-interval=N` (rarely useful) — force GC every N allocations.

What usually fixes GC pressure isn't a flag, it's allocating less:

- Reuse buffers (`Buffer.allocUnsafe` from a pool) instead of `Buffer.alloc` per request.

- Stream large responses instead of buffering.
- Avoid huge object literals in hot paths.
- Watch for accidental closures keeping big graphs alive.

Q221. HTTP/2, HTTP/3, keep-alive

- **HTTP/1.1** — one request at a time per connection (head-of-line blocking). Use `keep-alive` and per-host connection pools.
- **HTTP/2** — multiplexed streams over one TCP connection. Eliminates HoL at the HTTP layer; at the TCP layer one lost packet still stalls all streams.
- **HTTP/3** — over QUIC (UDP). No transport-level HoL; faster handshakes (0-RTT). Best for high-latency / lossy networks.

In Node, `http2` is built-in for servers. For clients, `undici` is much faster than `http` and supports HTTP/2 and connection pooling out of the box. **Always reuse an HTTP agent** for outbound calls — creating a new connection per request is the #1 hidden latency cost.

Q222. Diagnosing event loop lag end-to-end

```
import { monitorEventLoopDelay } from 'node:perf_hooks';
const h = monitorEventLoopDelay({ resolution: 10 });
h.enable();
setInterval(() => {
  console.log({ p50: h.percentile(50), p99: h.percentile(99), max: h.max });
  h.reset();
}, 5000);
```

If p99 > 50ms regularly, something is blocking the loop. Investigate in this order:

1. **Sync CPU work** — JSON parsing of a 5MB body, synchronous crypto, regex catastrophic backtracking. Move to `worker_threads` or stream.
2. **Massive promise micro-task floods** — `Promise.all([...10k items])` resolving at once. Process in chunks.
3. **Native addon blocking** — some addons run on the JS thread. `node-report` shows native frames.
4. **GC pauses** — `--trace-gc` will show them. Often a symptom of allocation pressure (above).
5. **libuv pool saturation** — fs/crypto queueing. `UV_THREADPOOL_SIZE`.

Export the lag as a metric (`event_loop_lag_p99_ms`) and alert on it — it's the single best leading indicator of a Node service in trouble.

Chapter — Deep Node.js & V8 Internals

18. Deep Dive — Node.js & V8 Internals

Q149. How does V8 compile and optimize JavaScript?

V8 is a **multi-tier JIT**:

1. **Parser** → AST.
2. **Ignition** — bytecode interpreter. Runs the code immediately and gathers type feedback (which types each operation actually sees).
3. **Sparkplug** — non-optimizing baseline compiler that turns bytecode into machine code very quickly. No optimizations, just a speed boost over the interpreter.
4. **Maglev** (newer mid-tier) — fast optimizing compiler for "hot enough" code.
5. **TurboFan** — top-tier optimizer. Speculatively inlines, eliminates dead code, unboxes numbers, uses inline caches. Produces highly optimized machine code based on the type feedback.

If a speculative assumption is violated (e.g., a function that always saw integers suddenly sees a string), V8 **deoptimizes** back to bytecode and may re-optimize later.

Why it matters: keep functions monomorphic (one shape of input) for as long as possible — that's what TurboFan rewards.

Q150. Hidden classes and inline caches

V8 doesn't store JS objects as hashmaps. It assigns each object a **hidden class** (a.k.a. shape/map) describing its property layout. Two objects with the same properties added in the same order share a hidden class.

```
function Point(x, y) { this.x = x; this.y = y; } // shape A → B → C

const a = new Point(1, 2);
const b = new Point(3, 4);  // same hidden class as a
b.z = 5;                     // b transitions to a NEW hidden class
```

Inline caches (ICs) at each property access remember the hidden class they last saw, so future accesses become a pointer lookup at a fixed offset.

- **Monomorphic IC** (1 shape) — fastest.
- **Polymorphic** (2–4 shapes) — fast-ish.
- **Megamorphic** (>4) — falls back to a generic dictionary lookup. Avoid.

Practical rules:

- Always initialize all properties in the constructor, in the same order.
- Avoid `delete obj.prop` — converts the object to dictionary mode.
- Don't add properties later in the lifetime.

Q151. V8's heap layout and garbage collection

The V8 heap is split into **generations**:

Space	What	GC algorithm
New space (young)	Newly allocated objects, ~1–8 MB	**Scavenge** (Cheney's copying GC) — very fast, runs often
Old space	Objects that survived 2 scavenges	**Mark-sweep-compact** with **incremental + concurrent** marking
Large object space	Objects ≥ ~512 KB	Allocated directly, swept in place
Code space	Compiled JIT code	
Map space	Hidden classes	

Most objects die young (the **generational hypothesis**). Scavenge cleans them in microseconds. Survivors are promoted to old space, where collection is more expensive but rarer.

V8 uses **incremental marking** (small slices interleaved with JS execution) and **concurrent marking** (on a helper thread) to keep stop-the-world pauses to a few ms even on multi-GB heaps.

Tune with:

- `--max-old-space-size=4096` (MB) — raise the old-space cap.
- `--max-semi-space-size` — young-space size; bigger means fewer scavenges but longer ones.
- `--expose-gc` + `global.gc()` for tests.

Q152. Common V8 optimization killers

- **`try/catch` around hot loops** — historically prevented optimization (much better since V8 ~6.0, but still costs).
- **`with` statement and `eval` with non-strict scope** — deoptimize aggressively.
- **`arguments` object leaks** — passing `arguments` to another function. Use rest parameters: `(...args) => ...`.
- **Mixing types in one variable** — `let x = 1; x = 'hi';`.
- **Polymorphic functions** — same function called with too many object shapes.
- **`delete obj.prop`** — pushes objects into dictionary mode.
- **Sparse arrays / holes** — `arr[1000] = 1` on an empty array creates a holey array, slower than packed.
- **Constructor-less property addition** — adding properties after construction creates new hidden class transitions for every instance.

Q153. libuv thread pool — when does it matter?

libuv has a default pool of **4 threads** (cap 1024). These handle:

- File I/O (`fs.*`)
- DNS (`dns.lookup` — the synchronous resolver. `dns.resolve` uses async network.)
- Some `crypto` ops (`pbkdf2` , `randomBytes` , `scrypt`)
- `zlib` (gzip/deflate)
- Some `dgram` ops

Symptom of saturation: under load, file/crypto requests queue, latency rises but CPU is low. Tune with:

```
UV_THREADPOOL_SIZE=16 node server.js
```

Network I/O (TCP, UDP) does **not** use the pool — it's fully async via epoll/kqueue/IOCP, polled in the event loop's poll phase.

Rule: bump the pool only if you have measured pool starvation (`async_hooks` or `uv_metrics_info` in newer Node).

Q154. The event loop's "ref count" — when does Node exit?

The event loop keeps running while there's **at least one referenced handle** (timer, socket, server, etc.) or pending request. Node exits when the count hits zero.

```
const t = setTimeout(() => {}, 60_000);
t.unref();   // doesn't keep the loop alive
t.ref();     // does
```

`server.unref()` lets you keep a debug HTTP server alive only while there's other work. Same on streams, sockets, child processes. This is what lets a CLI script exit cleanly without manually tearing things down.

Q155. AsyncLocalStorage — what is it and what does it cost?

`AsyncLocalStorage` (ALS) is Node's built-in mechanism for **per-request context propagation** across async boundaries. It uses `async_hooks` under the hood to carry a context object through promises and callbacks without passing it explicitly.

```
import { AsyncLocalStorage } from 'node:async_hooks';
const als = new AsyncLocalStorage();

app.use((req, res, next) => {
  als.run({ requestId: req.headers['x-request-id'] }, next);
});

function log(msg) {
  const ctx = als.getStore();
  console.log(`[${ctx?.requestId}] ${msg}`);
}
```

Use cases: request IDs, tenant context, transaction objects, OpenTelemetry spans.

Cost: modest in modern Node (post-19, native AsyncContextFrame implementation). Each promise chain creation has a small overhead. Don't enable global `async_hooks` (different API) — that's the slow one.

Q156. Production diagnostics toolkit

Symptom	Tool
High CPU	`node --prof` then `--prof-process`; `0x` for flamegraphs; clinic.js `flame`
Slow event loop	clinic.js `doctor` / `bubbleprof`; `perf_hooks.monitorEventLoopDelay()`
Memory leak	`node --inspect` + Chrome DevTools heap snapshots; diff three snapshots; `heapdump` module
Native memory leak	`mimalloc` / `jemalloc` allocator + `MALLOC_CONF`; `valgrind` (Linux); `--heap-prof`
GC pauses	`--trace-gc`, `--trace-gc-verbose`; `perf_hooks.PerformanceObserver` on `'gc'`
Async stalls	`async_hooks` + clinic `bubbleprof`; OpenTelemetry traces
Crash on prod	`--report-on-fatalerror` writes `report-*.json` with stack/heap stats
Live debugging	`node --inspect=0.0.0.0:9229` + Chrome DevTools (gate by SSH tunnel)

Always take **3 heap snapshots** during a leak investigation — between snapshot 1 and 2 you exercise the suspect path, between 2 and 3 you let it settle. Objects that grow across both intervals are leaking.

Q157. Native addons — N-API in 60 seconds

Three options, in order of preference:

1. **Node-API (N-API / `node-addon-api`)** — ABI-stable C/C++ API. The addon you compile against Node 20 keeps working on Node 22 without recompiling. **Default choice.**
2. **NAN** — older, requires recompile per Node major version. Legacy only.
3. **Internal V8 API** — fastest but breaks every release. Don't.

Build with `node-gyp` or `cmake-js`. For Rust, **napi-rs** generates N-API bindings and is what most modern native addons use (e.g., SWC, Parcel).

When to reach for native: CPU-heavy hot paths (image processing, hashing, parsing) where worker_threads + JS isn't fast enough, or wrapping an existing C/C++ library.

Q158. WebAssembly in Node

Node ships with V8's WebAssembly engine. You can:

```
const wasm = await WebAssembly.instantiate(bytes, imports);
wasm.instance.exports.myFn(42);
```

When wasm beats JS: numerical loops, parsers, codecs, anything compiled from C/Rust/Zig. **WASI** (`node --experimental-wasi-unstable-preview1`) lets wasm modules touch the filesystem.

Trade-offs: copying data across the JS/wasm boundary is the bottleneck. Use shared `ArrayBuffer` (`memory.buffer`) views, not per-call copies.

For most Node web servers, wasm is overkill — but for CPU-bound services (image transforms, regex-heavy parsers, ML inference) it's a serious lever.

Part XII — Distributed Systems and Kafka

20. Deep Dive — Distributed Systems & Kafka

Q171. Kafka architecture in one picture

```
Producers ──► [ Topic A ]                          ┌─► Consumer Group X (lag trac
              ├ Partition 0 ──► Broker 1 (leader), Broker 2 (follower) ──┤
              ├ Partition 1 ──► Broker 2 (leader), Broker 3 (follower)   ├─► Co
              └ Partition 2 ──► Broker 3 (leader), Broker 1 (follower) ──┘

KRaft / ZooKeeper ── stores metadata (which broker leads which partition, ACLs,
```

Key facts:

- A **topic** is a log split into **partitions**. Order is guaranteed **per partition only**.
- Each partition has one **leader** and N-1 **followers**. Producers and consumers always talk to the leader.
- The **In-Sync Replica (ISR)** set = followers that are caught up. Leader election picks from ISR.
- The **high watermark** = the latest offset replicated to all ISRs. Consumers can only read up to the HWM.
- A partition is one **log directory** of segment files (`.log`, `.index`, `.timeindex`). Old segments are deleted by retention or compaction.
- **KRaft** (Kafka Raft) replaces ZooKeeper from 3.3+. ZK is removed entirely in 4.0.

Q172. Producer guarantees — acks, idempotence, transactions

```
const producer = kafka.producer({
  idempotent: true,        // dedup by (producer-id, sequence) per partition
  maxInFlightRequests: 5,  // safe with idempotence
  acks: -1,                // 'all' — wait for ISR to ack
});
```

- `acks=0` — fire and forget. Loss likely.
- `acks=1` — leader ack only. Loss if leader dies before followers replicate.
- `acks=all` (-1) — leader waits for ISR. Combined with `min.insync.replicas=2` (broker-side), this is the durable setting.
- `enable.idempotence=true` — producer assigns sequence numbers; broker drops duplicates from the same producer ID for that partition. Solves the "retry causes duplicate" problem.
- **Transactions** — write to multiple topics/partitions atomically. Combined with `read_committed` consumers, gives exactly-once across topics. Used by Kafka Streams.

Q173. Consumer groups and rebalancing

A **consumer group** is a set of consumers that split partitions among themselves. Each partition is consumed by exactly one consumer in the group. If you have 6 partitions and 3 consumers, each gets 2; add a 4th and rebalance redistributes.

Rebalance protocols:

- **Eager (range / round-robin)** — everyone stops, partitions reassigned, everyone resumes. Stop-the-world for the whole group.
- **Cooperative sticky** (default in modern clients) — only revokes the partitions that need to move. Much smoother during rolling deploys.
- **Static membership** (`group.instance.id`) — consumer keeps its assignment across restarts within `session.timeout.ms` . Avoids rebalance on every deploy.

Offsets are stored in the internal `__consumer_offsets` topic. Commit them only after processing succeeds (or use exactly-once with the producer below).

Q174. Exactly-once semantics (EOS)

Three pieces:

1. **Idempotent producer** — no duplicates per partition due to retry.
2. **Transactions** — atomic write across multiple partitions/topics.
3. `isolation.level=read_committed` consumer — skips messages from aborted transactions.

The classic EOS pattern (consume → process → produce) is implemented via:

```
await producer.transaction(async (txn) => {
  await txn.send({ topic: 'out', messages: [...] });
  await txn.sendOffsets({                  // commits input offset INSIDE the t
    consumerGroupId: 'my-group',
    topics: [{ topic: 'in', partitions: [{ partition: 0, offset: '12345' }] }],
  });
});
```

This makes "consume offset N from `in` AND publish to `out`" atomic. Outside Kafka (e.g. writing to Postgres), you need the **outbox pattern** instead — Kafka EOS doesn't extend to external systems.

Q175. Partition key strategy & hot partitions

The producer picks a partition by `hash(key) % numPartitions` (default partitioner). Implications:

- **Same key → same partition → ordered** (good for "all events for user X are processed in order").
- **Skewed keys → hot partition** — one partition does 80% of the work, one consumer is overloaded while others idle.
- **Repartitioning is painful** — adding partitions changes the hash mapping, so old keys land in different partitions. Plan headroom upfront.

Mitigations: composite keys (`userId:bucket(0..9)`), sticky partitioner for keyless messages, monitor `BytesInPerSec` per partition.

Number of partitions = max parallelism per consumer group. Rule of thumb: 2–4× peak consumer count, capped by per-broker partition limits (~4000 per broker).

Q176. Schema Registry & schema evolution

Confluent Schema Registry stores Avro/Protobuf/JSON schemas keyed by subject (usually `<topic>-value`). Producers serialize with the schema ID prefixed; consumers fetch the schema by ID and deserialize.

Compatibility modes (per subject):

- `BACKWARD` (default) — new schema can read data written by old. Safe to deploy new consumers first.

- `FORWARD` — old schema can read new data. Deploy new producers first.
- `FULL` — both.
- `NONE` — wild west.

Rules of thumb: **add fields with defaults**, never remove or rename, never change types. Use Avro unions (`["null","string"]`) for nullables.

Q177. Dead letter topics & retry strategy

Don't loop on a poison pill — you'll block the partition forever. Pattern (popularized by Uber):

```
main_topic ──fail──► retry_5s ──fail──► retry_30s ──fail──► retry_5m ──fail──►
```

Each retry topic has its own consumer with a delay (sleep, or a dedicated scheduler). The DLQ is alarmed on, hand-inspected, and replayed manually. For lower-volume use cases, a single `dlq` + a "redrive" job is enough.

Q178. Kafka vs RabbitMQ vs SQS vs Kinesis

	Kafka	RabbitMQ	SQS	Kinesis
Model	Distributed log (consumers control offset)	Queue + exchange routing	Queue (msg deleted on ack)	Distributed log
Ordering	Per partition	Per queue (single consumer)	FIFO queues only	Per shard
Throughput	Very high (MB/s per partition)	High	High (auto-scales)	High
Replay	Yes (retention)	No	No	Yes (24h–365d)
Consumers	Pull	Push	Pull (long poll)	Pull (KCL)
Routing	Topics + keys	Exchanges, headers, RPC	Topics via SNS fanout	None
Ops	Heavy (or use MSK / Confluent Cloud)	Medium	Zero	Medium
Sweet spot	Event streaming, CDC, audit	Task queues, RPC, complex routing	Decoupled microservices on AWS	AWS-native streaming

Picking: need to replay history or fan out to many independent consumers → Kafka/Kinesis. Need flexible routing or RPC → Rabbit. Don't want to run anything → SQS/SNS.

Q179. Outbox pattern with Kafka

Problem: writing to your DB **and** publishing to Kafka can't be atomic across systems.

Solution: write the event to an `outbox` table **in the same transaction** as the business change. A relay reads the outbox and publishes to Kafka. With **Debezium** or a custom poller doing CDC on the outbox table, the publish is decoupled from the request path.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
INSERT INTO outbox(aggregate_id, type, payload) VALUES (1, 'MoneyDebited', '{..
COMMIT;
-- Debezium streams the outbox INSERT to Kafka, dedup'd by event id
```

Consumers must be idempotent (the relay can retry, so duplicates happen).

Q180. Saga pattern — orchestration vs choreography

A **saga** is a sequence of local transactions across services. If step N fails, prior steps are undone via **compensating transactions** (no global rollback exists in distributed systems).

- **Choreography** — each service emits events; others react. No central coordinator. Loose coupling but the business flow is hard to see.
- **Orchestration** — a coordinator (e.g., Temporal, AWS Step Functions, a Saga service) explicitly drives the steps. Easier to reason about and observe; the coordinator is a new component to operate.

Choose orchestration when the saga has > 3 steps or branching logic. Choose choreography for simple two-service flows.

Q181. Two-phase commit — why it's avoided

2PC has a **prepare** then **commit** phase coordinated by a transaction manager. It works on paper but in distributed systems:

- **Blocking on coordinator failure** — if the coordinator dies after prepare, participants are stuck holding locks until it returns.
- **Latency** — every participant pays multiple network round-trips.
- **Heterogeneous systems** — most modern stores (Kafka, DynamoDB, Mongo) don't speak XA.
- **Reduces availability** — one slow participant blocks everyone.

Use sagas + idempotency + outbox instead.

Q182. Consensus — Raft in one picture

Raft elects a leader; the leader appends entries to its log and replicates to followers. An entry is **committed** once a majority has acknowledged. Followers apply committed entries to their

state machine.

```
[Client] ──► Leader ──append──► Followers ──ack──► Leader ──commit──► state mac
```

Leadership requires a quorum. With 5 nodes, you can lose 2 and still make progress. Used by etcd, CockroachDB, Consul, KRaft. **Paxos** is the older cousin — Raft is just easier to implement and explain. **ZAB** (ZooKeeper) is similar in spirit.

Q183. Idempotency keys at the API boundary

Client sends `Idempotency-Key: <uuid>`. Server stores `(key, response)` for a TTL (24h is common). Replays return the cached response.

```
First call:  POST /payments + key=abc-123  → 201, store (abc-123, response)
Retry:       POST /payments + key=abc-123  → 201, return stored response
```

Implementation traps:

- Hash the request body and reject if the same key arrives with a different body (clients re-using keys).
- Lock per key during the first call to prevent the "two concurrent firsts" race (e.g., `INSERT ... ON CONFLICT DO NOTHING`, or Redis `SETNX`).
- Cache only **terminal** responses (2xx/4xx). Don't cache 5xx — let the client retry.

Stripe's API is the canonical example.

Q184. CAP and PACELC

CAP — under a network **P**artition, choose **C**onsistency or **A**vailability. (You can't ditch P; partitions happen.)

PACELC extends it: **i**f **P**artitioned, choose A or C; **e**lse (normal operation), choose **L**atency or **C**onsistency.

Examples:

- DynamoDB, Cassandra: **PA / EL** — prioritize availability and low latency, eventual consistency.
- Spanner, CockroachDB: **PC / EC** — strong consistency, accept higher latency.
- MongoDB (default): **PA / EC**.

This framework matters because "we want strong consistency" usually also means "we accept higher p99 latency" — make that explicit.

Q185. Leader election in distributed systems

Options:

- **Single-leader systems** (Kafka, Postgres): the cluster's metadata service (KRaft, Patroni) elects.
- **etcd / Consul / ZooKeeper** lease — `PUT /lock` with TTL; whoever holds it leads. Renew before expiry.
- **Database advisory lock** — `SELECT pg_try_advisory_lock(...)` in a singleton goroutine. Simple, works for small clusters.
- **Bully algorithm / Raft** — for self-contained systems.

Always include a **fencing token** (monotonic ID from the lock service) and have downstream services reject requests with stale tokens — prevents "zombie leader" double-writes.

Part XIII — Cloud and AWS

Chapter — AWS Fundamentals

11. AWS & Cloud

Q93. Services you must know for a Node backend role

- **EC2** — VMs.
- **Lambda** — serverless functions.
- **ECS / Fargate / EKS** — container orchestration.
- **API Gateway** — HTTP entry, rate limiting, auth.
- **S3** — blob storage.
- **CloudFront** — CDN.
- **RDS / Aurora** — managed SQL.
- **DynamoDB** — managed key-value/document.
- **ElastiCache** — Redis/Memcached.
- **SQS** — queue.
- **SNS** — pub/sub.
- **EventBridge** — event bus with routing.
- **Step Functions** — workflow orchestration.
- **Secrets Manager / SSM Parameter Store** — secrets.
- **IAM** — permissions.
- **CloudWatch** — logs/metrics/alarms.
- **VPC** — networking.

Q94. SQS vs SNS vs EventBridge

- **SQS** — a queue. One consumer per message. Use for work distribution with back-pressure.
- **SNS** — pub/sub. Fan-out to many subscribers.

- **EventBridge** — smart event bus with content-based routing and schema registry; best for decoupled microservices.

Common pattern: **SNS → SQS fan-out** (each subscriber gets a durable queue).

Q95. Lambda — key concepts

- **Cold start** — first invocation after deploy/idle has extra latency while the runtime boots.
- **Provisioned concurrency** — keep N sandboxes warm to avoid cold starts.
- **Execution context** is reused across invocations — put DB connections/SDK clients **outside** the handler.
- Lambda has execution time limits (15 min) and payload size limits (6 MB sync, 256 KB async).
- Layers let you share code between functions.

```
// ✅ reused across warm invocations
const ddb = new DynamoDBClient({});

export const handler = async (event) => {
  await ddb.send(new PutItemCommand({ ... }));
};
```

Q96. S3 — how would you upload large files?

Use **multipart upload** for >100 MB:

1. `CreateMultipartUpload` → get `UploadId`.
2. Upload parts in parallel with `UploadPart` (5 MB–5 GB each, max 10 000 parts).
3. `CompleteMultipartUpload`.

For uploads from a browser without exposing credentials: server issues a **presigned URL**, client uploads directly to S3.

Q97. DynamoDB design — why is it different?

DynamoDB is schemaless key-value. You design **around access patterns** up front, not ER diagrams.

Key concepts:

- **Partition key** — hashed to distribute data; must be high-cardinality.
- **Sort key** — ranges within a partition.
- **Single-table design** — store multiple entity types in one table, distinguished by composite keys (`USER#123`, `ORDER#...`).
- **GSI** — alternative access pattern.

Hot partitions are the main pitfall — avoid low-cardinality partition keys.

Q98. IAM best practices

- **Least privilege** — grant the minimum permissions needed.
- Use **roles**, not long-lived access keys, wherever possible (EC2 instance profile, Lambda role, IRSA for EKS).
- **Separate accounts** per environment (Organizations).
- Avoid `*` in actions. Use condition keys (`aws:SourceIp`, `aws:ResourceTag`).
- Rotate credentials; audit with CloudTrail.

Q99. How would you store secrets?

- **Secrets Manager** (rotation, cross-region replication) for DB credentials, API keys.
- **SSM Parameter Store** — cheaper, no rotation.
- In the app, fetch at boot or via SDK; cache in memory.
- Never in env vars baked into images, never in git, never in logs.

Chapter — AWS Deep Dive

24. Deep Dive — AWS

Q223. VPC networking essentials

A **VPC** is a logically isolated network in a region. Inside it:

- **Subnets** are AZ-scoped slices of the VPC's CIDR. **Public** = route to IGW. **Private** = no IGW.
- **Internet Gateway (IGW)** — egress + ingress to the internet for public subnets.

- **NAT Gateway** — egress-only internet for private subnets (so they can reach S3 / external APIs without inbound exposure). Cost trap: ~$33/month + data processing per AZ.
- **Route table** per subnet — defines where traffic goes.
- **Security Group** — stateful firewall on ENIs (instances/RDS/ALB targets). Allow rules only.
- **NACL** — stateless firewall on subnets, allow + deny. Rarely customized.
- **VPC Endpoints** — private connectivity to AWS services without traversing the internet/NAT. **Gateway endpoints** (free) for S3 and DynamoDB; **Interface endpoints** (cost per hour + data) for everything else.
- **Transit Gateway** — hub for connecting many VPCs / on-prem.
- **VPC Peering** — point-to-point, doesn't transit.

Standard layout: 3 AZs × (public + private + private-data) = 9 subnets. ALB in public, app in private, RDS in private-data.

Q224. ECS vs EKS vs Lambda vs Fargate

	When to use
Lambda	Event-driven, sporadic load, < 15-min runs, ops you don't want. Cold starts (~100ms Node, ~1s Java) are the catch.
Fargate (with ECS or EKS)	Long-running containers, you don't want to manage EC2. Pay per task-second.
ECS on EC2	Steady high load where EC2 (esp. spot) is cheaper than Fargate. AWS-only.
EKS	Kubernetes ecosystem (Helm, operators, multi-cloud portability). Heaviest ops.

Lambda also runs containers (up to 10 GB image). Sweet spot for "Lambda for API + Fargate for stateful workers" hybrid.

Q225. ALB vs NLB vs CloudFront

- **ALB** (L7) — HTTP/HTTPS routing, host/path rules, OIDC integration, gRPC, WebSocket, sticky sessions. Targets: instances, IPs, Lambda, ECS tasks. Don't put a static IP on it; use DNS.
- **NLB** (L4) — TCP/UDP, ultra-low latency, **static IP per AZ**, source IP preservation. Use for non-HTTP, gaming, or when you need a fixed IP for allowlisting.

- **CloudFront** — CDN. Edge cache, WAF, custom origins (ALB/S3/anything). Use for static assets, signed URLs, geographic distribution, DDoS protection (with Shield).

Common pattern: `Route53 → CloudFront → ALB → ECS service`.

Q226. RDS vs Aurora vs DynamoDB — picking

	RDS Postgres/MySQL	Aurora Postgres/MySQL	DynamoDB
Engine	Standard OSS	AWS-rewritten storage	NoSQL KV
Scaling	Vertical, read replicas	Up to 15 aurora replicas, faster failover	Effectively unlimited (partitioned)
Storage	64 TB max	128 TB, auto-grows	Unlimited
Failover	60–120s (Multi-AZ)	< 30s	N/A (multi-AZ by default)
Cost shape	Hourly + storage	~20% more than RDS but more I/O	Pay per RCU/WCU or per request (on-demand)
When	Standard relational, modest scale	Higher throughput, faster failover, serverless v2	Massive scale, single-digit ms, well-known access patterns

Aurora **Serverless v2** scales compute in seconds (ACUs), good for spiky workloads. **DynamoDB on-demand** pricing is great for unpredictable load, expensive at sustained high QPS — switch to provisioned + autoscaling when you know the floor.

Q227. DynamoDB single-table design

DynamoDB rewards precomputing your access patterns. Single-table = put all entity types into one table, distinguished by composite keys.

```
PK              SK                  type      attrs
USER#42         PROFILE             User      name, email
USER#42         ORDER#101           Order     total, status
USER#42         ORDER#102           Order     total, status
ORDER#101       ITEM#A              OrderItem qty, price
ORDER#101       ITEM#B              OrderItem qty, price
```

Queries:

- "Get user + their orders" — `Query PK = USER#42` returns the profile and all orders in one call.
- "Get an order's items" — `Query PK = ORDER#101 AND begins_with(SK, "ITEM#")`.
- "All orders by status" — needs a **GSI** with `GSI1PK = STATUS#PENDING, GSI1SK = ORDER#101`.

Rules:

- **Know access patterns first**, design schema second.
- Hot partition = throttle. Use composite or randomized PKs for skew.
- Use **transactions** (TransactWriteItems) for multi-item atomicity, but it costs 2× WCU and is limited to 100 items.
- **Streams + Lambda** = built-in CDC for projections, audit, search sync.
- **GSI sparseness** — GSIs only index items that have the GSI key, so a GSI keyed on `status` only when set is automatically a "pending orders only" index.

Read Alex DeBrie's *DynamoDB Book* — this is genuinely a different mental model from SQL.

Q228. SQS — visibility timeout, DLQ, long polling

- **Visibility timeout** — when a consumer receives a message, it's invisible to others for this duration. If the consumer doesn't `DeleteMessage` in time, the message reappears. Set it to slightly longer than your worst-case processing time.
- **Long polling** — `WaitTimeSeconds=20` reduces empty receives and cost.
- **DLQ** — after N receives, message moves to a DLQ for manual inspection. Always configure one.
- **FIFO queues** — ordered by **MessageGroupId**, exactly-once via deduplication ID (5-min window). Lower throughput than standard (300 TPS without batching).
- **Standard queues** — at-least-once, best-effort ordering, virtually unlimited throughput.

Idempotency at the consumer is mandatory — duplicates are a feature, not a bug.

Q229. SNS fanout + filter policies

SNS is pub/sub. One topic, many subscribers (SQS, Lambda, HTTPS, email, mobile push, Kinesis).

```
Producer → SNS topic → SQS queue (orders-service)
                     → SQS queue (analytics)
                     → Lambda    (search-indexer)
```

Filter policies let each subscriber receive only matching messages without app-level filtering:

```
{ "eventType": ["OrderPlaced"], "totalUsd": [{ "numeric": [">", 1000] }] }
```

This trims the fan-out at the source and reduces SQS/Lambda invocations.

Q230. EventBridge — buses, rules, Pipes

EventBridge = SNS+ for AWS-native events.

- **Default bus** receives events from AWS services (EC2 state changes, S3 PutObject, etc.).
- **Custom buses** for your app events.
- **Rules** match events by pattern → route to targets (Lambda, SQS, Step Functions, API Destinations, etc.).
- **Schemas** (with discovery) auto-generate types for emitted events.
- **Pipes** — point-to-point, optionally filter and enrich, between sources (SQS, Kinesis, DynamoDB streams, MQ) and targets — replaces the boilerplate "Lambda that reads from SQS and forwards to X".

When to choose EventBridge over SNS: cross-account routing, schema discovery, event replay, third-party SaaS integrations, more sophisticated filter expressions. SNS is still cheaper for simple high-throughput fan-out.

Q231. Step Functions — when to use

Step Functions execute **state machines** defined in JSON (Amazon States Language).

- **Standard workflows** — long-running (up to 1 year), exactly-once, pay per state transition. Use for sagas, business workflows, ETL.
- **Express workflows** — sub-second, at-least-once, pay per duration. Use for high-volume request processing, IoT.

Native integrations call Lambda, ECS, DynamoDB, SQS, etc. without you writing glue code. Built-in retry/catch/parallel/map. Visual editor + execution history makes orchestrated sagas observable.

Reach for it when your workflow has > 3 sequential steps, branching, or human-in-the-loop. Don't use it as a Lambda router for simple fan-out.

Q232. KMS, Secrets Manager, Parameter Store

- **KMS** — managed keys for encryption. You don't see the key bytes; you ask KMS to encrypt/decrypt or generate data keys (envelope encryption). Audited via CloudTrail. Multi-region keys for cross-region failover.
- **Secrets Manager** — secrets store with **automatic rotation** (built-in for RDS/Aurora/DocumentDB; custom Lambda for the rest). Cross-region replication. ~$0.40/secret/month.
- **Parameter Store (SSM)** — config + secrets store. **Standard** is free; **Advanced** is paid. No rotation. Versioning included. Good for config; OK for secrets if rotation isn't required.

Rule: secrets that need rotation → Secrets Manager; everything else → Parameter Store. Both integrate with ECS/Lambda env vars (`secrets`, not `environment`, so values are decrypted at startup and not exposed in task definitions).

Q233. IAM — identity vs resource policies

- **Identity policy** — attached to a principal (user/role/group). "What can this principal do?"
- **Resource policy** — attached to a resource (S3 bucket, KMS key, SQS queue). "Who can do what to this resource?"

Both can grant access. The effective permission = **union** for same-account access; **both required** for cross-account access (the resource policy must explicitly allow the foreign principal).

Best practices:

- **Roles, not users.** No long-lived access keys. Use OIDC federation (e.g., GitHub Actions → AWS) for CI.
- **Least privilege** — start narrow, widen on AccessDenied.
- **Permission boundaries** — cap the maximum permissions of any role created within a team.
- **SCPs** (Service Control Policies, at AWS Organizations) — guardrails the account can't escape (deny all unencrypted S3 PutObjects, deny outside approved regions).
- **IAM Access Analyzer** — flags policies that allow unintended external access.

Q234. Multi-AZ vs multi-region

Failure	Multi-AZ in 1 region	Multi-region
AZ outage	✅ ~30s failover	✅
Region outage	❌	✅
RPO	0 (sync replica)	seconds (async)
Cost / complexity	Low	High

Multi-region needs a strategy:

- **Active/passive (warm standby)** — secondary region runs at low capacity; promote on failover. Route 53 health-checked failover. Easier; some downtime.
- **Active/active** — both regions serve traffic; data layer needs conflict resolution (DynamoDB Global Tables, Aurora Global, CRDTs). Hardest.
- **Pilot light** — only critical infra running in standby; rest brought up on demand. Cheapest, highest RTO.

Most companies don't actually need multi-region. AZ failover handles the common case; regional outages are rare and your customers usually accept "AWS is down" with you. Multi-region is justified by **regulatory requirements**, **financial impact of regional outage**, or **global low-latency reads**.

Q235. Cost optimization levers

- **Compute** — Savings Plans (1y/3y commit, 30–70% off On-Demand) for steady workloads. **Spot** (60–90% off) for stateless / batch / replicated workloads. Graviton (ARM) saves another 20%.
- **Storage** — S3 Intelligent-Tiering (auto-moves cold data to cheaper tiers). Lifecycle policies to Glacier for backups. Delete incomplete multipart uploads (default-on lifecycle rule).
- **Data transfer** — the silent killer. Cross-AZ traffic is ~$0.01/GB; cross-region is ~$0.02/GB; egress to internet is $0.05–0.09/GB. Use VPC endpoints for S3/DynamoDB (free), CloudFront for egress (cheaper than direct), and **co-locate** services with their data.
- **NAT Gateway** — $33/AZ/month + per-GB processing. Use VPC endpoints for AWS calls; consider NAT instances for low traffic; or a single shared NAT in non-prod.
- **Idle resources** — unused EBS volumes, idle ELBs, old snapshots, unattached EIPs. Tag everything; run `aws-nuke` in non-prod.

- **Right-sizing** — Compute Optimizer recommends.
- **Reservations vs Savings Plans** — RIs are stricter (instance type/region locked); Savings Plans are flexible. Default to Compute Savings Plan unless you have a stable instance family.

Set a **monthly budget alert** in AWS Budgets the day you start an account.

Part XIV — Testing, DevOps, and Deployment

Chapter — Testing

14. Testing

Q119. Testing pyramid

- **Unit** (70%) — pure functions, small, fast, isolated.
- **Integration** (20%) — modules together (service + DB, controller + route).
- **E2E** (10%) — full system from an HTTP client.

Inversion (ice-cream cone) — too many slow E2E tests, too few units — is a common smell.

Q120. Jest basics + mocking

```
import { sum } from './sum';

jest.mock('./db');
const db = require('./db');

test('sums', () => {
  db.load.mockResolvedValue([1, 2, 3]);
  expect(sum([1,2])).toBe(3);
});
```

Prefer **dependency injection** over deep mocking — it keeps production code simple and tests honest.

Q121. Integration testing a REST API

```
import request from 'supertest';
import { createApp } from '../src/app';

it('POST /users creates a user', async () => {
  const res = await request(app).post('/users').send({ email: 'a@b.c' });
  expect(res.status).toBe(201);
  expect(res.body).toMatchObject({ email: 'a@b.c' });
});
```

Run DB tests against a real Postgres in Docker (`testcontainers-node`) — better than mocking the DB layer.

Q122. TDD — when is it worth it?

- When requirements are clear and well-defined.
- When the component is pure/algorithmic.
- When refactoring legacy code (tests protect behavior).

Skip TDD for throwaway scripts, UI experiments, or when you're still exploring the design.

Q123. Flaky tests — common causes and fixes

- Time-dependence — mock `Date` , use fake timers.
- Network calls — mock HTTP (`nock` , `msw`).
- Order-dependence — clean state between tests.
- Race conditions in async — await everything, use deterministic scheduling.

Chapter — DevOps, Docker, and Deployment

15. DevOps, Docker & Deployment

Q124. Dockerfile for a Node service — what does a good one look like?

```
# Use LTS alpine for smaller images
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json .
USER node                       # never run as root
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Key points: **multi-stage** (smaller runtime image), `npm ci` (reproducible), **non-root user**, **no dev deps in runtime**.

Q125. CI/CD pipeline — typical stages

1. Install + cache deps
2. Lint + typecheck
3. Unit tests

4. Build Docker image
5. Integration tests against real services (via Testcontainers / compose)
6. Security scan (Snyk, Trivy)
7. Push image
8. Deploy to staging, run smoke/E2E
9. Manual approval → deploy to prod (blue/green or canary)

Q126. Zero-downtime deployment strategies

- **Rolling** — replace pods/instances N at a time.
- **Blue/green** — deploy new stack; flip LB.
- **Canary** — route X% of traffic to new version; ramp up.

Make DB migrations **backward-compatible** to survive the window where old and new code both run.

Q127. Logging in production

- Structured JSON logs (pino).
- Correlation ID per request (propagate across services).
- Log levels by env (info in prod, debug in dev).
- Ship logs to a central store (CloudWatch, Loki, Datadog, ELK).
- Never log PII, secrets, full request bodies.

Q128. Metrics — the 4 golden signals (Google SRE)

1. **Latency** — request duration (p50, p95, p99).
2. **Traffic** — requests/sec.
3. **Errors** — error rate.
4. **Saturation** — how "full" your system is (CPU, queue depth).

Export via Prometheus (`prom-client`), visualize in Grafana.

Q129. Tracing with OpenTelemetry

OTel gives vendor-neutral distributed tracing. Auto-instrumentation covers HTTP, Express, Nest, Postgres, Mongo, Redis. Propagate `traceparent` header across services to stitch spans.

Part XV — Interview Practice and Career

Chapter — Coding Challenges

16. Coding Challenges

Q130. Implement `once(fn)` — run a function only once

```
function once(fn) {
  let called = false, result;
  return (...args) => {
    if (!called) { called = true; result = fn(...args); }
    return result;
  };
}
```

Q131. Implement `memoize(fn)`

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

Q132. Implement a simple EventEmitter

```
class MyEmitter {
  #listeners = new Map();
  on(event, fn)   { (this.#listeners.get(event) ?? this.#listeners.set(event, [
  off(event, fn)  { const ls = this.#listeners.get(event); if (ls) this.#listen
  emit(event, ...args) { (this.#listeners.get(event) ?? []).slice().forEach(fn
  once(event, fn) { const wrap = (...a) => { this.off(event, wrap); fn(...a); }
}
```

Q133. Retry with exponential backoff

```
async function retry(fn, { attempts = 5, base = 100, factor = 2, jitter = true
  let attempt = 0;
  while (true) {
    try { return await fn(); }
    catch (e) {
      if (++attempt >= attempts) throw e;
      const delay = base * factor ** (attempt - 1) * (jitter ? (0.5 + Math.rand
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

Q134. Deep clone

```
// Modern, handles cycles, Maps, Sets, Dates, ArrayBuffers
const copy = structuredClone(obj);
```

Interview-style manual version:

```
function deepClone(o, seen = new WeakMap()) {
  if (o === null || typeof o !== 'object') return o;
  if (seen.has(o)) return seen.get(o);
  const out = Array.isArray(o) ? [] : {};
  seen.set(o, out);
  for (const k of Reflect.ownKeys(o)) out[k] = deepClone(o[k], seen);
  return out;
}
```

Q135. Flatten a nested array

```
// Built-in
arr.flat(Infinity);

// Manual
const flatten = (a) => a.reduce((acc, v) => acc.concat(Array.isArray(v) ? flatt
```

Q136. Implement `pipe(...fns)` and `compose(...fns)`

```
const pipe    = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);

const toUpperTrim = pipe((s) => s.trim(), (s) => s.toUpperCase());
toUpperTrim('  hi  '); // "HI"
```

Q137. Fix an async `forEach` that doesn't wait

```
// ❌ runs in parallel, nothing awaited
arr.forEach(async (x) => await doWork(x));

// ✅ sequential
for (const x of arr) await doWork(x);

// ✅ parallel
await Promise.all(arr.map(doWork));

// ✅ parallel with concurrency limit
import pLimit from 'p-limit';
const limit = pLimit(5);
await Promise.all(arr.map(x => limit(() => doWork(x))));
```

Q138. Throttle concurrent API calls while preserving order of results

```
async function mapConcurrent(items, fn, concurrency = 5) {
  const results = new Array(items.length);
  let i = 0;
  const workers = Array.from({ length: concurrency }, async () => {
    while (i < items.length) {
      const idx = i++;
      results[idx] = await fn(items[idx], idx);
    }
  });
  await Promise.all(workers);
  return results;
}
```

Q139. Build a simple rate limiter (token bucket)

```
class TokenBucket {
  constructor(capacity, refillPerSec) {
    this.capacity = capacity;
    this.tokens = capacity;
    this.refillPerSec = refillPerSec;
    this.last = Date.now();
  }
  take() {
    const now = Date.now();
    this.tokens = Math.min(this.capacity, this.tokens + ((now - this.last) / 10
    this.last = now;
    if (this.tokens >= 1) { this.tokens -= 1; return true; }
    return false;
  }
}
```

Q140. LRU Cache

```
class LRU {
  constructor(max) { this.max = max; this.map = new Map(); }
  get(k) {
    if (!this.map.has(k)) return undefined;
    const v = this.map.get(k);
    this.map.delete(k); this.map.set(k, v); // move to tail
    return v;
  }
  set(k, v) {
    if (this.map.has(k)) this.map.delete(k);
    else if (this.map.size >= this.max) this.map.delete(this.map.keys().next().
    this.map.set(k, v);
  }
}
```

Q141. Big-O — know the basics

- Array push/pop: O(1). unshift/shift: O(n).
- `Map` / `Set` : O(1) avg for get/set/delete.

- Object property access: O(1) avg.
- Sort: O(n log n).
- Searching sorted array with binary search: O(log n).
- Graph BFS/DFS: O(V + E).

Chapter — Behavioral / Senior-Level Questions

17. Behavioral / Senior-Level Questions

Q142. "Tell me about a time you had a serious production incident."

Use **STAR** (Situation, Task, Action, Result). A good story includes:

- What signal tipped you off (alerts, user reports).
- How you triaged (rollback vs. forward-fix).
- What root cause analysis revealed.
- Postmortem actions (blameless, systemic fixes, not "be more careful").

Q143. "How do you mentor juniors?"

Things to mention:

- Pair programming.
- PR reviews that explain *why*, not just *what*.
- Giving them safe, well-scoped tasks before ambitious ones.
- Encouraging them to propose solutions, then guiding.
- Dev2dev talks, knowledge sharing.

Q144. "How do you make technical decisions in a team?"

- Write an **ADR** (Architecture Decision Record) or design doc.
- Present options with tradeoffs (latency, cost, complexity).
- Engage stakeholders early; disagree-and-commit when needed.
- Measure outcomes; revisit if data says otherwise.

Q145. "How do you balance speed vs quality?"

- Tests for the **critical path**; less coverage in experimental areas.
- Feature flags for risky changes.
- Fast rollback paths.
- Tech-debt budget in sprints, not "someday".

Q146. "A colleague insists on a bad technical decision. What do you do?"

- Ask questions first — make sure I understand their reasoning.
- Present concrete counter-evidence (benchmarks, postmortems, industry practice).
- If still deadlocked, escalate the decision owner (tech lead, architect).
- Once decided, support the decision even if I disagreed — but log the risk.

Q147. "Why are you leaving your current job?"

Be honest but positive. Don't bash your employer. Frame in terms of what you're **looking for** (larger scale, more ownership, new domain).

Q148. Questions to ask the interviewer

- How is the engineering team organized, and what does decision-making look like?
- What does the on-call rotation look like?
- How do you balance product work vs. tech debt?
- What's the biggest technical challenge the team is facing right now?
- What does success look like in this role after 3 / 6 / 12 months?
- How is feedback given (1:1s, review cycles)?
- What's the deployment frequency, and who owns production?

Appendix — Core Interview Questions Reference

The book's Parts I–IV teach the language and runtime. These appendices are the same material restated as short-answer interview questions — useful the day before an on-site.

Appendix A — Node.js Core & Internals (Q&A)

1. Node.js Core & Internals

Q1. What is Node.js and how does it work under the hood?

Node.js is a JavaScript runtime built on Chrome's **V8 engine**. It uses a **single-threaded, non-blocking, event-driven** architecture powered by **libuv** (a C library that implements the event loop and thread pool).

Key components:

- **V8** — compiles JS to machine code
- **libuv** — provides event loop + thread pool (default 4 threads) for async I/O
- **Node bindings** — C++ bridge between JS and system calls
- **Core modules** — `fs`, `http`, `net`, etc.

When you call `fs.readFile`, V8 hands the work off to libuv's thread pool, and your JS keeps running. When the file is ready, libuv pushes a callback to the event loop queue.

Q2. Explain the Event Loop in detail

The event loop has **6 phases**, executed in order:

```
   ┌───────────────────────────┐
┌─>│           timers          │  setTimeout, setInterval callbacks
│  └─────────────┬─────────────┘
│  ┌─────────────┴─────────────┐
│  │     pending callbacks     │  deferred I/O callbacks (e.g. TCP errors)
│  └─────────────┬─────────────┘
│  ┌─────────────┴─────────────┐
│  │       idle, prepare       │  internal use
│  └─────────────┬─────────────┘
│  ┌─────────────┴─────────────┐      ┌─────────────────┐
│  │           poll            │<─────┤   incoming:     │
│  │                           │      │  connections,   │
│  └─────────────┬─────────────┘      │   data, etc.    │
│  ┌─────────────┴─────────────┐      └─────────────────┘
│  │           check           │  setImmediate callbacks
│  └─────────────┬─────────────┘
│  ┌─────────────┴─────────────┐
└──│      close callbacks      │  e.g. socket.on('close')
   └───────────────────────────┘
```

Between **each phase**, Node drains the **microtask queue**: `process.nextTick` callbacks first, then resolved Promise callbacks.

Q3. `process.nextTick` vs `setImmediate` vs `setTimeout(fn, 0)`

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

console.log('sync');

// Output:
// sync
// nextTick
// promise
// timeout     (order with immediate is non-deterministic outside I/O)
// immediate
```

- `process.nextTick` — fires **before** any other I/O event, highest priority. Can starve the event loop if abused.
- **Promises (microtasks)** — drained right after `nextTick` queue.
- `setTimeout(fn, 0)` — runs in the **timers** phase; minimum delay is actually ~1ms.
- `setImmediate` — runs in the **check** phase, right after poll. Use this when you want to yield to I/O first.

Rule of thumb: inside an I/O callback, `setImmediate` always fires before `setTimeout(fn, 0)`.

Q4. Is Node.js single-threaded?

Partially. Your JavaScript runs on a single main thread, but Node.js as a whole is multi-threaded:

- **libuv thread pool** (default 4 threads, configurable via `UV_THREADPOOL_SIZE`) handles: file I/O, DNS lookups (`dns.lookup`), CPU-heavy crypto, zlib.
- **Network I/O** uses the OS's async primitives (epoll/kqueue/IOCP) — no thread pool needed.
- **Worker threads** let you run JS on multiple threads.
- **V8 itself** uses several threads internally (GC, compilation).

Q5. `worker_threads` vs `cluster` vs `child_process`

Feature	`child_process`	`cluster`	`worker_threads`
Memory	Separate process	Separate processes	Shared memory possible
Use case	Spawning external bins	Scaling HTTP servers across CPUs	CPU-heavy JS work
Communication	IPC (JSON messages)	IPC	MessageChannel, SharedArrayBuffer
Overhead	High (full process)	High	Lower

```
// worker_threads — CPU-bound work
const { Worker, isMainThread, parentPort } = require('worker_threads');

if (isMainThread) {
  const worker = new Worker(__filename);
  worker.on('message', msg => console.log('Result:', msg));
  worker.postMessage(1_000_000);
} else {
  parentPort.on('message', (n) => {
    let sum = 0;
    for (let i = 0; i < n; i++) sum += i;
    parentPort.postMessage(sum);
  });
}
```

```
// cluster — fork one worker per CPU for HTTP scaling
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  os.cpus().forEach(() => cluster.fork());
  cluster.on('exit', () => cluster.fork()); // restart on crash
} else {
  require('http').createServer((req, res) => res.end('hi')).listen(3000);
}
```

Q6. Streams — what are they and why use them?

A **stream** is an abstraction for processing data piece-by-piece instead of loading it all into memory. Everything that implements the stream interface works with `.pipe()`.

Four types:

- **Readable** — `fs.createReadStream`, `http.IncomingMessage`
- **Writable** — `fs.createWriteStream`, `http.ServerResponse`
- **Duplex** — both (e.g. TCP sockets)
- **Transform** — Duplex where output is derived from input (e.g. `zlib.createGzip`)

```
// Classic streaming: file -> gzip -> file, using only a small buffer in RAM
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream/promises');

await pipeline(
  fs.createReadStream('input.txt'),
  zlib.createGzip(),
  fs.createWriteStream('input.txt.gz')
);
```

Always prefer `pipeline` over `.pipe()` — it handles errors and cleanup. `.pipe()` leaks on error.

Q7. What is backpressure?

When a writable stream can't consume data as fast as it's produced, memory fills up. Streams handle this by making `.write()` return `false` — you should pause the reader until `'drain'` fires. `pipe()` and `pipeline()` do this automatically.

```
readable.on('data', (chunk) => {
  const ok = writable.write(chunk);
  if (!ok) {
    readable.pause();
    writable.once('drain', () => readable.resume());
  }
});
```

Q8. EventEmitter — how does it work?

`EventEmitter` is a simple pub/sub primitive. Most Node built-ins extend it (streams, HTTP, process).

```
const { EventEmitter } = require('events');

class Order extends EventEmitter {}
const order = new Order();

order.on('created', (id) => console.log('Order', id));
order.emit('created', 42);

// Memory leak check — default max is 10 listeners per event
order.setMaxListeners(20);
```

Gotchas:

- Listeners run **synchronously** in registration order.
- Throwing inside a listener kills the process (unless you handle `'error'`).
- Always handle `'error'` events on emitters — unhandled `'error'` crashes Node.

Q9. Buffer vs string, and when do you care?

A `Buffer` is a fixed-size chunk of memory outside V8's heap, useful for binary data (files, network, crypto). Strings in JS are UTF-16; converting blindly between them can corrupt binary data.

```
const buf = Buffer.from('hello', 'utf8');
buf.toString('base64'); // "aGVsbG8="
Buffer.alloc(10);       // zero-filled (safe)
Buffer.allocUnsafe(10); // faster but may contain old memory — never send to cl
```

Q10. CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Syntax	`require` / `module.exports`	`import` / `export`
Loading	Synchronous	Asynchronous (static)
Tree-shaking	No	Yes
`__dirname`	Yes	No — use `import.meta.url`
Top-level await	No	Yes
Circular deps	Returns partial exports	Handled via live bindings

```
// ESM in Node — use "type": "module" in package.json, or .mjs
import { readFile } from 'node:fs/promises';
const data = await readFile('a.txt', 'utf8'); // top-level await
```

Q11. How does `require` work?

When you call `require('x')`:

1. **Resolve** the path (core module? node_modules? relative?)
2. **Load** the file (read from disk)
3. **Wrap** it in a function: `(function (exports, require, module, __filename, __dirname) { ... })`
4. **Evaluate** the wrapped function
5. **Cache** the result in `require.cache` by absolute path

The cache means subsequent `require` calls return the **same object** — that's why modules are singletons by default.

Q12. Memory management and garbage collection

V8 uses **generational GC**:

- **New space (young generation)** — small (1–8 MB), collected with fast **Scavenge** (copying collector). Most objects die here.

- **Old space (old generation)** — grown objects go here, collected with **Mark-Sweep-Compact** (stop-the-world pauses).

Flags you'll see in production:

- `--max-old-space-size=4096` — increase heap to 4 GB
- `--expose-gc` — enables manual `global.gc()` for profiling

Q13. Common causes of memory leaks in Node.js

1. **Unbounded caches** (Map/object growing forever) — use `lru-cache`.
2. **Lingering event listeners** — forgetting `removeListener`, leading to `MaxListenersExceededWarning`.
3. **Global variables** accidentally holding references.
4. **Closures** capturing large objects.
5. **Timers** (`setInterval`) never cleared.
6. **Promises that never settle** — retained forever.

Debugging: take heap snapshots with `node --inspect` and Chrome DevTools, or use `clinic.js`, `heapdump`.

Q14. What is `process` in Node.js?

A global object representing the current Node process.

```
process.env.NODE_ENV;          // env vars
process.argv;                  // CLI args
process.exit(1);               // exit with code
process.on('uncaughtException', handler);
process.on('unhandledRejection', handler);
process.memoryUsage();         // heap info
process.cwd();                 // current working dir
process.pid;                   // process id
```

Never `process.exit` inside a request handler — it kills in-flight requests. Drain gracefully instead.

Q15. Graceful shutdown — how do you implement it?

```
const server = app.listen(3000);

const shutdown = async (signal) => {
  console.log(`Received ${signal}, shutting down...`);
  server.close(async () => {        // stop accepting new connections
    await db.close();               // close DB pool
    await redis.quit();
    process.exit(0);
  });
  // Force-kill after 10s
  setTimeout(() => process.exit(1), 10_000).unref();
};

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT',  () => shutdown('SIGINT'));
```

Appendix B — JavaScript & TypeScript (Q&A)

2. JavaScript & TypeScript Deep Dive

Q16. Explain closures

A closure is a function that "remembers" variables from the lexical scope where it was created, even after that scope exits.

```
function counter() {
  let count = 0;                      // captured by the returned function
  return () => ++count;
}
const c = counter();
c(); c(); c(); // 3
```

Practical use: private state, memoization, currying, event handlers with bound data.

Q17. `this` binding in JavaScript

Rules, in order of precedence:

1. **`new` binding** — `new Foo()` → `this` is the new instance.
2. **Explicit** — `.call(obj)`, `.apply(obj)`, `.bind(obj)`.
3. **Implicit** — `obj.method()` → `this` is `obj`.
4. **Default** — plain call → `undefined` (strict) or `globalThis`.
5. **Arrow functions** don't have their own `this` — they inherit from the enclosing lexical scope.

```
class Api {
  constructor() { this.base = '/api'; }
  // arrow method keeps `this` when passed as callback
  get = (path) => fetch(this.base + path);
}
```

Q18. Prototypes and prototypal inheritance

Every object has a hidden `[[Prototype]]` link. Property lookup walks this chain until `null`.

```
const animal = { eats: true };
const dog = Object.create(animal);
dog.barks = true;

dog.eats;  // true, found on prototype
Object.getPrototypeOf(dog) === animal; // true
```

ES6 `class` is syntactic sugar over prototypes.

Q19. `var` vs `let` vs `const`

Feature	`var`	`let`	`const`
Scope	Function	Block	Block
Hoisting	Yes, initialized as `undefined`	Yes, but in temporal dead zone	Same as `let`
Reassignment	Yes	Yes	No
Global-as-property	Yes (`window.x`)	No	No

Q20. `==` vs `===`

`==` coerces types; `===` does not. Always use `===` unless you have a specific reason. One notable case: `== null` matches both `null` and `undefined`, sometimes useful.

Q21. Explain hoisting

Variable/function declarations are "moved" to the top of their scope at compile time.

- `function` declarations: fully hoisted (callable before definition).
- `var` : hoisted and initialized as `undefined` .
- `let` / `const` / `class` : hoisted but uninitialized (Temporal Dead Zone).

Q22. What are Symbols?

A primitive type producing unique, immutable identifiers — useful for "hidden" property keys and well-known protocol hooks (`Symbol.iterator` , `Symbol.asyncIterator`).

```
const ID = Symbol('id');
const user = { name: 'Alice', [ID]: 123 };
Object.keys(user);           // ['name'] — Symbol is hidden
user[ID];                    // 123
```

Q23. Generators and iterators

Generators produce a sequence lazily via `yield` .

```
function* range(from, to) {
  for (let i = from; i <= to; i++) yield i;
}
for (const n of range(1, 5)) console.log(n);

// Async generators power async iteration
async function* paginate(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield* res.items;
    next = res.nextPage;
  }
}

for await (const item of paginate('/api/users')) { ... }
```

Q24. Debounce vs throttle

- **Debounce** — wait until user stops firing for N ms, then run once. (e.g., search-as-you-type.)
- **Throttle** — run at most once every N ms. (e.g., scroll handler.)

```
const debounce = (fn, ms) => {
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms);
};

const throttle = (fn, ms) => {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
};
```

Q25. `call`, `apply`, `bind`

```
function greet(greeting, punct) { return `${greeting}, ${this.name}${punct}`; }
const user = { name: 'Alice' };
greet.call(user, 'Hi', '!');   // "Hi, Alice!"
greet.apply(user, ['Hi', '!']);// same, args as array
const bound = greet.bind(user, 'Hi');
bound('.');                     // "Hi, Alice."
```

Q26. TypeScript — generics

```
function wrapInArray<T>(value: T): T[] { return [value]; }
wrapInArray(5);       // T inferred as number
wrapInArray('hi');    // T inferred as string

// Generic constraints
function byId<T extends { id: string }>(items: T[], id: string): T | undefined
  return items.find(i => i.id === id);
}
```

Q27. TypeScript — utility types

- `Partial<T>` — all fields optional
- `Required<T>` — all fields required
- `Pick<T, K>` / `Omit<T, K>` — select/exclude fields
- `Readonly<T>` — immutable
- `Record<K, V>` — map type
- `ReturnType<T>` / `Parameters<T>` — infer from function type
- `Awaited<T>` — unwrap a Promise

```
type User = { id: string; name: string; email: string };
type UserUpdate = Partial<Omit<User, 'id'>>; // id immutable, others optional
```

Q28. `unknown` vs `any` vs `never`

- `any` — opt out of type checking (avoid).
- `unknown` — "something, but narrow before use" (safer `any`).

- `never` — impossible value; return type of functions that throw or loop forever.

Final Tips

Final Tips

1. **When you don't know something, say so** — then reason out loud. Seniors aren't expected to know everything; they're expected to think clearly and know their limits.
2. **Think in tradeoffs, not absolutes.** "It depends, because..." is a senior answer. "X is always better" is not.
3. **Mention failure modes.** When you propose a solution, volunteer how it could break and how you'd detect/mitigate it.
4. **Use real numbers.** "p95 latency went from 450ms to 80ms after we added the Redis cache" lands far better than "it got faster".
5. **Drive the conversation.** Ask clarifying questions at the start of system design problems — requirements, scale, SLAs — before diving in.
6. **Practice out loud.** Explaining concepts verbally is a different skill from understanding them.
7. **Prepare 3–5 stories** you can adapt to behavioral questions (incident, conflict, leadership, tradeoff, learning).

Good luck — you've got this.